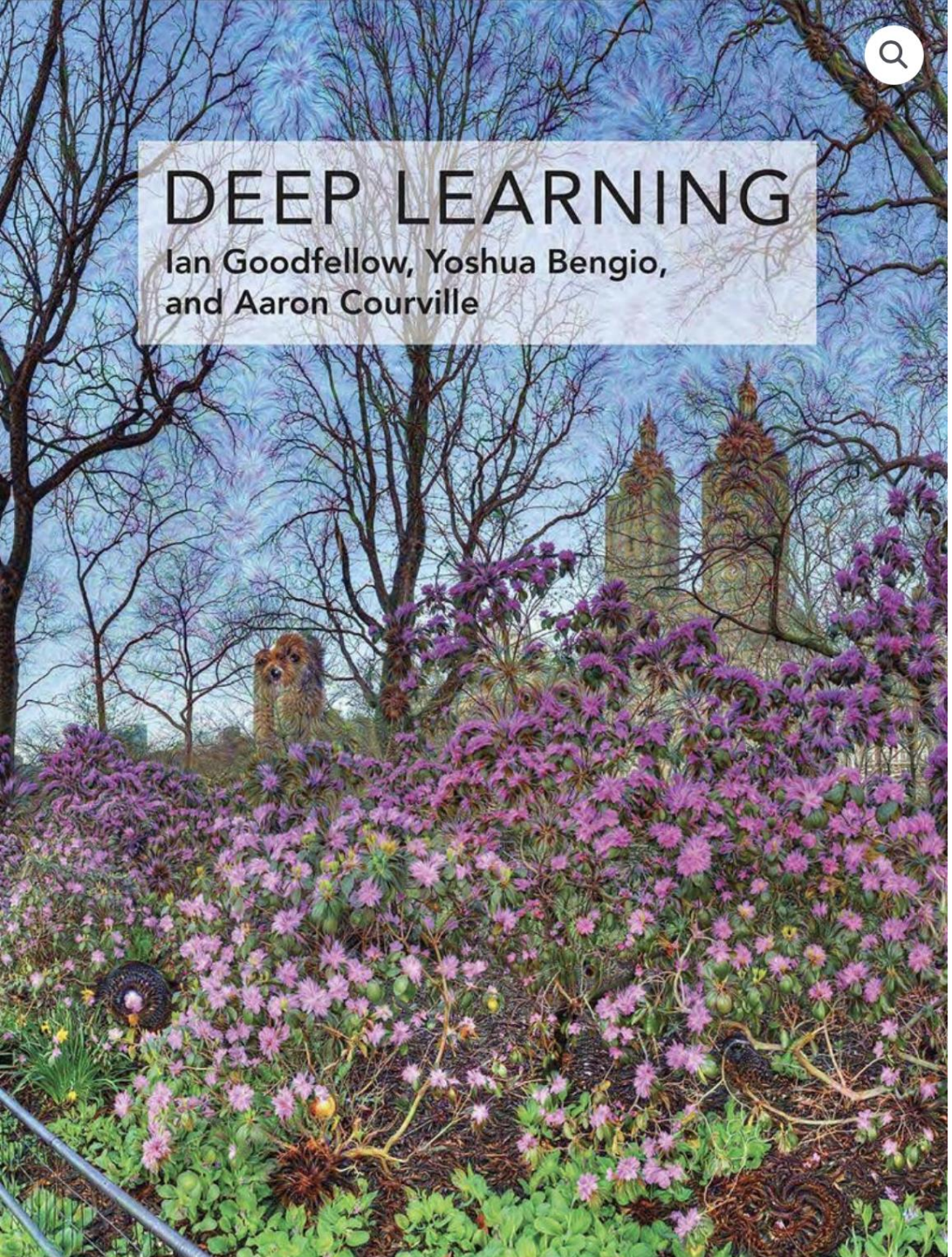
Deep Learning Systems 2024:
Lecture 4 – Brains to Backprop Part 1



Crescat scientia; vita excolatur

Rick Stevens
Argonne National Laboratory
The University of Chicago



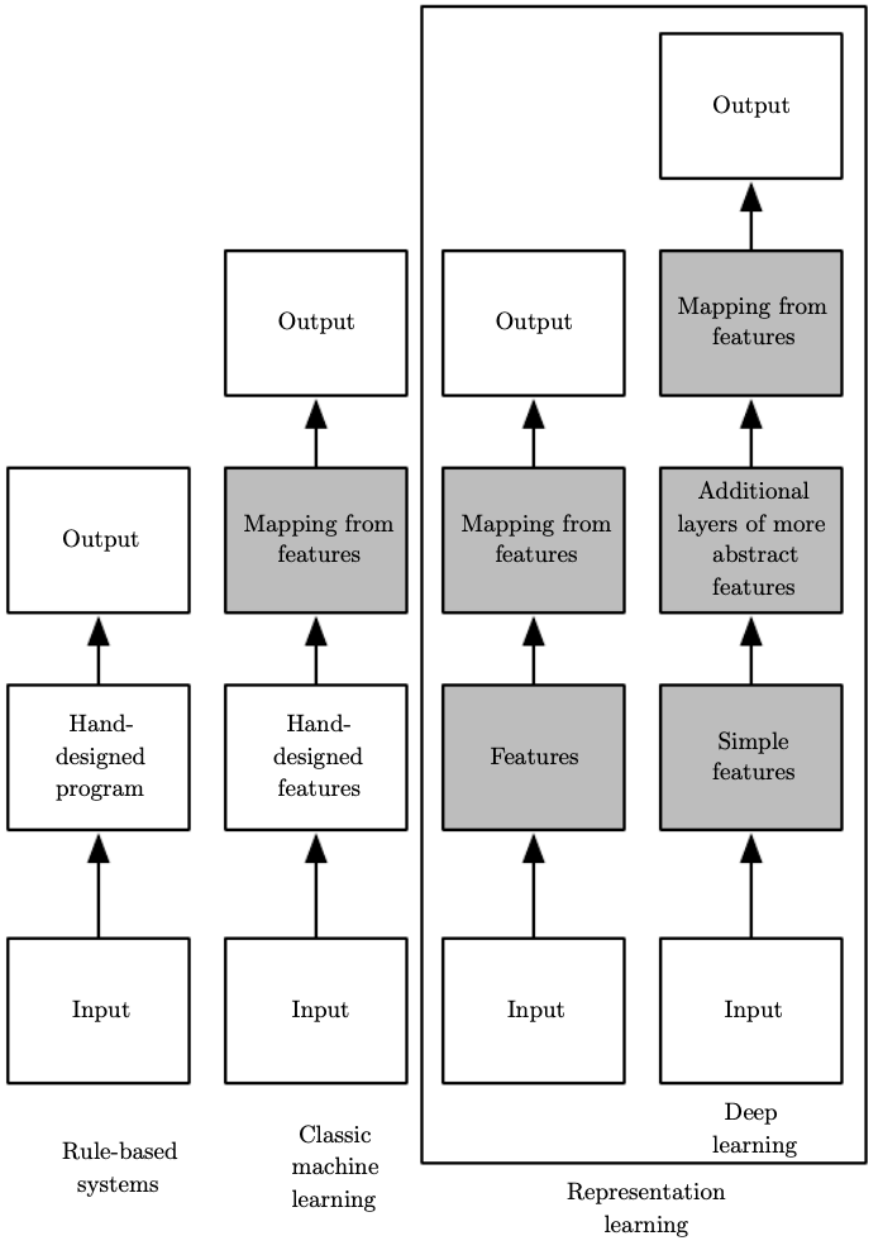


Figure 1.5: Flowcharts showing how the different parts of an AI system relate to each other within different AI disciplines. Shaded boxes indicate components that are able to learn from data.

Connections per Neuron

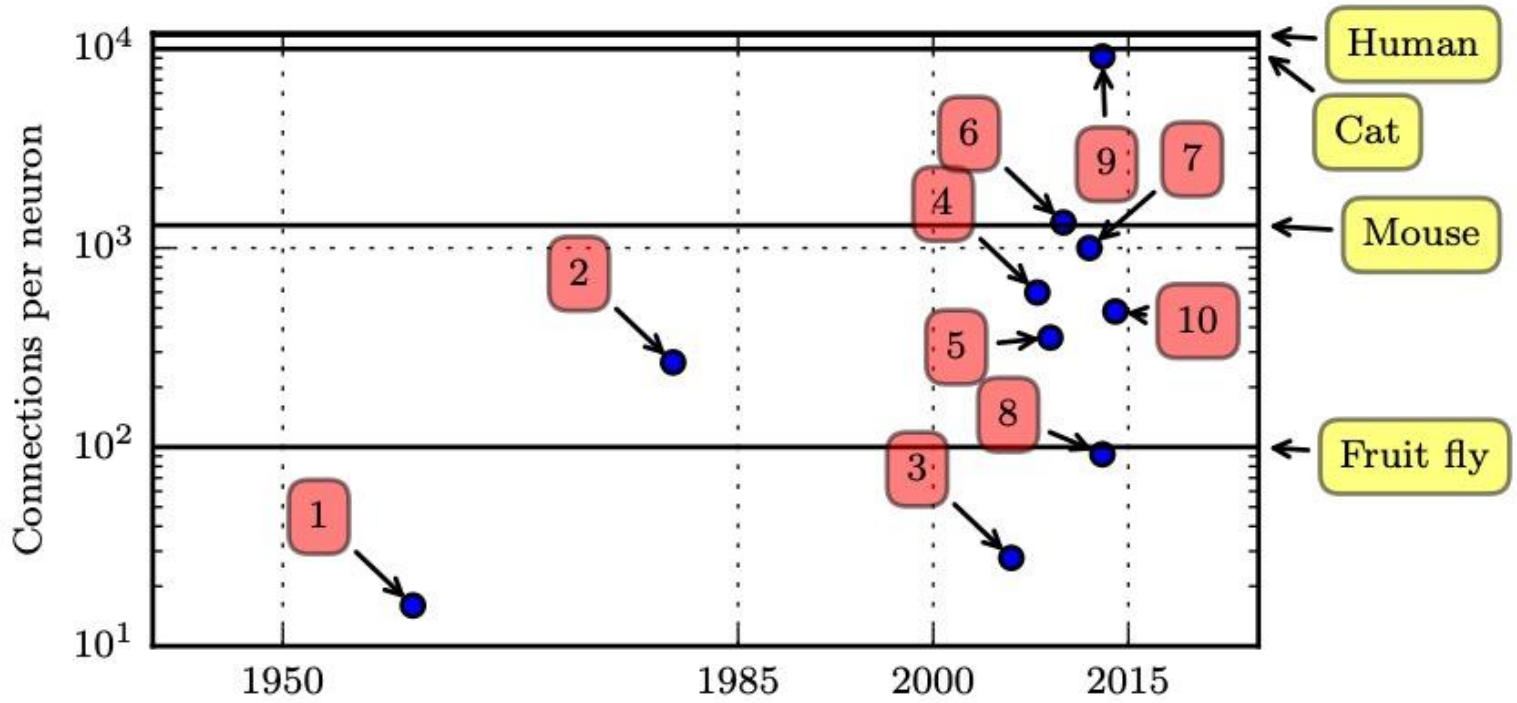


Figure 1.10

Number of Neurons

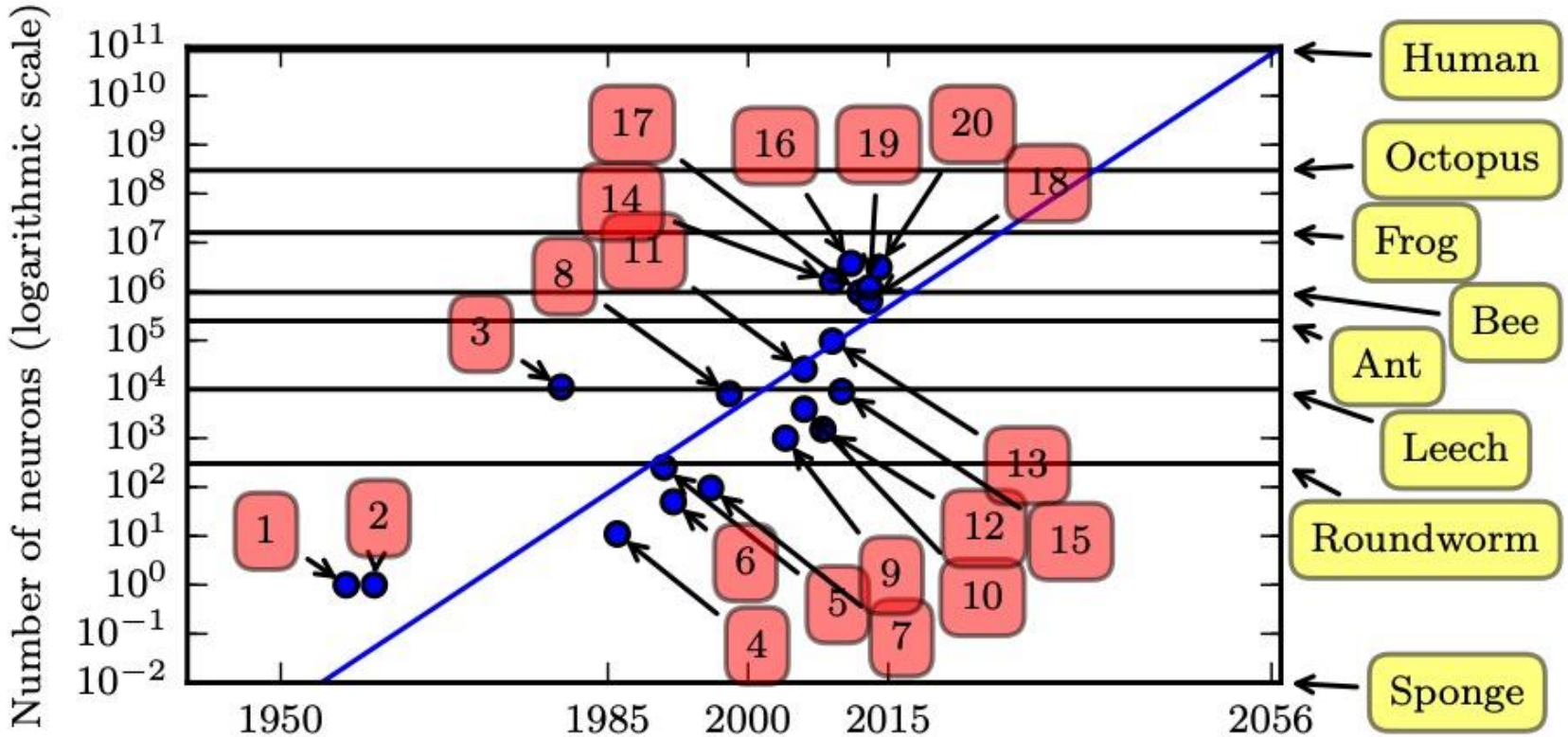


Figure 1.11

(Goodfellow 2016)

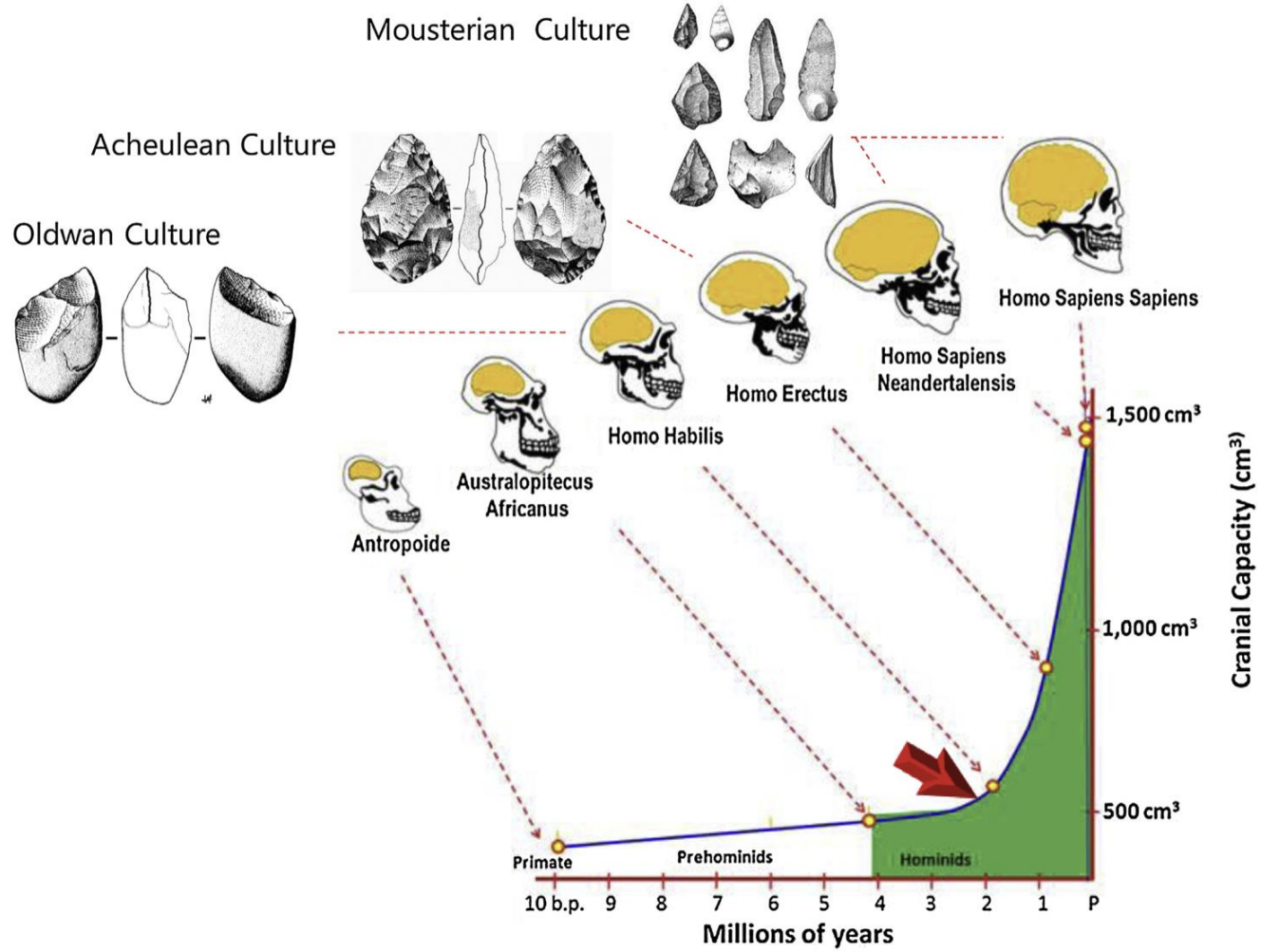
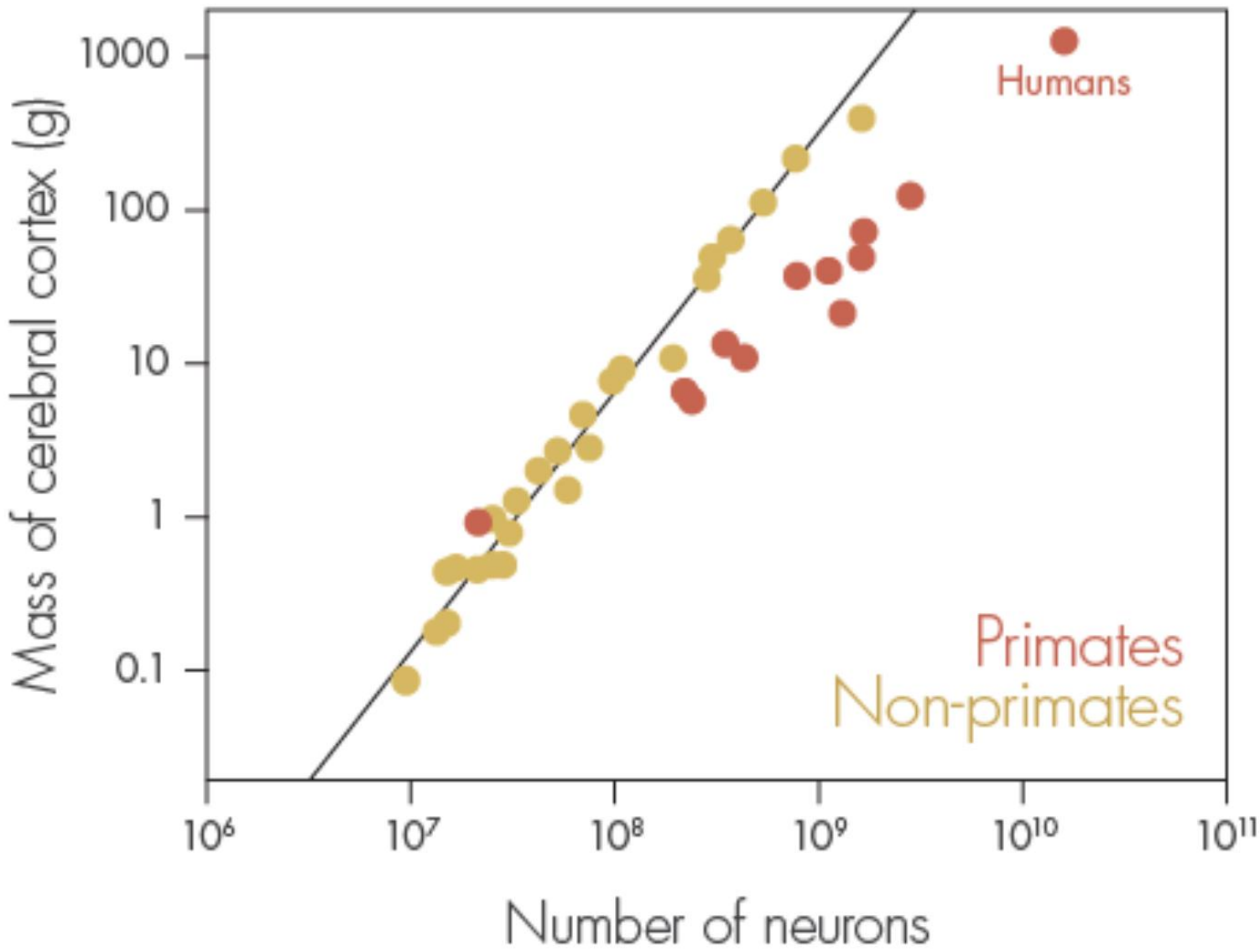


Fig. 1. Transition of cranial capacity (ordinate) along the evolutionary process of various ancestral prehominids and hominids (abscissa) (adapted from "Evolution du volume cérébral des Hominidés, " n.d.). Inset illustrations of a skull and brain depict representative hominids plotted on the graph. Brain expansion suddenly accelerated when Homo Habilis started manufacturing and using stone tools (oblique red arrow). Insets of stone tools (top left) depict the development level of stone tools in each culture. Oldwan culture: The earliest widespread stone tools were simple, usually made with one or a few flakes chipped off with another stone, used by Homo Habilis. Acheulean culture: Stone tools characterized by distinctive oval and pear-shaped "hand-axes," manufactured and used by Homo Erectus. Simple syntax in vocal communication, a primitive form of human language, is thought to have been required for the transmission of this culture. Mousterian culture: Technically complex and significant archaeological industry of lithic tools developed from Homo Sapiens Neanderthalensis throughout early Homo Sapiens Sapiens. Complex and rich semantic contents and syntax, close to modern human language, were necessary for the transmission and inheritance of this culture.

BRAIN DENSITY



Brains and Models

1. Structural lens: synapses and weights

A simple model assigns one independent scalar weight to each synapse.

$P \approx S = N \times k$

N = neurons in the chosen region k = mean incoming synapses per neuron

Regional scope. Use N and k from the same region. For a whole-brain estimate, sum N × k across regions and count each synapse once.

Human example. About 86 billion neurons in the brain, about 16 billion in the cerebral cortex. One neocortical study estimated 1.64×10^{14} synapses. 164 Trillion

Limit. This approximation counts connections. Weight sharing, synaptic dynamics and neuron properties change the relationship to model parameters.

Azevedo et al., 2009; Tang et al., 2001. Full citations in speaker notes.

2. Storage lens: bits per synapse and per weight

Evidence. Bartol et al. inferred at least 26 distinguishable strength levels (about 4.7 bits) in sampled rat hippocampal synapses.

This is a lower-bound estimate under that study's assumptions. Applying it to the human brain requires an explicit extrapolation.

Equivalent weight slots ≈ S × b / q

S = synapses b = assumed bits per synapse q = stored bits per weight

What it measures. A nominal storage comparison at a stated precision.

Limits. Stored bit width differs from learned information. Correlations and unequal state probabilities affect capacity. Other biological states add detail.

Bartol et al., 2015. Full citation and assumptions in speaker notes.

3. Graph lens: a description of the wiring

A simple edge list stores both endpoints and a strength for each connection.

Bits per edge ≈ $2\lceil \log_2 N \rceil + b$

With N = 86 billion neurons: 37 bits per endpoint, or 74 address bits.

If b = 4.7, the idealized total is 78.7 bits per edge.
That equals about 4.9 storage words of 16 bits each.

Encoding matters. Adjacency lists, spatial structure and compression can reduce this description length. It is a property of the chosen encoding.

Meaning. These are storage-word equivalents. Dividing by 16 does not turn connection addresses into additional learned parameters.

Address arithmetic is illustrative. Inputs: Azevedo et al., 2009; Bartol et al., 2015.

4. Dynamics lens: reuse of connections over time

Unrolling a recurrent model reuses the same weights at successive time steps.

Connection evaluations ≈ a × S × T

a = fraction of connections evaluated per step T = number of steps

Illustrative example. $S = 1.6 \times 10^{14}$, $T = 100$ and $a = 1$ give 1.6×10^{16} connection evaluations.

Unique weights. The model still has 1.6×10^{14} synaptic weights under the one-weight-per-synapse assumption.

Caveat. T and a depend on the model. A synaptic event has no universal conversion to FLOPs, and biological activity is asynchronous.

Recurrent weight reuse: PyTorch RNN documentation. Biological scale: Tang et al., 2001.

5. Regional lens: a brain area and a model stage

A regional comparison keeps the biological scope closer to the task.

Scope. Choose a region, such as primary visual cortex (V1),
and define which local and long-range inputs count.

Connections. Estimate $S_r = N_r \times k_r$ using compatible regional data.
An ANN with untied scalar weights can match that connection count.

Function. Test predictions of neural responses or behavior.
Matching counts alone does not establish comparable capability.

Biological detail. A neuron can have nonlinear dendrites and changing internal state. Feedback and plasticity affect the comparison.

Azevedo et al., 2009; Beniaguev et al., 2021; PyTorch Conv2d documentation.

Worked example: human neocortical scale

Use $S \approx 1.6 \times 10^{14}$ synapses, rounded from a neocortical study.
For the storage calculation only, assume b = 4.7 bits per synapse.

Comparison	Calculation	Result
Synaptic weights	$P \approx S$	160 trillion weights
16-bit equivalents	$S \times 4.7 / 16$	47 trillion slots
8-bit equivalents	$S \times 4.7 / 8$	94 trillion slots

The first row counts independent weights in an idealized model.
The other rows compare nominal storage under an extrapolated assumption.

Tang et al., 2001 (human neocortex); Bartol et al., 2015 (rat hippocampus).

Comparisons across mammals

1. Define the population. Use the species, brain region and age group relevant to the question. Record sources and uncertainty.

2. Count consistently. Use regional N and incoming k, or measured synapse density × regional volume. Sum regions only with compatible boundaries.

3. Choose the quantity. Report connections, storage bits, wiring description or connection evaluations. Each quantity has different assumptions.

4. Check sensitivity. Vary uncertain inputs. State weight sharing, precision and time horizon before interpreting the result.

For example, about 71 million neurons is a published whole-mouse-brain estimate. A cortical synapse-per-neuron value needs a cortical neuron count.

Herculano-Houzel et al., 2006. Full citation in speaker notes.

Summary: five ways to compare brains and models

Lens	Quantity	What it tells us
Structure	Synapses S	Count of connections
Storage	S × assumed bits per synapse	Nominal storage budget
Wiring	Address bits + strength bits	Description length
Dynamics	Connection evaluations	Work over a defined interval
Region	A brain area and model stage	A functional hypothesis to test

Parameter count, storage and computation answer different questions.
The biological scope and modeling assumptions determine the comparison.

Summary: the human numbers and their limits

≈86 billion neurons. A whole-brain estimate, with about 16 billion in the cerebral cortex. Regional differences matter.

≈160 trillion synapses. A rounded estimate for the human neocortex from one study, rather than a whole-brain total.

≈47 trillion 16-bit slots. A conditional storage equivalent if every synapse contributes 4.7 bits. That assumption extrapolates from rat hippocampus.

These estimates describe different quantities and populations. They do not establish a human-equivalent AI model size.

Azevedo et al., 2009; Tang et al., 2001; Bartol et al., 2015.

Summary: what makes a useful comparison

A clear question. The relevant brain region and task define the scope.

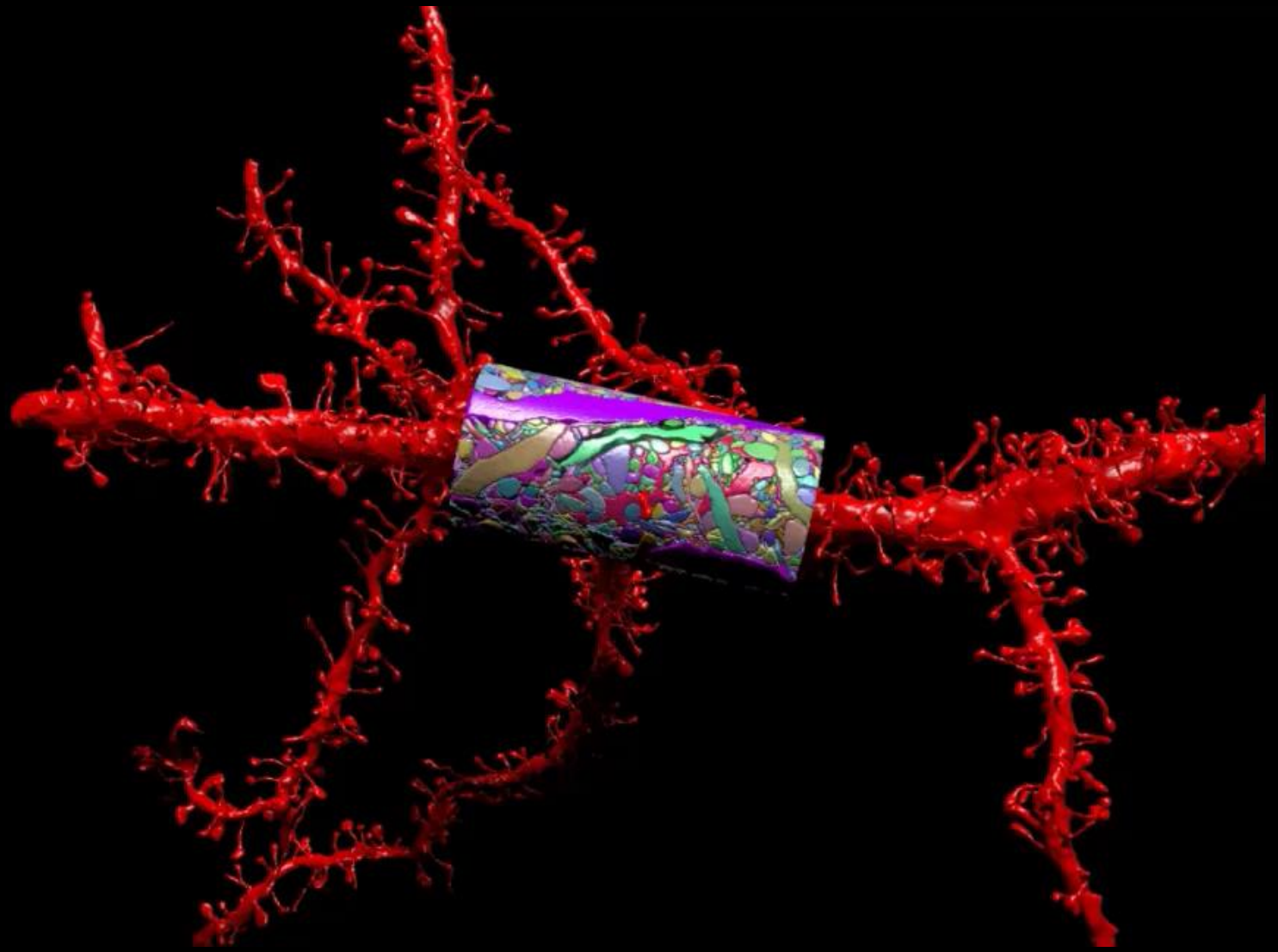
Explicit units. Connections, bits and operations remain separate quantities.

Visible assumptions. Precision, weight sharing and time determine how a biological count maps to a model.

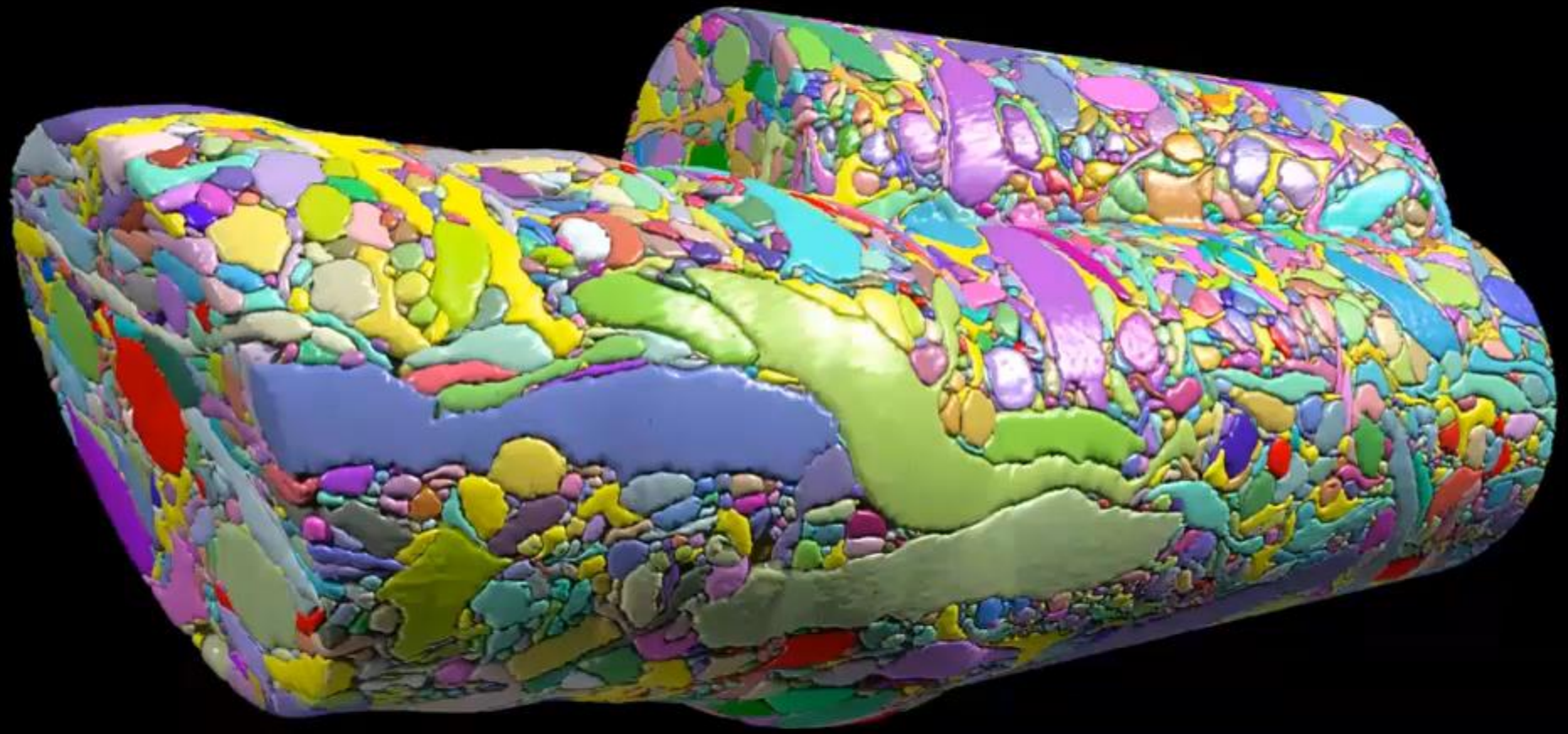
Functional evidence. Neural or behavioral tests are needed to evaluate what a model captures about the brain.

There is no single conversion from synapses to AI capability.

Progress on the (micro) Connectome

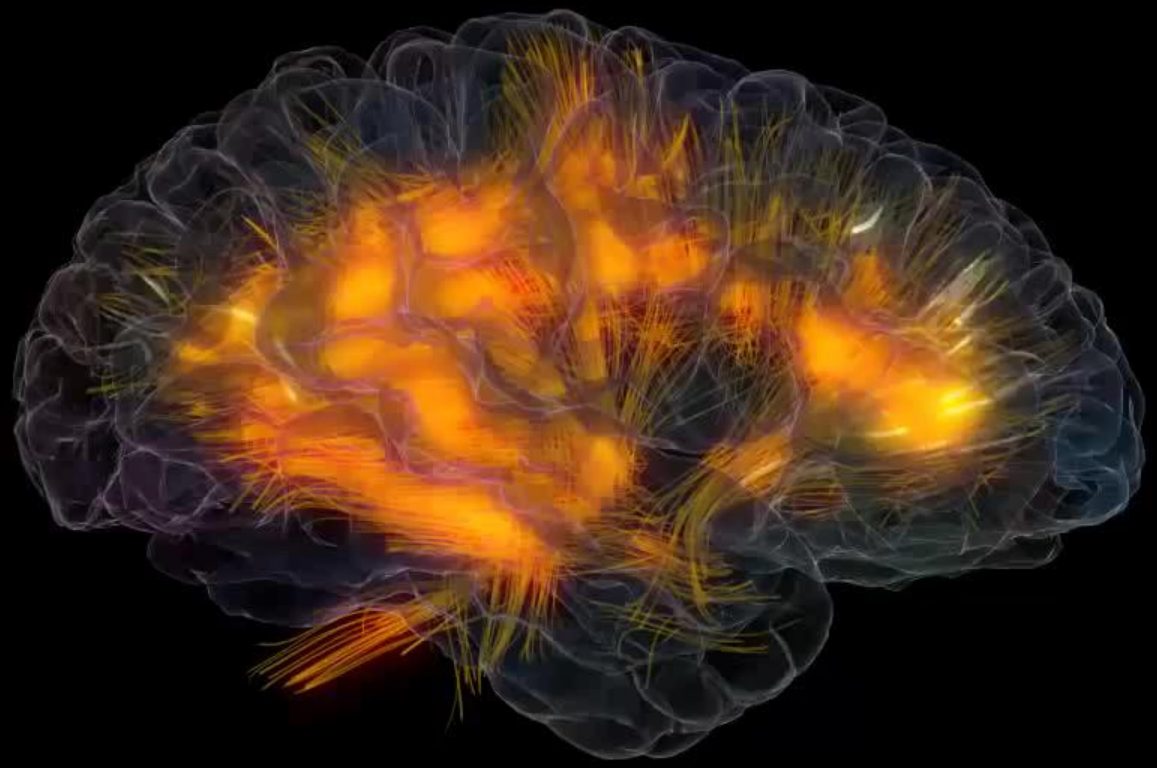


Bobby Kasthuri, et. al. Argonne, Uchicago and Harvard



Kasthuri et al., Cell 2015

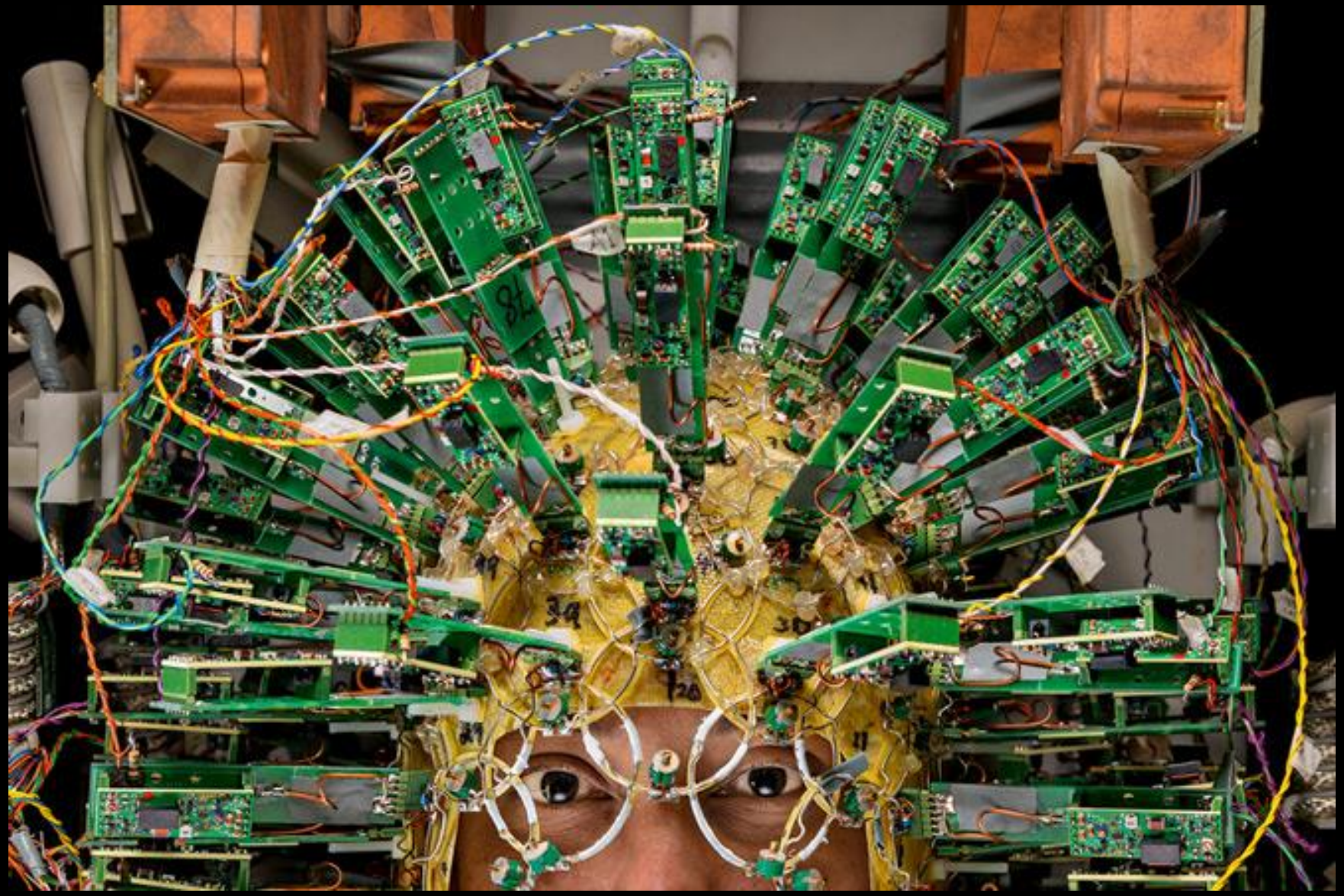
GlassBrain — Visualization



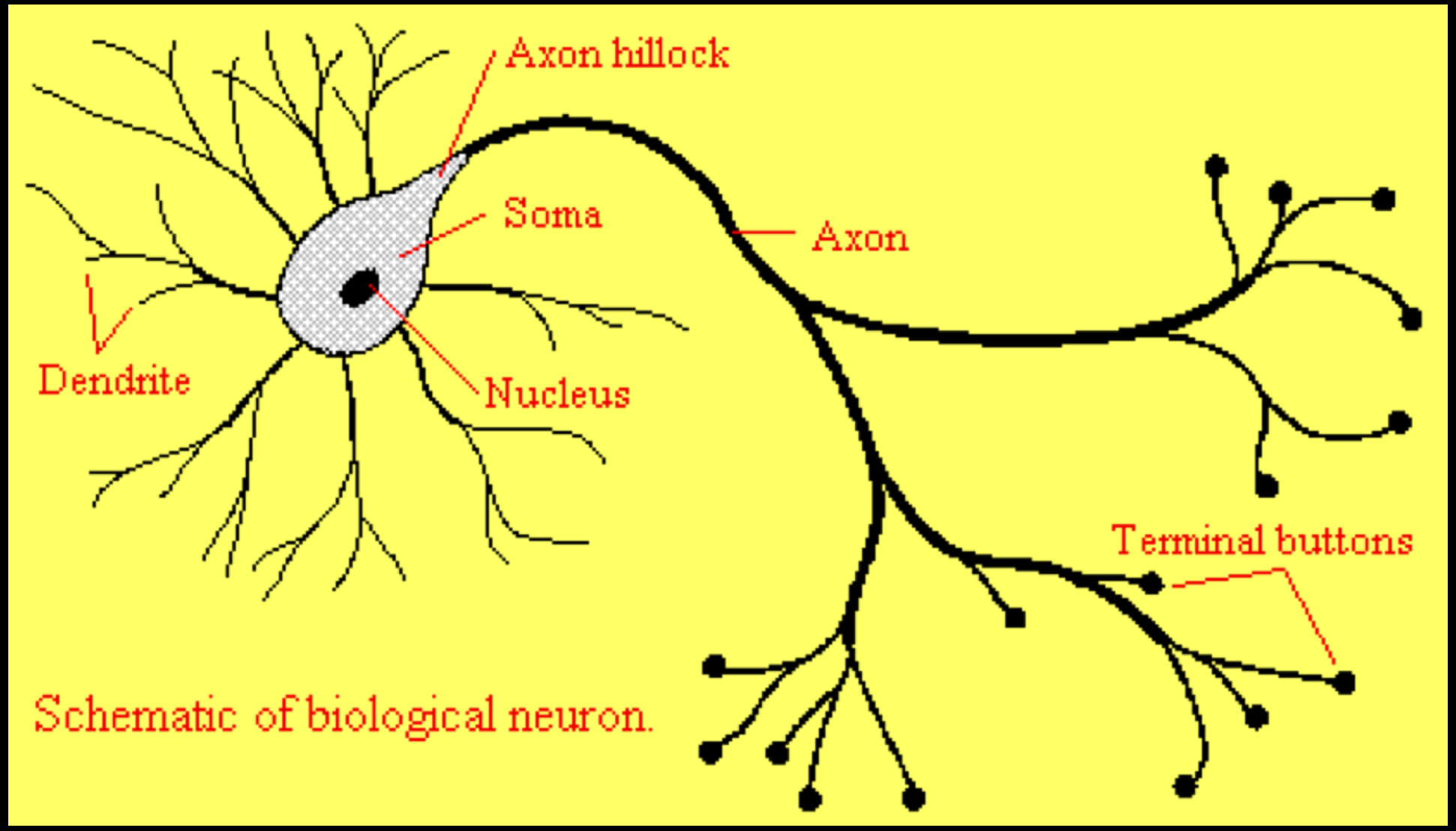
EEG (color indicates “power” and frequency, MRI (Diffusion Tensor Imaging) for structure

Adam Gazzeley Lab - UCSF

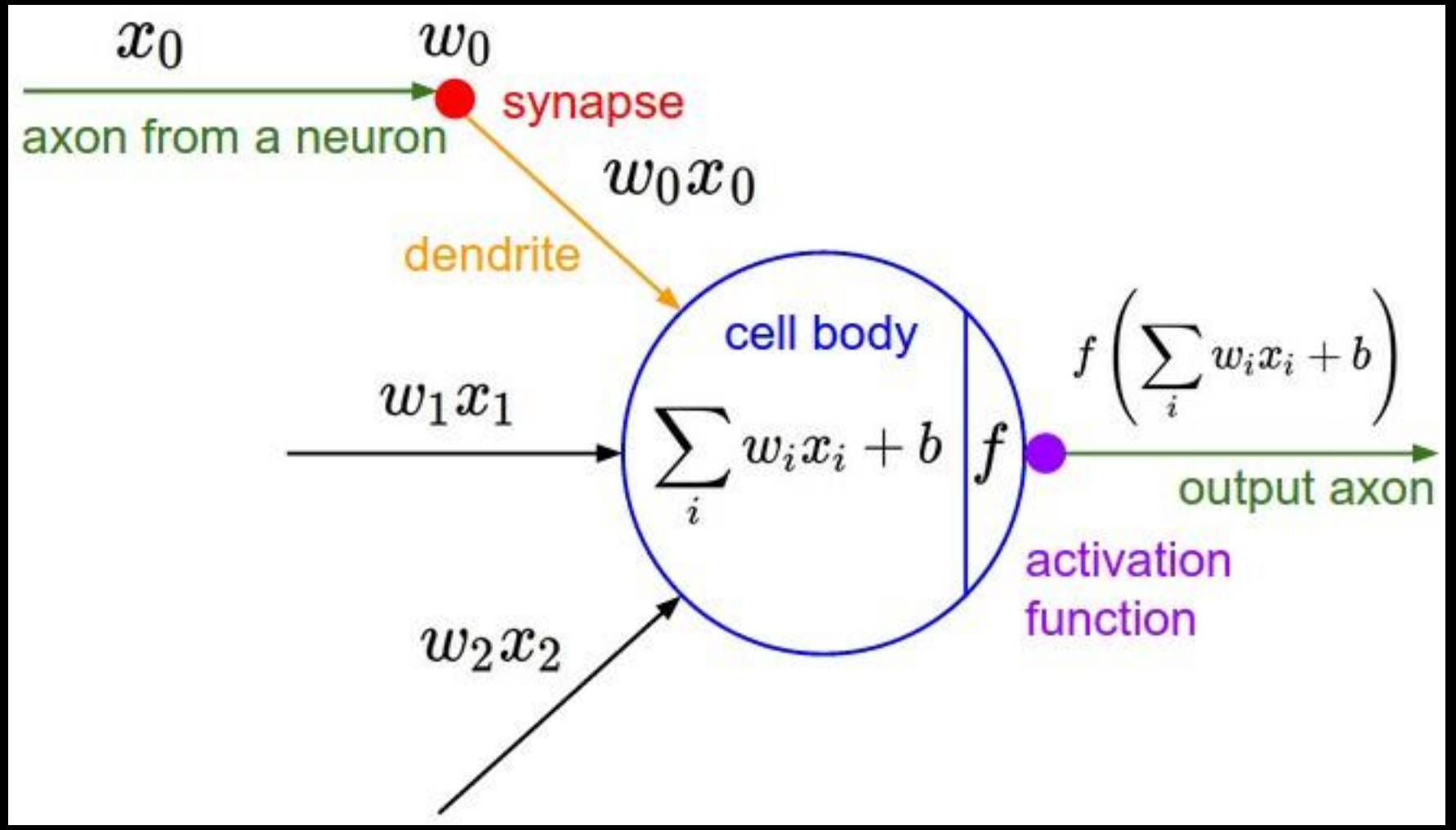
Brain-mapping tools

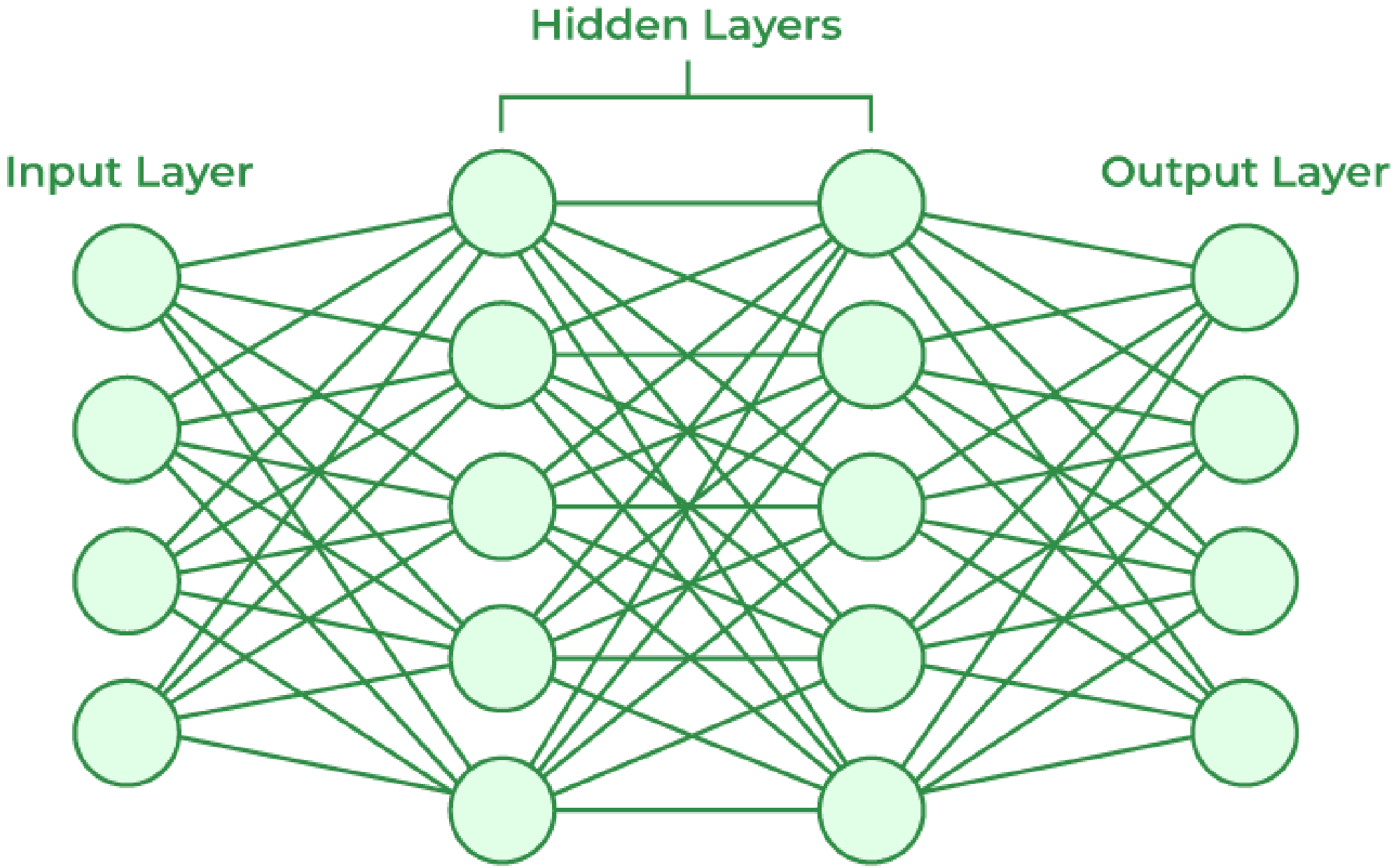


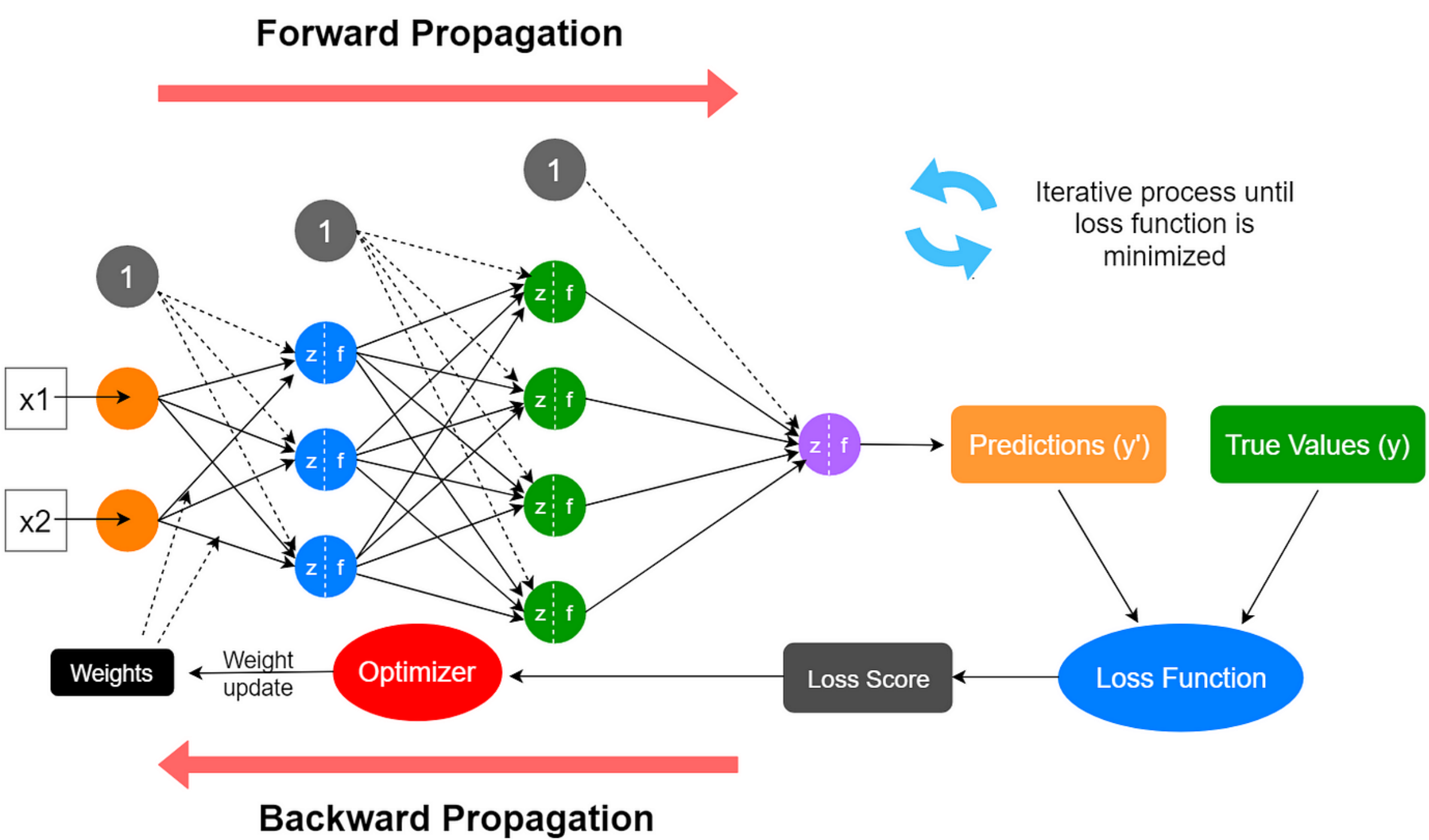
What is a Neuron?



Mathematical Abstraction




Hidden Layers
Input Layer
Output Layer


Forward Propagation
1
1
1
x1
x2
z
f
Iterative process until loss function is minimized
Predictions (y')
True Values (y)
Loss Function
Loss Score
Optimizer
Weight update
Weights
Backward Propagation

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton† & Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA
† Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure[1].

There have been many attempts to design self-organizing neural networks. The aim is to find a powerful synaptic modification rule that will allow an arbitrarily connected neural network to develop an internal structure that is appropriate for a particular task domain. The task is specified by giving the desired state vector of the output units for each state vector of the input units. If the input units are directly connected to the output units it is relatively easy to find learning rules that iteratively adjust the relative strengths of the connections so as to progressively reduce the difference between the actual and desired output vectors[2]. Learning becomes more interesting but more difficult when we introduce hidden units whose actual or desired states are not specified by the task. (In perceptrons, there are 'feature analysers' between the input and output that are not true hidden units because their input connections are fixed by hand, so their states are completely determined by the input vector: they do not learn representations.) The learning procedure must decide under what circumstances the hidden units should be active in order to help achieve the desired input-output behaviour. This amounts to deciding what these units should represent. We demonstrate that a general purpose and relatively simple procedure is powerful enough to construct appropriate internal representations.

The simplest form of the learning procedure is for layered networks which have a layer of input units at the bottom; any number of intermediate layers; and a layer of output units at the top. Connections within a layer or from higher to lower layers are forbidden, but connections can skip intermediate layers. An input vector is presented to the network by setting the states of the input units. Then the states of the units in each layer are determined by applying equations (1) and (2) to the connections coming from lower layers. All units within a layer have their states set in parallel, but different layers have their states set sequentially, starting at the bottom and working upwards until the states of the output units are determined.

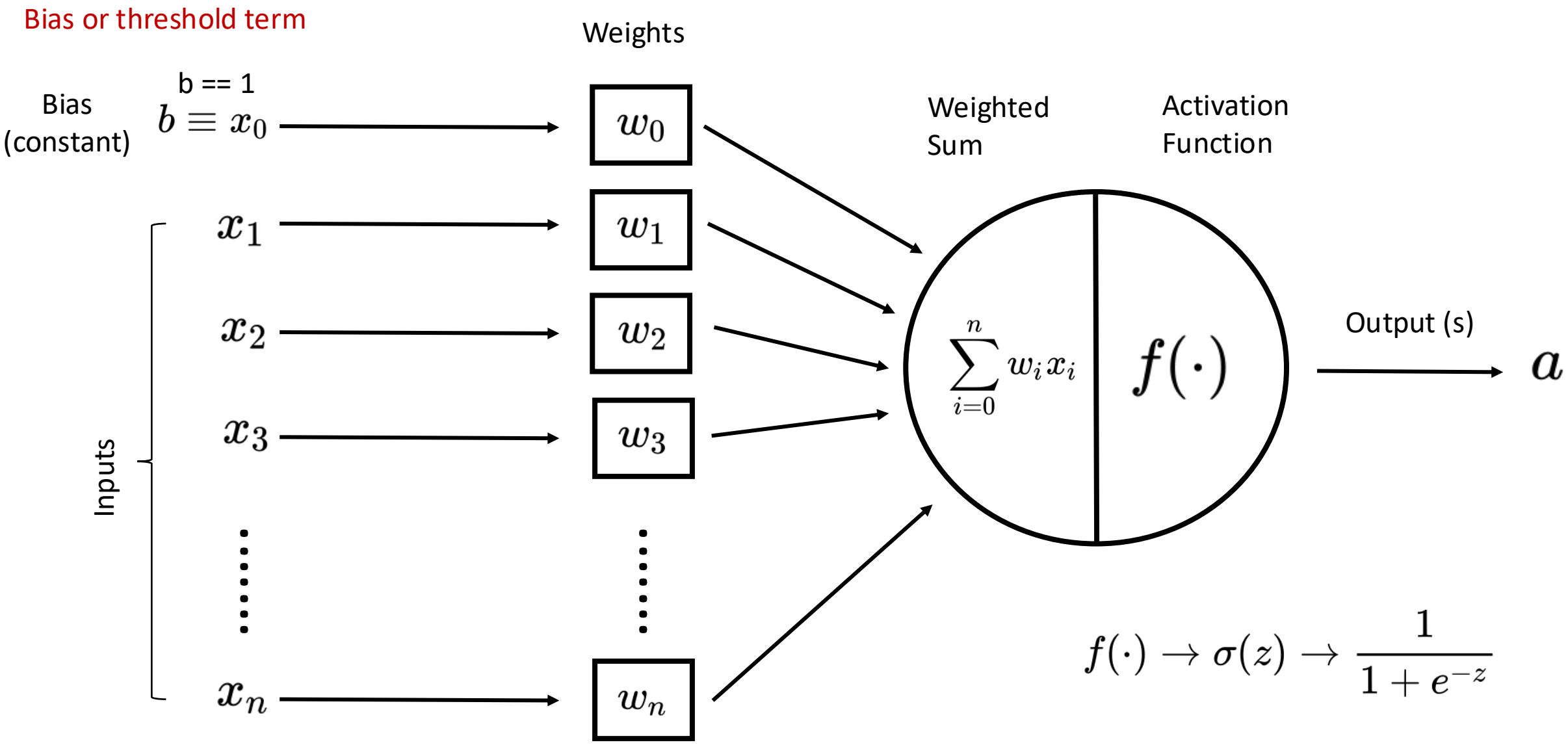
The total input, x_j, to unit j is a linear function of the outputs, y_i, of the units that are connected to j and of the weights, w_{ji}, on these connections

$$x_j = \sum_i y_i w_{ji} \tag{1}$$

Units can be given biases by introducing an extra input to each unit which always has a value of 1. The weight on this extra input is called the bias and is equivalent to a threshold of the opposite sign. It can be treated just like the other weights.

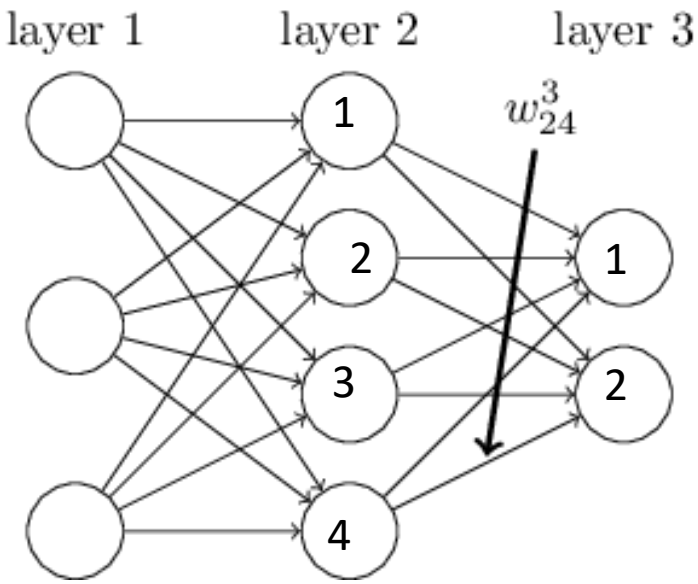
A unit has a real-valued output, y_j, which is a non-linear function of its total input

$$y_j = \frac{1}{1+e^{-x_j}} \tag{2}$$


Bias or threshold term
Weights
b == 1
Bias
(constant)
$b \equiv x_0$
w_0
Weighted
Sum
Activation
Function
x_1
w_1
x_2
w_2
x_3
w_3
Inputs
$\sum_{i=0}^{n} w_i x_i$
$f(\cdot)$
Output (s)
a
x_n
w_n
$f(\cdot) \rightarrow \sigma(z) \rightarrow \frac{1}{1+e^{-z}}$
Example activation function

Convention for naming things

$(l-2)^{th}$ $(l-1)^{th}$ $(l)^{th}$



w^l_{jk} is the weight from the kth neuron in the $(l-1)^{\text{th}}$ layer to the jth neuron in the lth layer

a^l_j

Activation of neuron j in layer l

w^l_{jk}

Weight

l == to (⟶) layer

j == to (⟶) neuron

k == from (⟵) neuron

With these notations, the activation a^l_j of the jth neuron in the lth layer is related to the activations in the $(l-1)^{th}$ layer by the equation

$$a^l_j = \sigma\left(\sum_k w^l_{jk} a^{(l-1)}_k + b^l_j\right)$$

To rewrite this expression in a matrix form we define a *weight matrix* w^l for each layer, l. The entries of the weight matrix w^l are just the weights connecting to the lth layer of neurons, that is, the entry in the jth row and k^{th} column is w^l_{jk} . We can simplify our vector notation a bit by dropping the explicit bias term.

$$a^l = \sigma\left(w^l a^{(l-1)} + b^l\right) \rightarrow a^l = \sigma\left(w^l a^{(l-1)}\right)$$

Note that *bias is also known as threshold*, which will now make more sense to you.

Say bank approves credit if $(\sum_{i=1}^{d} WiXi > threshold)$ and deny if it is lesser. So this linear formula can be written as a hypothesis which is:

$$h(x) = (\sum_{i=1}^{d} WiXi - threshold)$$

Suppose $threshold = -W0$, above equation can be rewritten as

$$h(x) = sign((\sum_{i=1}^{d} WiXi) + W0)$$

Introducing X0=1 in the equation.

$$h(x) = sign((\sum_{i=1}^{d} WiXi) + W0X0)$$

Now we can simply write the hypothesis equation as

$$h(x) = sign(\sum_{i=0}^{d} WiXi).$$

This is the standard form. The vector form is $h(x) = sign(W'X)$.

can you write a simple MLP in PyTorch.

oss120

Thought for less than a second

Sure! Below is a **minimal, self-contained example** of a Multi-Layer Perceptron (MLP) built with PyTorch.
It includes:

1. **Model definition** (`SimpleMLP`)
2. **Synthetic dataset** (you can replace it with any real data, e.g. MNIST, tabular CSV, etc.)
3. **Training loop** (with loss, optimizer, and optional validation)
4. **Device-agnostic code** (CPU/GPU)

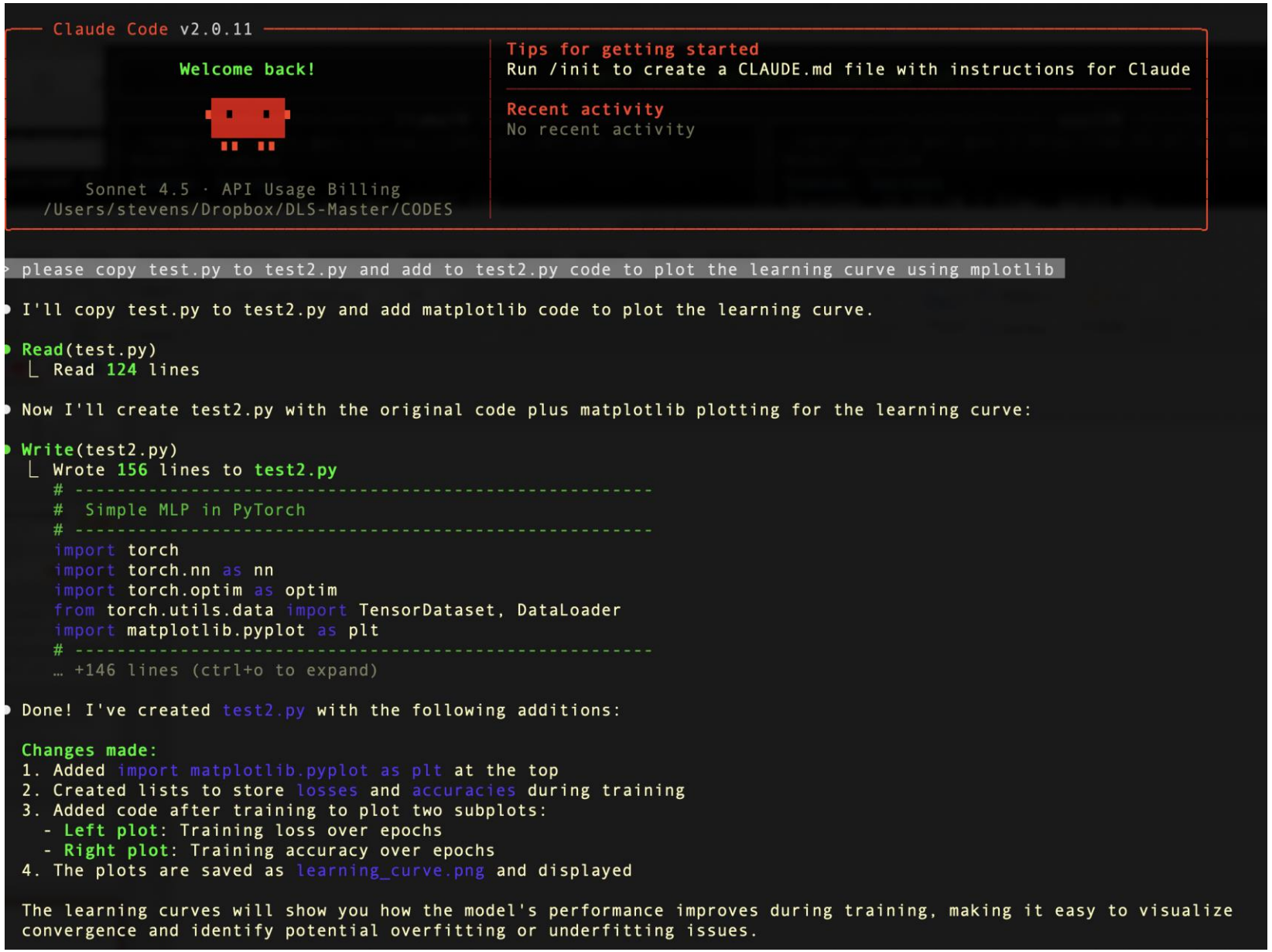
python Collapse Run Save Copy

```python
# ------------------------------------------------------------
#  Simple MLP in PyTorch
# ------------------------------------------------------------
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader

# ------------------------------------------------------------
#  1️⃣  Model definition
# ------------------------------------------------------------
class SimpleMLP(nn.Module):
    """A 3-layer fully-connected network."""
    def __init__(self, input_dim: int, hidden_dims: list[int], output_dim: int):
        """
        Args:
            input_dim  – dimensionality of the input features
            hidden_dims – list of hidden layer sizes, e.g. [64, 32]
            output_dim – number of output units (e.g. number of classes)
```

Send a Message

+ Code Interpreter

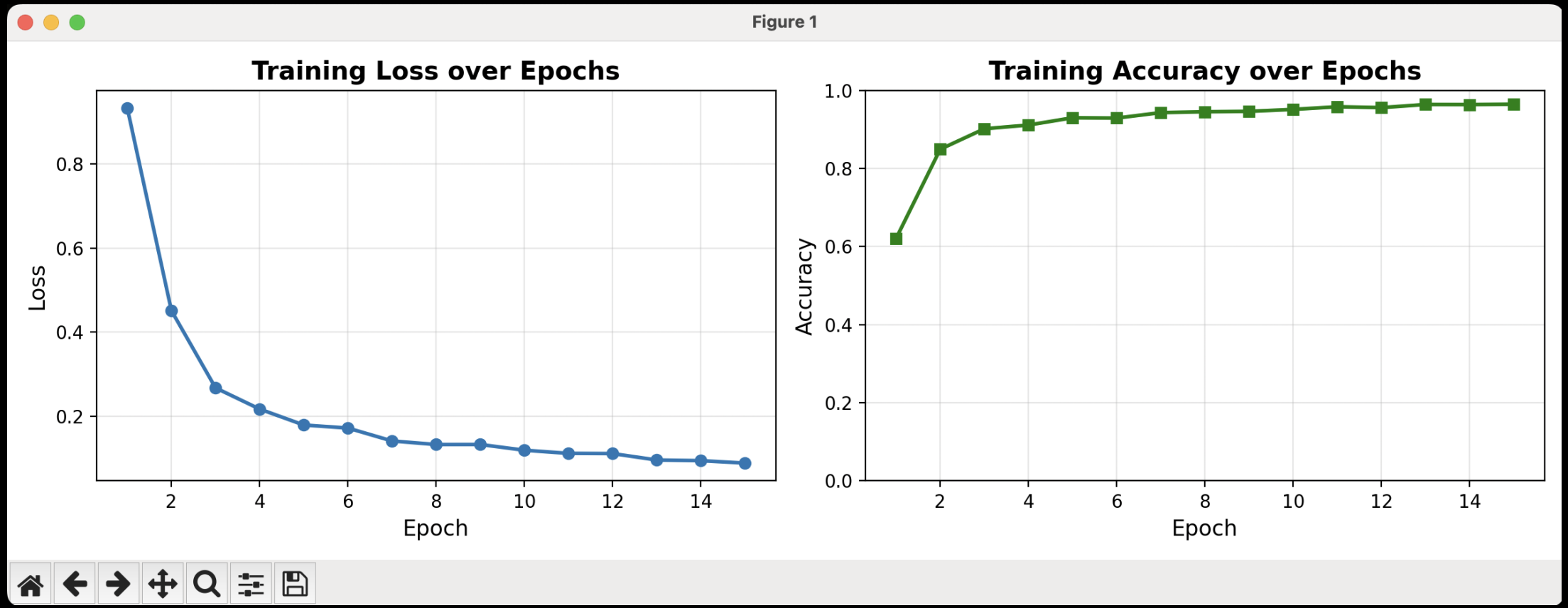

Claude Code v2.0.11
Welcome back!
Sonnet 4.5 · API Usage Billing
/Users/stevens/Dropbox/DLS-Master/CODES
Tips for getting started
Run /init to create a CLAUDE.md file with instructions for Claude
Recent activity
No recent activity
> please copy test.py to test2.py and add to test2.py code to plot the learning curve using mplotlib
I'll copy test.py to test2.py and add matplotlib code to plot the learning curve.
Read(test.py)
└ Read 124 lines
Now I'll create test2.py with the original code plus matplotlib plotting for the learning curve:
Write(test2.py)
└ Wrote 156 lines to test2.py

Simple MLP in PyTorch
----
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import matplotlib.pyplot as plt

… +146 lines (ctrl+o to expand)
Done! I've created test2.py with the following additions:
Changes made:
1. Added import matplotlib.pyplot as plt at the top
2. Created lists to store losses and accuracies during training
3. Added code after training to plot two subplots:
- Left plot: Training loss over epochs
- Right plot: Training accuracy over epochs
4. The plots are saved as learning_curve.png and displayed
The learning curves will show you how the model's performance improves during training, making it easy to visualize convergence and identify potential overfitting or underfitting issues.

CODES — -zsh — zsh — Homebrew — 104×37

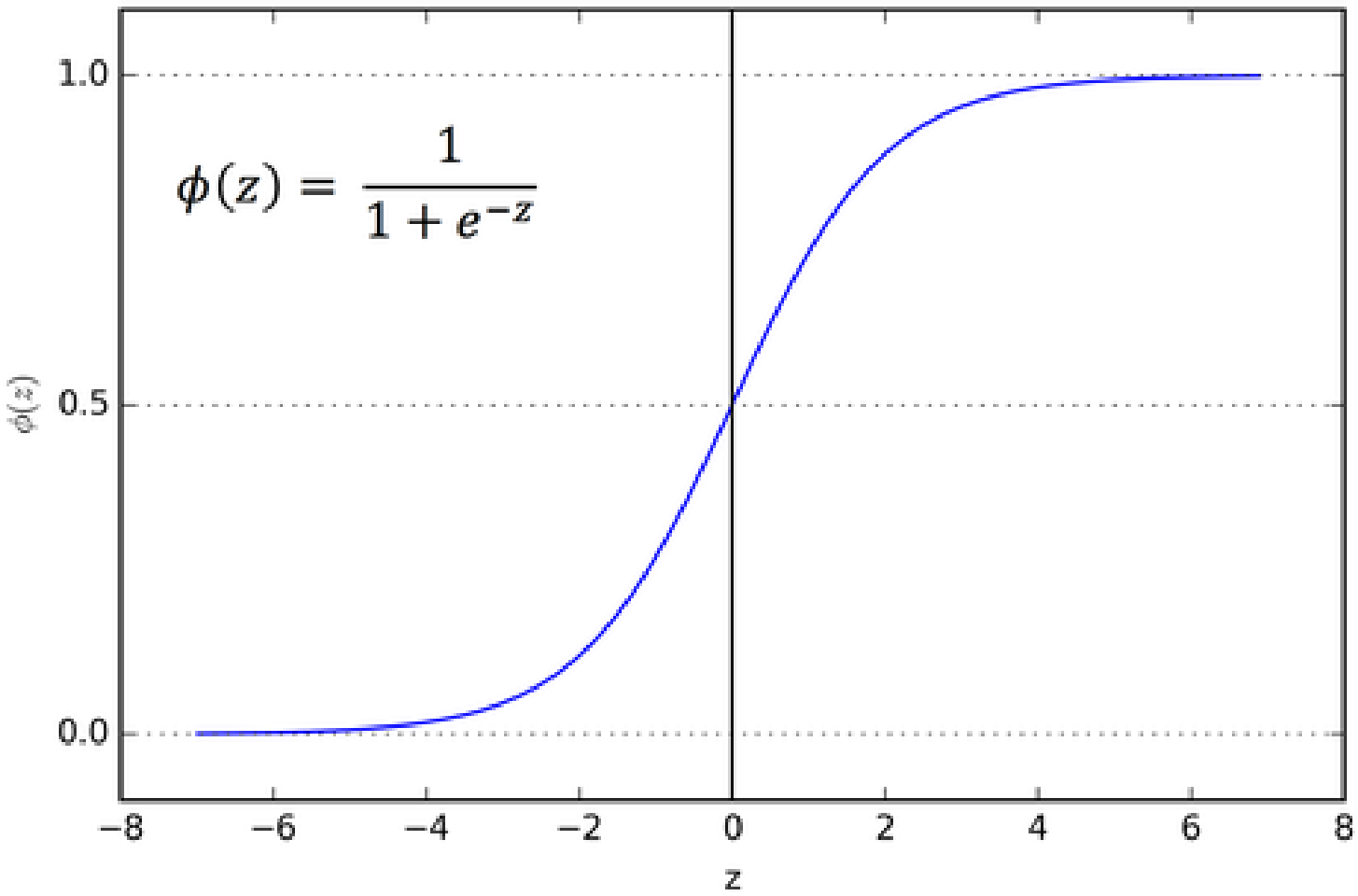
```
(base) stevens@M3acBook-Pro CODES % diff test.py test2.py
7a8
> import matplotlib.pyplot as plt
118a120,123
> # Store metrics for plotting
> losses = []
> accuracies = []
>
120a126,127
>     losses.append(loss)
>     accuracies.append(acc)
123a131,154
>
> # ------------------------------------------------------------
> #  6️⃣ Plot learning curves
> # ------------------------------------------------------------
> fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
>
> # Plot loss
> ax1.plot(range(1, EPOCHS + 1), losses, marker='o', linewidth=2, markersize=6)
> ax1.set_xlabel('Epoch', fontsize=12)
> ax1.set_ylabel('Loss', fontsize=12)
> ax1.set_title('Training Loss over Epochs', fontsize=14, fontweight='bold')
> ax1.grid(True, alpha=0.3)
>
> # Plot accuracy
> ax2.plot(range(1, EPOCHS + 1), accuracies, marker='s', color='green', linewidth=2, markersize=6)
> ax2.set_xlabel('Epoch', fontsize=12)
> ax2.set_ylabel('Accuracy', fontsize=12)
> ax2.set_title('Training Accuracy over Epochs', fontsize=14, fontweight='bold')
> ax2.grid(True, alpha=0.3)
> ax2.set_ylim([0, 1])
>
> plt.tight_layout()
> plt.savefig('learning_curve.png', dpi=150, bbox_inches='tight')
> print("\nLearning curves saved to 'learning_curve.png'")
(base) stevens@M3acBook-Pro CODES %
```


Figure 1
Training Loss over Epochs
Loss
Epoch
0.2
0.4
0.6
0.8
2
4
6
8
10
12
14
Training Accuracy over Epochs
Accuracy
Epoch
0.0
0.2
0.4
0.6
0.8
1.0
2
4
6
8
10
12
14

Activations

- **Originally “Squash” functions** – compressing values into a given interval
- **Differentiable** – we can compute the derivative
- **Monotonic** – entirely increasing or decreasing
- **Simple** – fast to compute

Sigmoid

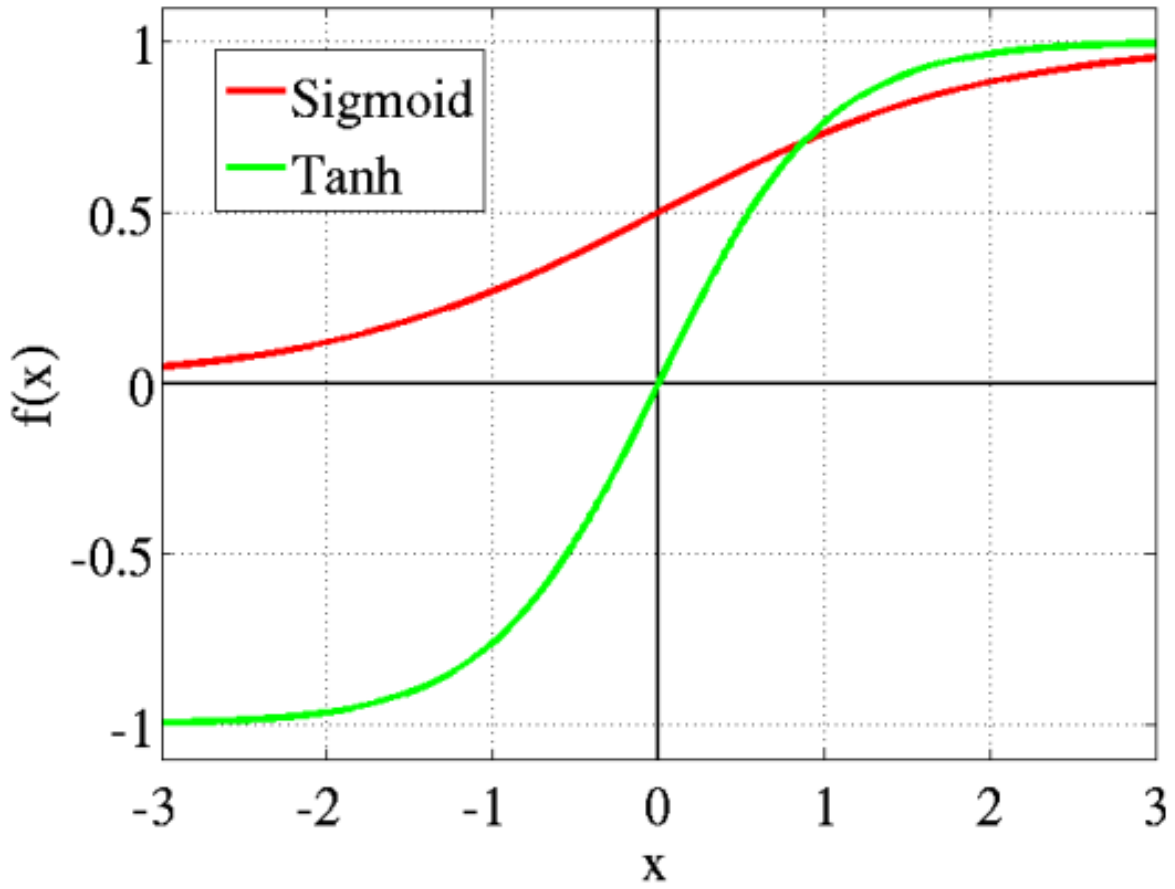


Sigmoid

- The main reason why we use sigmoid function is because it exists between **(0 to 1).** Therefore, it is especially used for models where we have to **predict the probability** as an output. Since probability of anything exists only between the range of **0 and 1,** sigmoid is the right choice
- The function is **differentiable**. That means, we can find the slope of the sigmoid curve at any point.
- The function is **monotonic** but the function's derivative is not.
- The logistic sigmoid function can cause a neural network to get stuck during training time
- The **softmax function** is a more generalized logistic activation function which is used for multiclass classification

$$\sigma(x_j) = \frac{e^{x_j}}{\sum_i e^{x_i}}$$

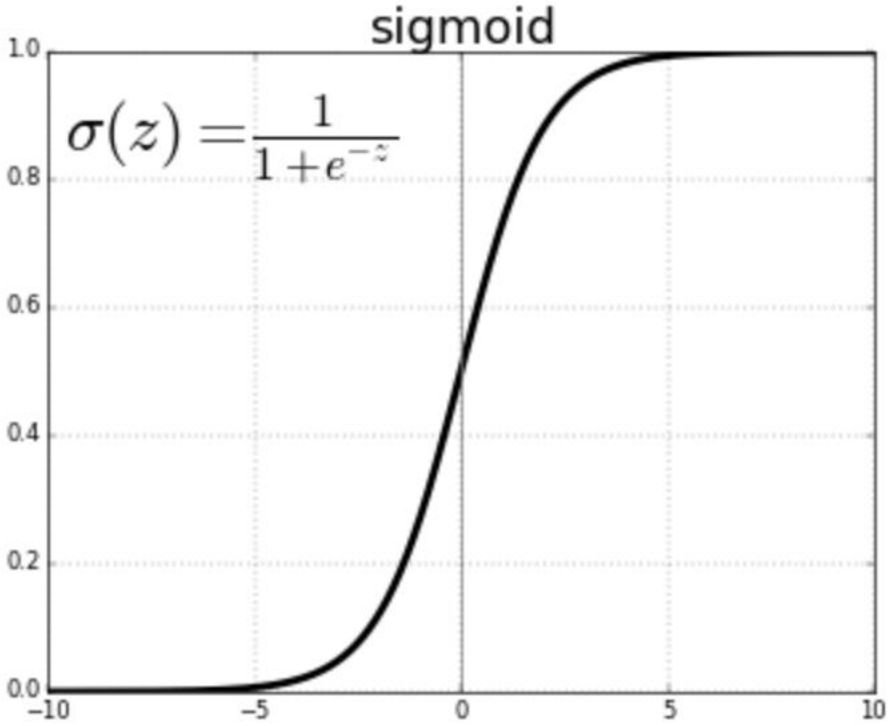
Tanh (hyperbolic tangent)

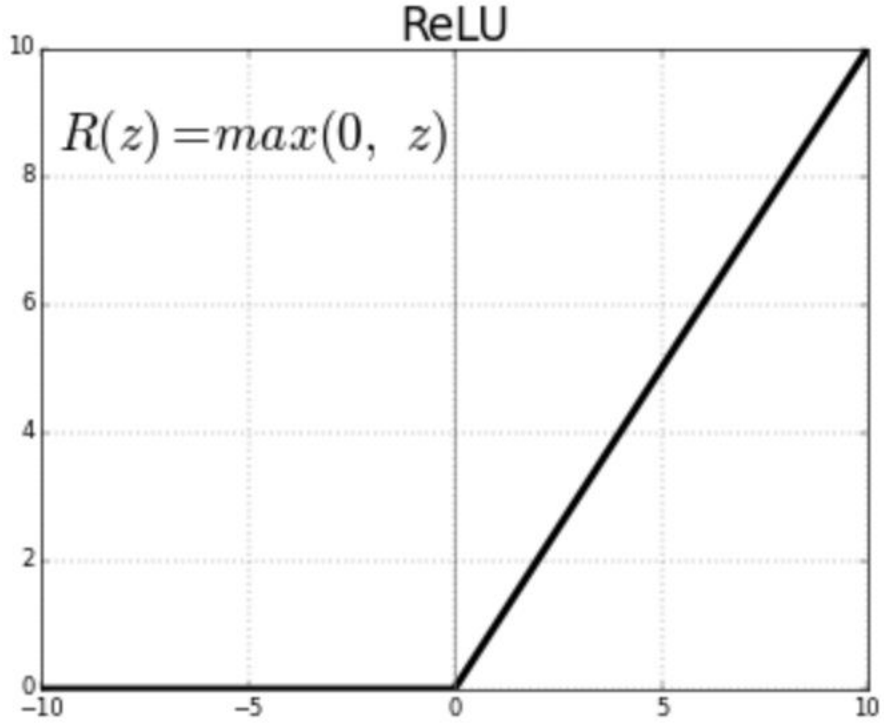


Tanh (hyperbolic tangent)

- Tanh is also like sigmoid but with -1 to 1 range
- Tanh is also sigmoidal (s - shaped)
- The advantage is that the negative inputs will be mapped strongly negative and the zero inputs will be mapped near zero in the tanh graph
- The function is **differentiable**
- The function is **monotonic** while its **derivative is not monotonic**
- The tanh function is mainly used in classification between two classes
- Both tanh and logistic sigmoid activation functions are used in feed-forward nets

ReLU (Rectified Linear Unit)

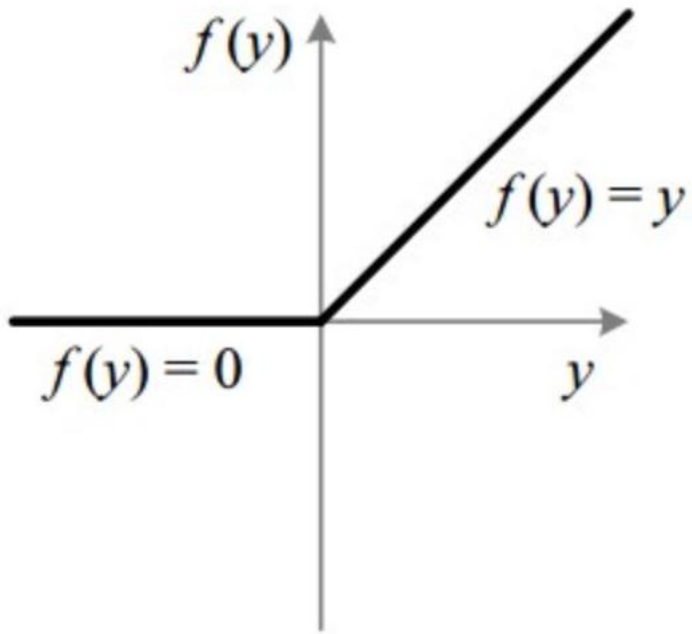


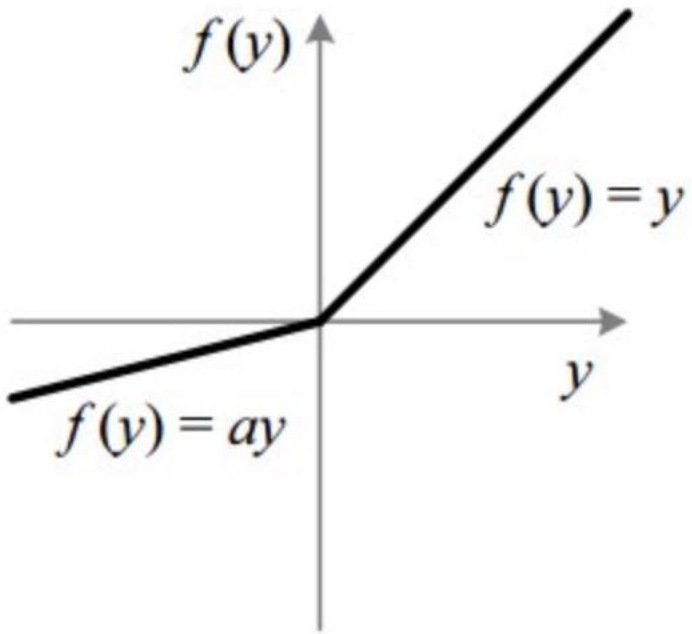


ReLU (Rectified Linear Unit)

- The ReLU is the most used activation function in the world right now. Since, it is used in almost all the convolutional neural networks in deep learning
- As you can see, the ReLU is half rectified (from bottom). f(z) is zero when z is less than zero and f(z) is equal to z when z is above or equal to zero.
- **Range:** [0 to infinity)
- The function and its derivative **both are monotonic**.
- But the issue is that all the negative values become zero immediately which can limit the ability of the model to fit or train from the data
- Negative input values given to the ReLU activation function turns the value into zero immediately in the graph, which in turns affects the resulting graph by not mapping the negative values appropriately.

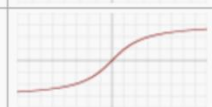
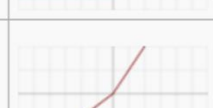
Leaky ReLU

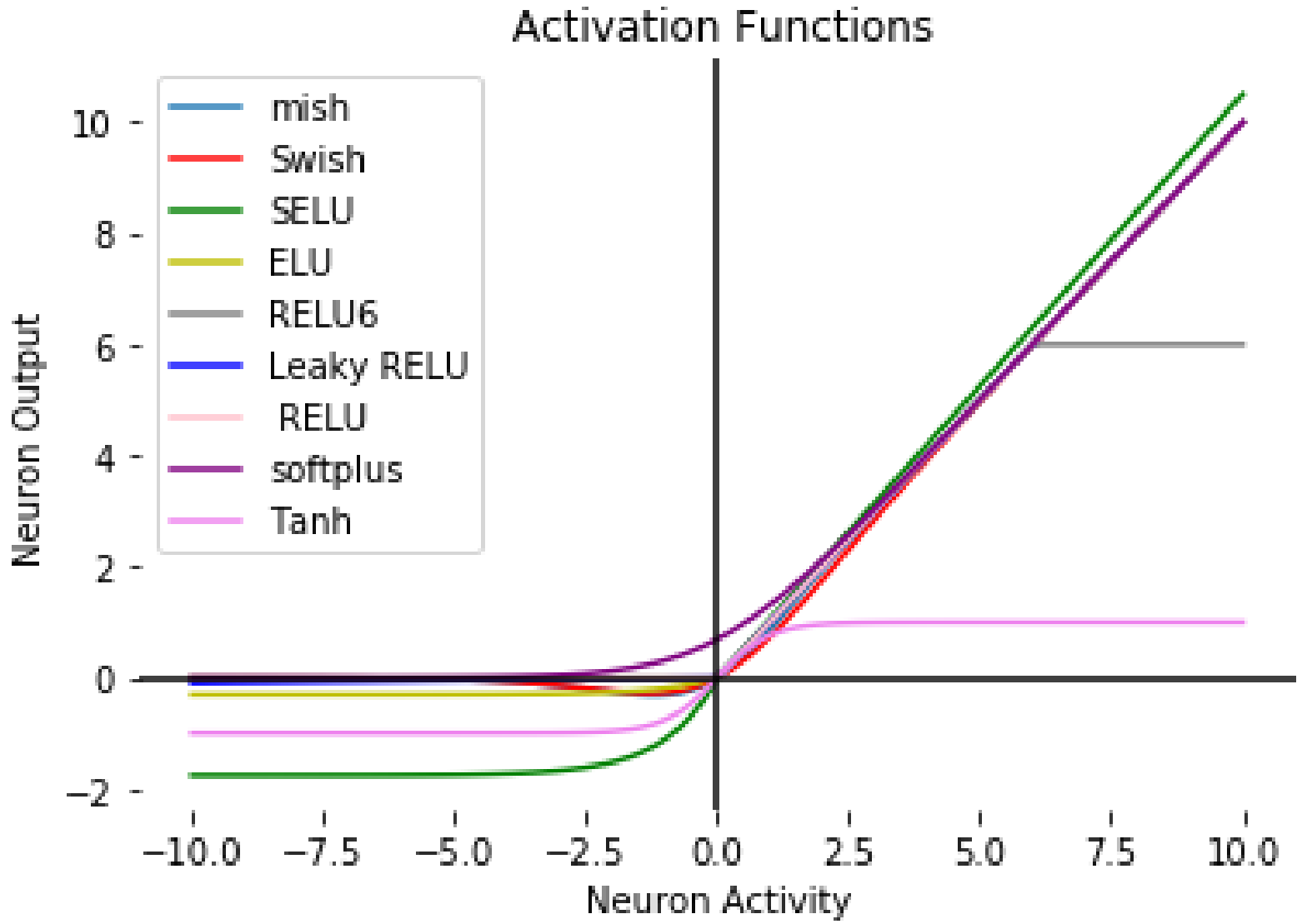




Leaky ReLU

- It is an attempt to solve the ReLU “zero” problem
- Can you see the Leak? 
- The leak helps to increase the range of the ReLU function. Usually, the value of **a** is 0.01 or so
- When **a is not 0.01** then it is called **Randomized ReLU**
- Therefore the **range** of the Leaky ReLU is (-infinity to infinity)
- Both Leaky and Randomized ReLU functions are monotonic, also, their derivatives also monotonic

Name	Plot	Equation	Derivative
Identity		$f(x) = x$	$f'(x) = 1$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
TanH		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parameteric Rectified Linear Unit (PReLU) [2]		$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Exponential Linear Unit (ELU) [3]		$f(x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
SoftPlus		$f(x) = \log_e(1 + e^x)$	$f'(x) = \frac{1}{1 + e^{-x}}$


Activation Functions
mish
Swish
SELU
ELU
RELU6
Leaky RELU
RELU
softplus
Tanh
Neuron Output
10
8
6
4
2
0
−2
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
Neuron Activity

Baselines	ReLU	LReLU	PReLU	Softplus	ELU	SELU	GELU
Swish > Baseline	9	7	6	6	8	8	8
Swish = Baseline	0	1	3	2	0	1	1
Swish < Baseline	0	1	0	1	1	0	0

Table 3: The number of models on which Swish outperforms, is equivalent to, or underperforms each baseline activation function we compared against in our experiments.

Model	ResNet	WRN	DenseNet
LReLU	94.2	95.6	94.7
PReLU	94.1	95.1	94.5
Softplus	94.6	94.9	94.7
ELU	94.1	94.1	94.4
SELU	93.0	93.2	93.9
GELU	94.3	95.5	94.8
ReLU	93.8	95.3	94.8
Swish-1	94.7	95.5	94.8
Swish	94.5	95.5	94.8

Table 4: CIFAR-10 accuracy.

Model	ResNet	WRN	DenseNet
LReLU	74.2	78.0	83.3
PReLU	74.5	77.3	81.5
Softplus	76.0	78.4	83.7
ELU	75.0	76.0	80.6
SELU	73.2	74.3	80.8
GELU	74.7	78.0	83.8
ReLU	74.2	77.8	83.7
Swish-1	75.1	78.5	83.8
Swish	75.1	78.0	83.9

Table 5: CIFAR-100 accuracy.

$$f(x) = x \cdot \sigma(x)$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$f(x) = x \cdot \sigma(\beta x) = \frac{x}{1 + e^{-\beta x}}$$

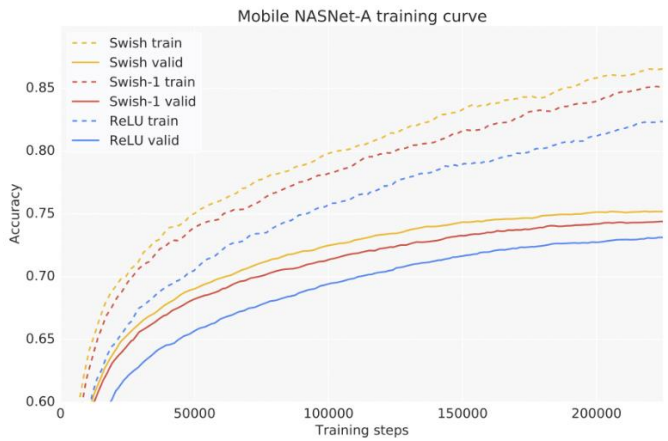


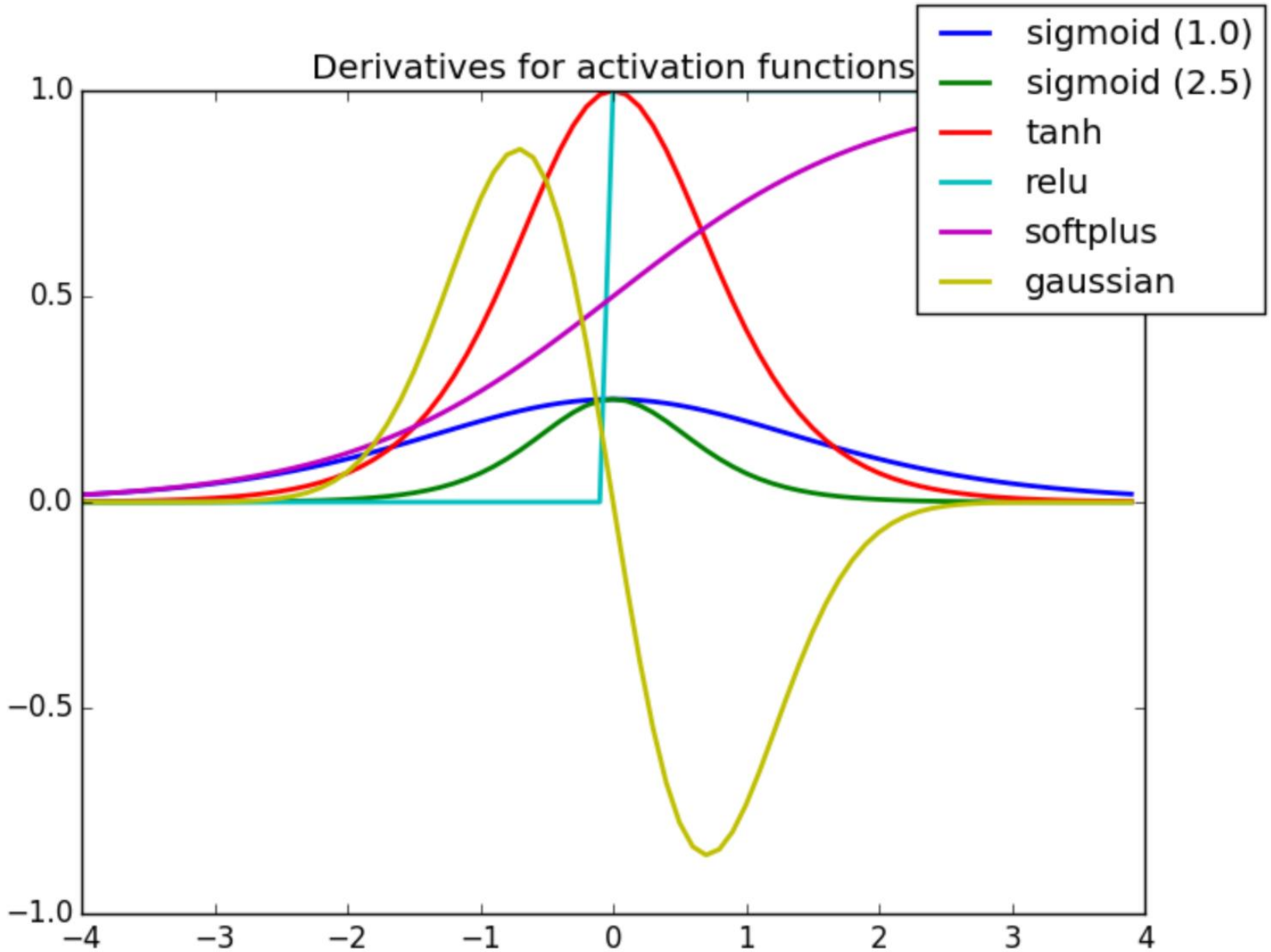
Figure 8: Training curves of Mobile NASNet-A on ImageNet. Best viewed in color

Model	Top-1 Acc. (%)			Top-5 Acc. (%)		
LReLU	73.8	73.9	74.2	91.6	91.9	91.9
PReLU	74.6	74.7	74.7	92.4	92.3	92.3
Softplus	74.0	74.2	74.2	91.6	91.8	91.9
ELU	74.1	74.2	74.2	91.8	91.8	91.8
SELU	73.6	73.7	73.7	91.6	91.7	91.7
GELU	74.6	-	-	92.0	-	-
ReLU	73.5	73.6	73.8	91.4	91.5	91.6
Swish-1	74.6	74.7	74.7	92.1	92.0	92.0
Swish	74.9	74.9	75.2	92.3	92.4	92.4

Table 6: Mobile NASNet-A on ImageNet, with 3 different runs ordered by top-1 accuracy. The additional 2 GELU experiments are still training at the time of submission.

Swish is the general form

SILU is with beta == 1


Derivatives for activation functions
sigmoid (1.0)
sigmoid (2.5)
tanh
relu
softplus
gaussian
1.0
0.5
0.0
−0.5
−1.0
−4
−3
−2
−1
0
1
2
3
4

Loss Functions

Definition

In a supervised deep learning context the **loss function** measures the **quality** of a particular set of parameters based on how well the output of the network **agrees** with the ground truth labels in the training data.

Nomenclature

loss function

=

cost function

=

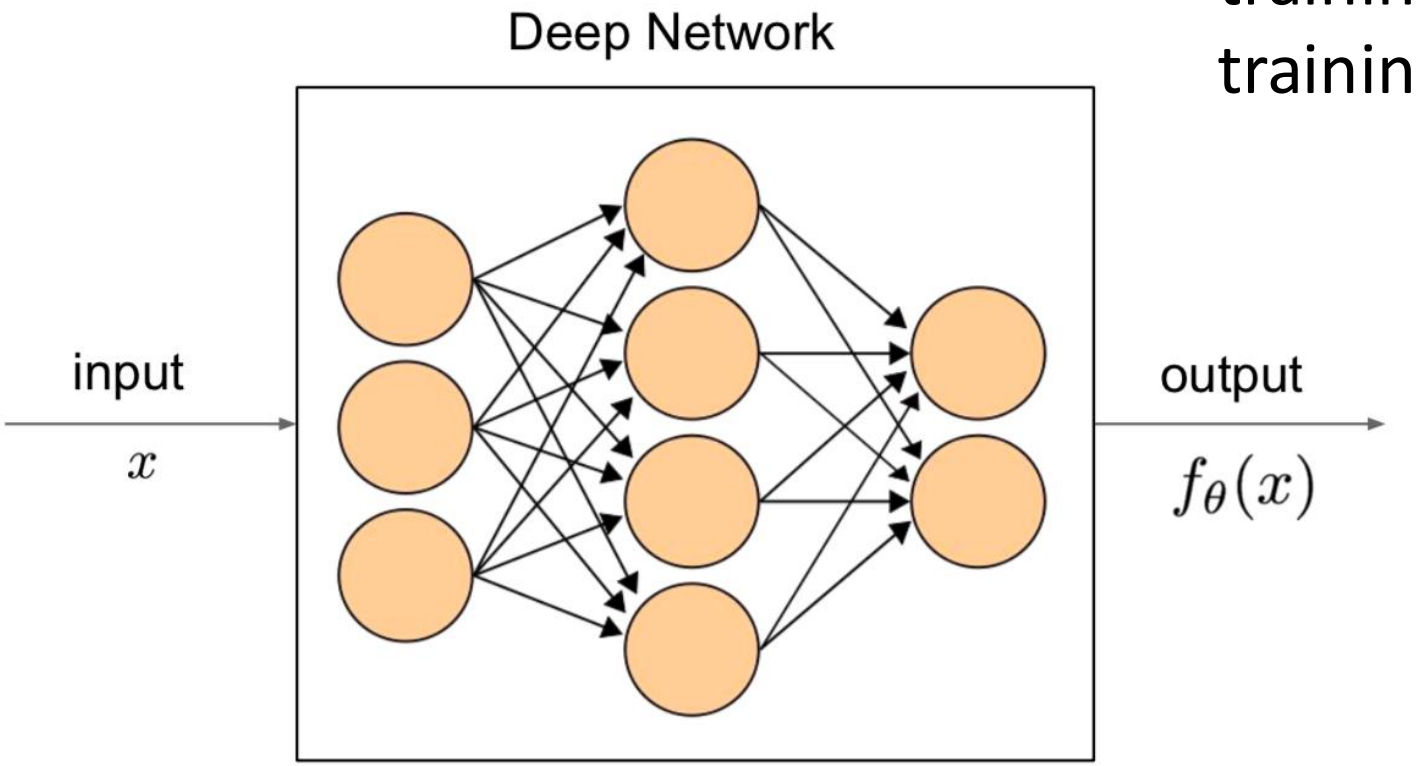
objective function

=

error function

Loss function

How close is the prediction of the model (trained from the training data) when compared to training data?



labels (ground truth)

input

$$\mathcal{L}(w) = distance(f_\theta(x), y)$$

error

parameters (weights, biases)

Training Loss vs Validation Loss (or Test Loss)

- The loss function is used to estimate the distance between the predictions the model makes and the labels in the training data
- This estimated distance (between what we are predicting and what the right answer is) or error is what will be used to improve the model

- The same function can be applied to compare the predictions of the model to labels in data not used to train the model (i.e. validation or testing data). This estimate is not used to improve the model directly and is referred to as "validation loss"
- Other metrics (perhaps more understandable to humans) might also be used to compare model predictions to validation data

Definition of Loss (MSE)

$$L(w,b) \equiv \frac{1}{2n} \sum_x \| \, y(x) - a \|^2$$

$y(x)$	The ground truth label for each sample *x*
a	The activation of the last layer for sample *x*

Loss Definition

$$L(w) \equiv \frac{1}{2n} \sum_{x} \| y - \hat{y} \|^2$$

y is the label for sample *x* and *y* (hat) is the prediction for sample *x*

we also collapse the bias into the weights for simplicity of presentation

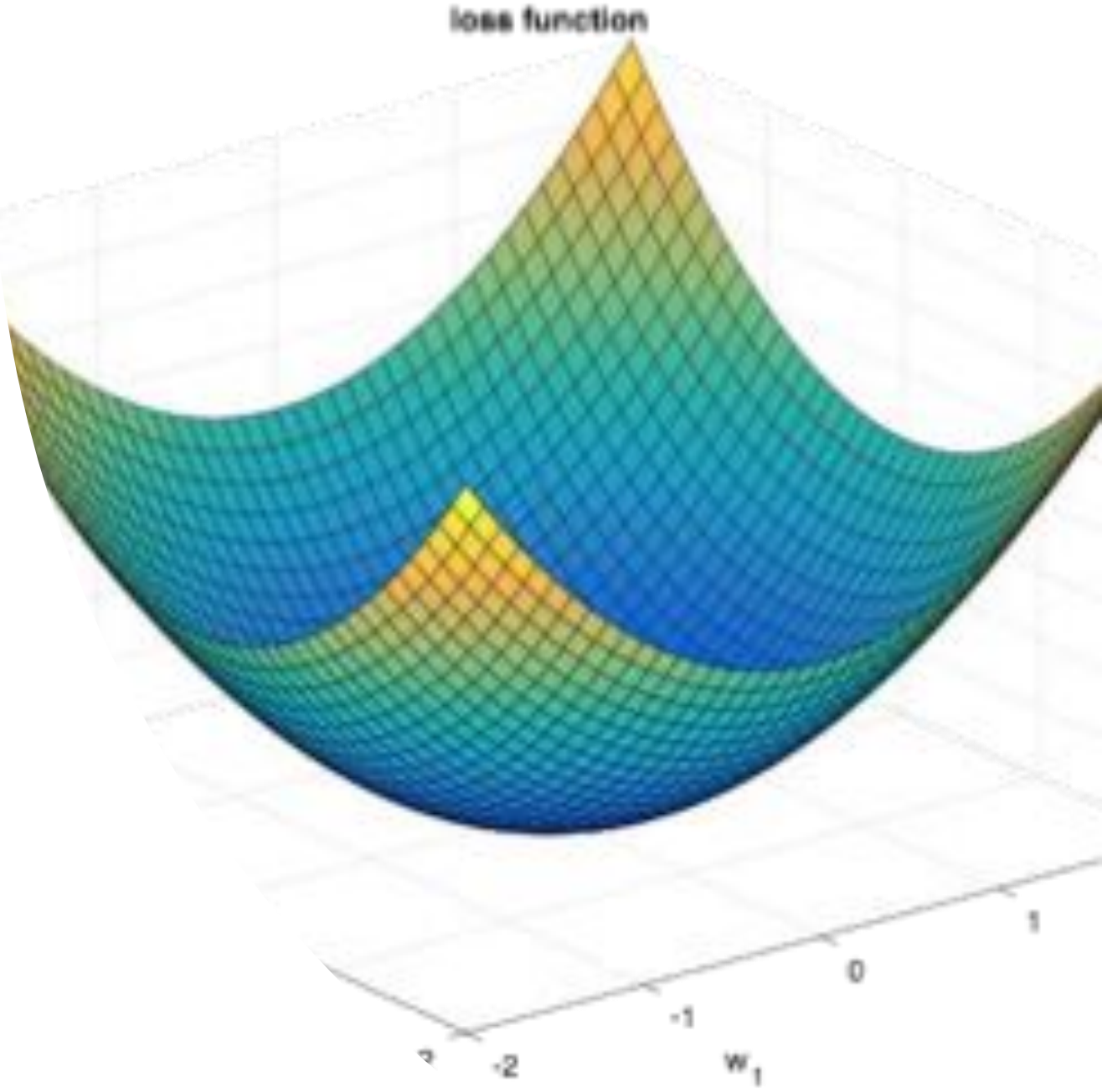
Overall Loss is Mean Loss over Samples

$$L = \frac{1}{n} \sum_{x} L_x$$

We simplify our notation to just refer to Loss and Loss for a specific sample

Properties we need in Loss Function

- Minimum (0 value) when the output of the network is equal to the ground truth data
- Increase value when output differs from ground truth
- Ideally $\Longrightarrow$ convex function
- In reality $\Longrightarrow$ (order millions of parameters and not convex)
- Varies smooth with changes in the output
  - Gradients for gradient descent
  - Easy to compute small changes in parameters to get an improvement in loss



Assumptions

- For **backpropagation** to work:
  - Loss function can be written as an average over loss functions for individual training examples:

empirical risk

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}_i$$

  - Loss functions can be written as a function of the output activations from the neural network.

Loss Functions depend on task type

- **Regression:** predict continuous numeric values
  - Absolute Error, Squared Error, etc.

- **Classification:** predict categorical values
  - Hinge, Cross-entropy, etc.

Gradient of the Loss

The gradient of the loss is equal to the mean of the gradient over each sample loss

$$\nabla L = \frac{1}{n} \sum_{x} \nabla L_x$$

This fact will be useful when we are trying to estimate the overall loss gradient from subset of samples

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton† & Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA
† Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure[1].

There have been many attempts to design self-organizing neural networks. The aim is to find a powerful synaptic modification rule that will allow an arbitrarily connected neural network to develop an internal structure that is appropriate for a particular task domain. The task is specified by giving the desired state vector of the output units for each state vector of the input units. If the input units are directly connected to the output units it is relatively easy to find learning rules that iteratively adjust the relative strengths of the connections so as to progressively reduce the difference between the actual and desired output vectors[2]. Learning becomes more interesting but more difficult when we introduce hidden units whose actual or desired states are not specified by the task. (In perceptrons, there are 'feature analysers' between the input and output that are not true hidden units because their input connections are fixed by hand, so their states are completely determined by the input vector: they do not learn representations.) The learning procedure must decide under what circumstances the hidden units should be active in order to help achieve the desired input-output behaviour. This amounts to deciding what these units should represent. We demonstrate that a general purpose and relatively simple procedure is powerful enough to construct appropriate internal representations.

The simplest form of the learning procedure is for layered networks which have a layer of input units at the bottom; any number of intermediate layers; and a layer of output units at the top. Connections within a layer or from higher to lower layers are forbidden, but connections can skip intermediate layers. An input vector is presented to the network by setting the states of the input units. Then the states of the units in each layer are determined by applying equations (1) and (2) to the connections coming from lower layers. All units within a layer have their states set in parallel, but different layers have their states set sequentially, starting at the bottom and working upwards until the states of the output units are determined.

The total input, x_j, to unit j is a linear function of the outputs, y_i, of the units that are connected to j and of the weights, w_{ji}, on these connections

$$x_j = \sum_i y_i w_{ji} \tag{1}$$

Units can be given biases by introducing an extra input to each unit which always has a value of 1. The weight on this extra input is called the bias and is equivalent to a threshold of the opposite sign. It can be treated just like the other weights.

A unit has a real-valued output, y_j, which is a non-linear function of its total input

$$y_j = \frac{1}{1+e^{-x_j}} \tag{2}$$

At its essence backpropagation is a clever application of the chain rule

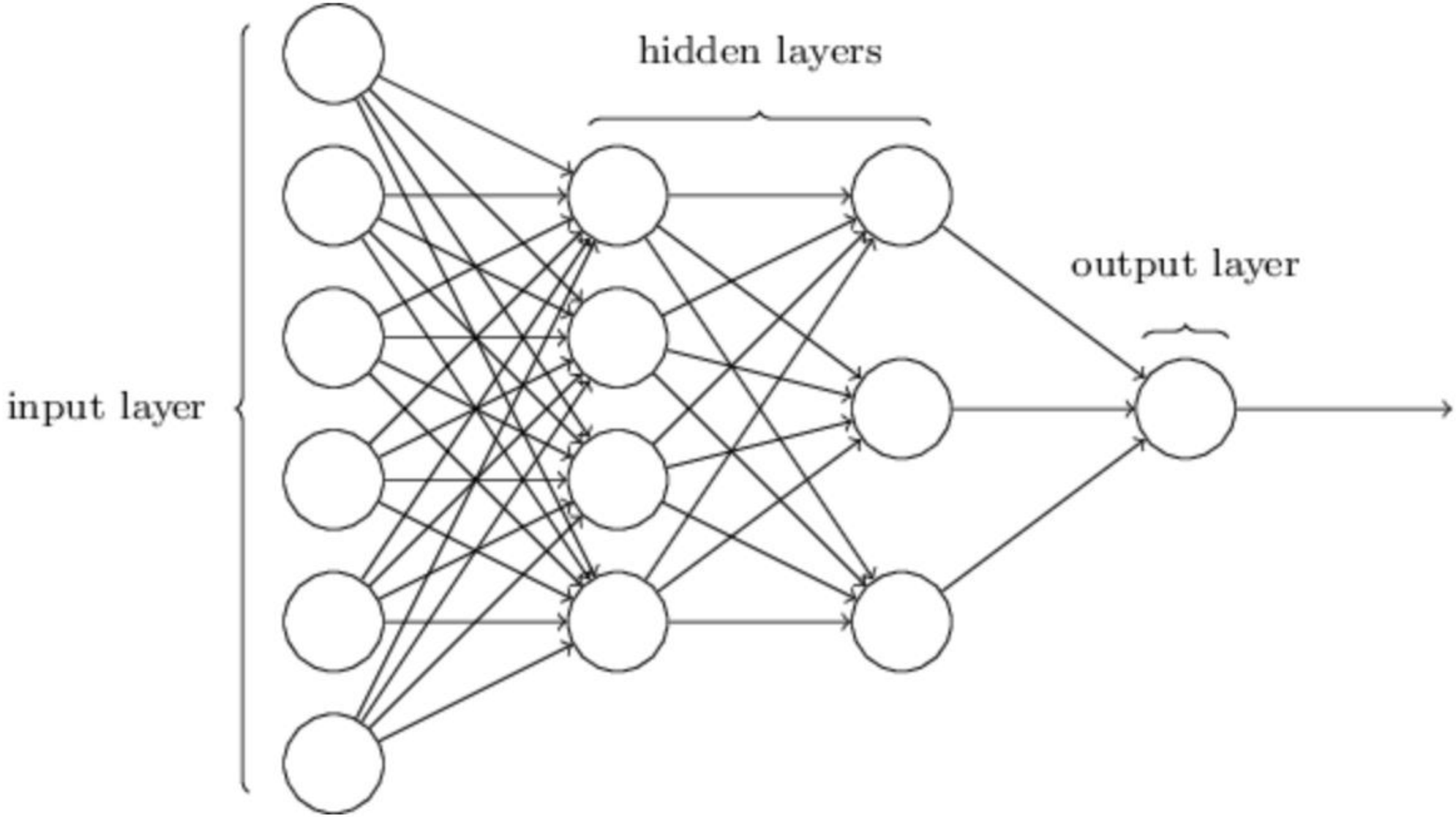
The chain rule is a fundamental property of derivatives taught in any undergraduate curriculum. It states that if you have 3 functions *f*, *g* and *h* with *f* being a function of *g* and *g* being a function of *h* then the derivative of *f* with respect to *h* is equal to the product of the derivative of *f* with respect to *g* and the derivative of *g* with respect to *h.* This can be neatly written as:

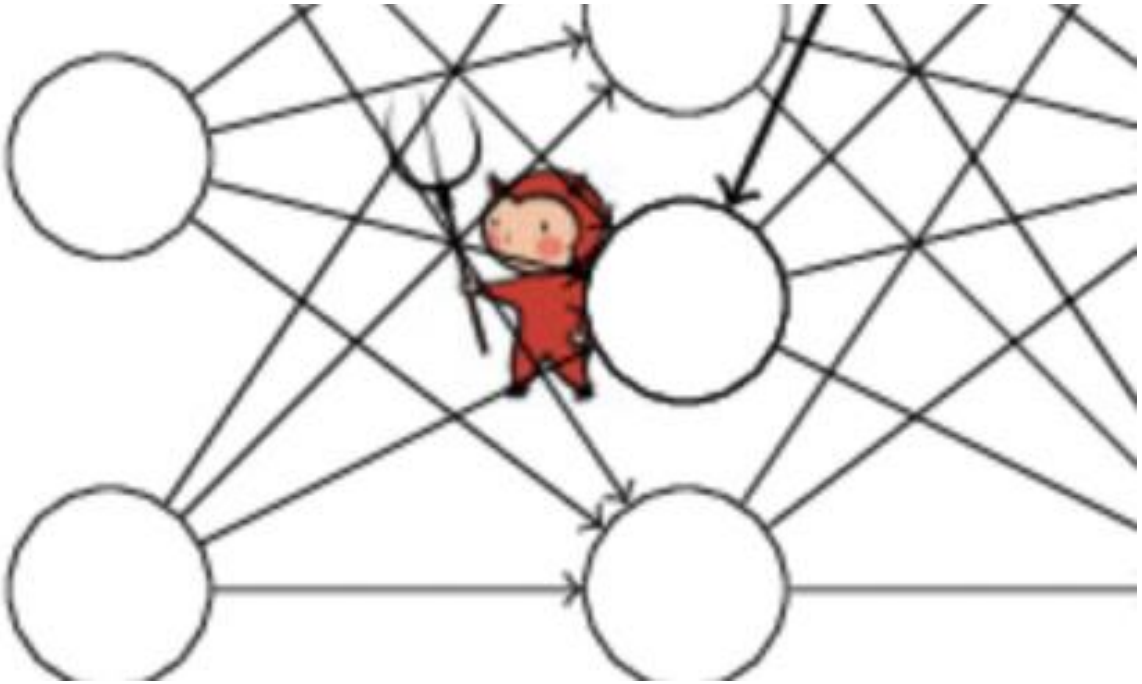
$$f(g(h(x)))$$

$$\frac{df}{dh} = \frac{df}{dg}\frac{dg}{dh}$$

$$\sigma(z_j^l) \rightarrow \sigma(z_j^l + \Delta z_j^l)$$

Our goal is to work out how to nudge weights to improve our error





$$\sigma(z_j^l) \rightarrow \sigma(z_j^l + \Delta z_j^l)$$

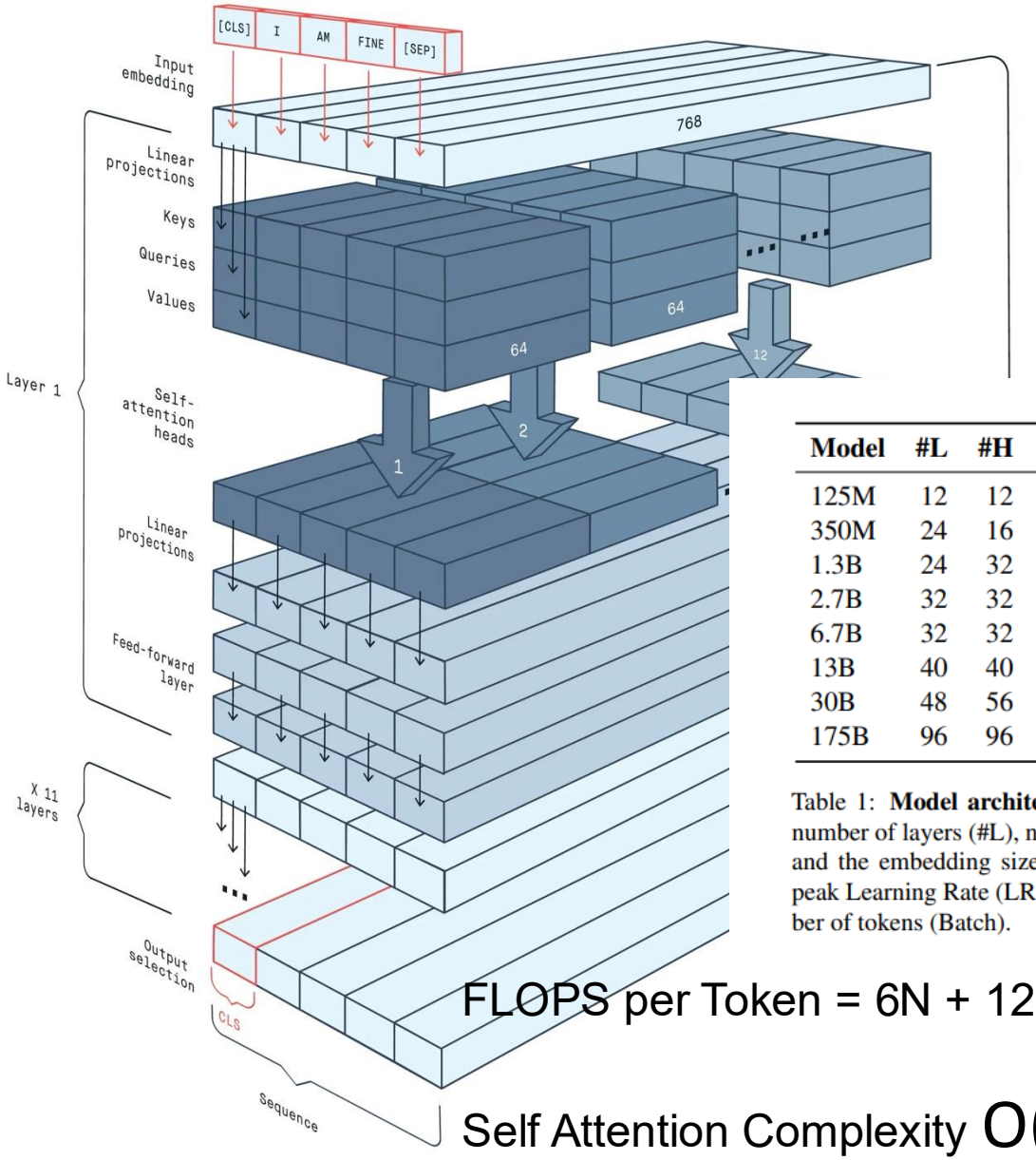
$$\Delta w = -\eta \nabla L$$

Small change to the weight is
learning rate times the gradient of Loss

The basic update operation

$$w_k \rightarrow w'_k = w_k - \frac{\eta}{m} \sum_j \frac{\partial L_{x_j}}{\partial w_k}$$

The Transformer with Self-Attention

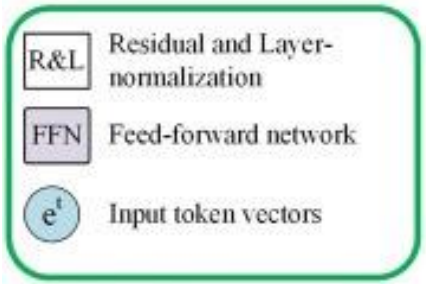


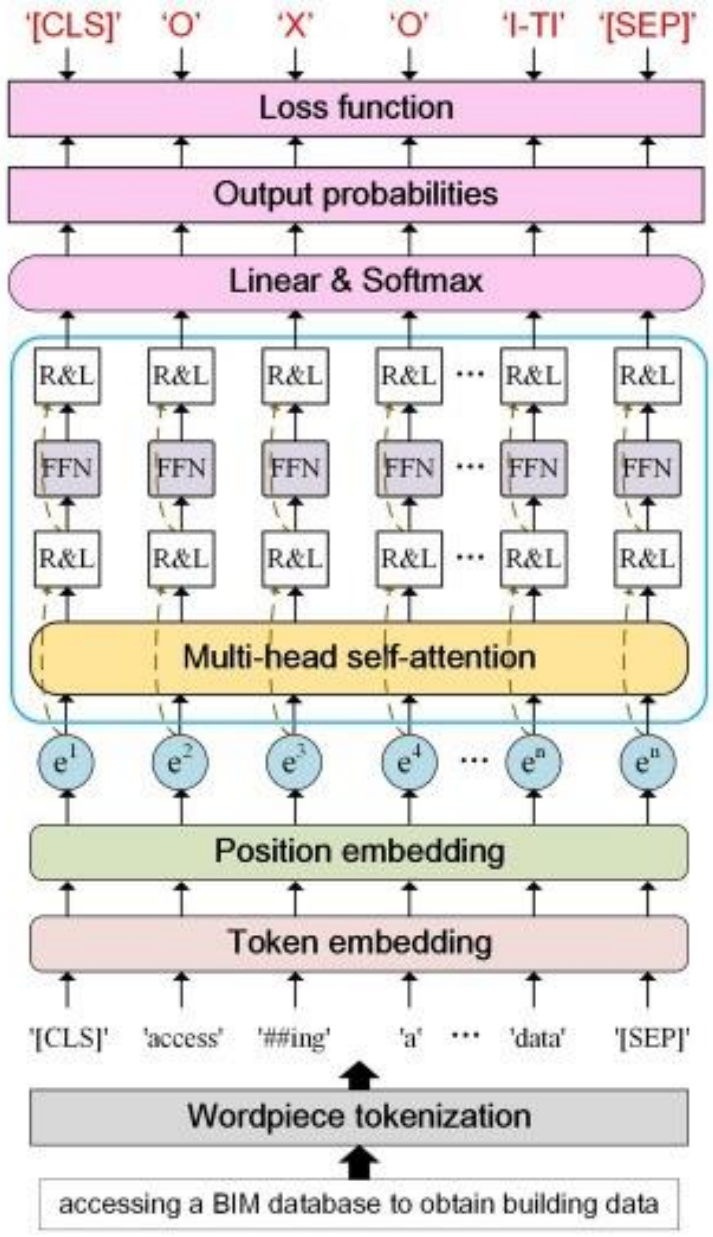
Model	#L	#H	d_{model}	LR	Batch
125M	12	12	768	$6.0e{-4}$	0.5M
350M	24	16	1024	$3.0e{-4}$	0.5M
1.3B	24	32	2048	$2.0e{-4}$	1M
2.7B	32	32	2560	$1.6e{-4}$	1M
6.7B	32	32	4096	$1.2e{-4}$	2M
13B	40	40	5120	$1.0e{-4}$	4M
30B	48	56	7168	$1.0e{-4}$	4M
175B	96	96	12288	$1.2e{-4}$	2M

Table 1: **Model architecture details.** We report the number of layers (#L), number of attention heads (#H), and the embedding size (d_{model}). We also report the peak Learning Rate (LR) and global batch size in number of tokens (Batch).

FLOPS per Token = 6N + 12LHQT

Self Attention Complexity $O(n^2d + nd^2)$



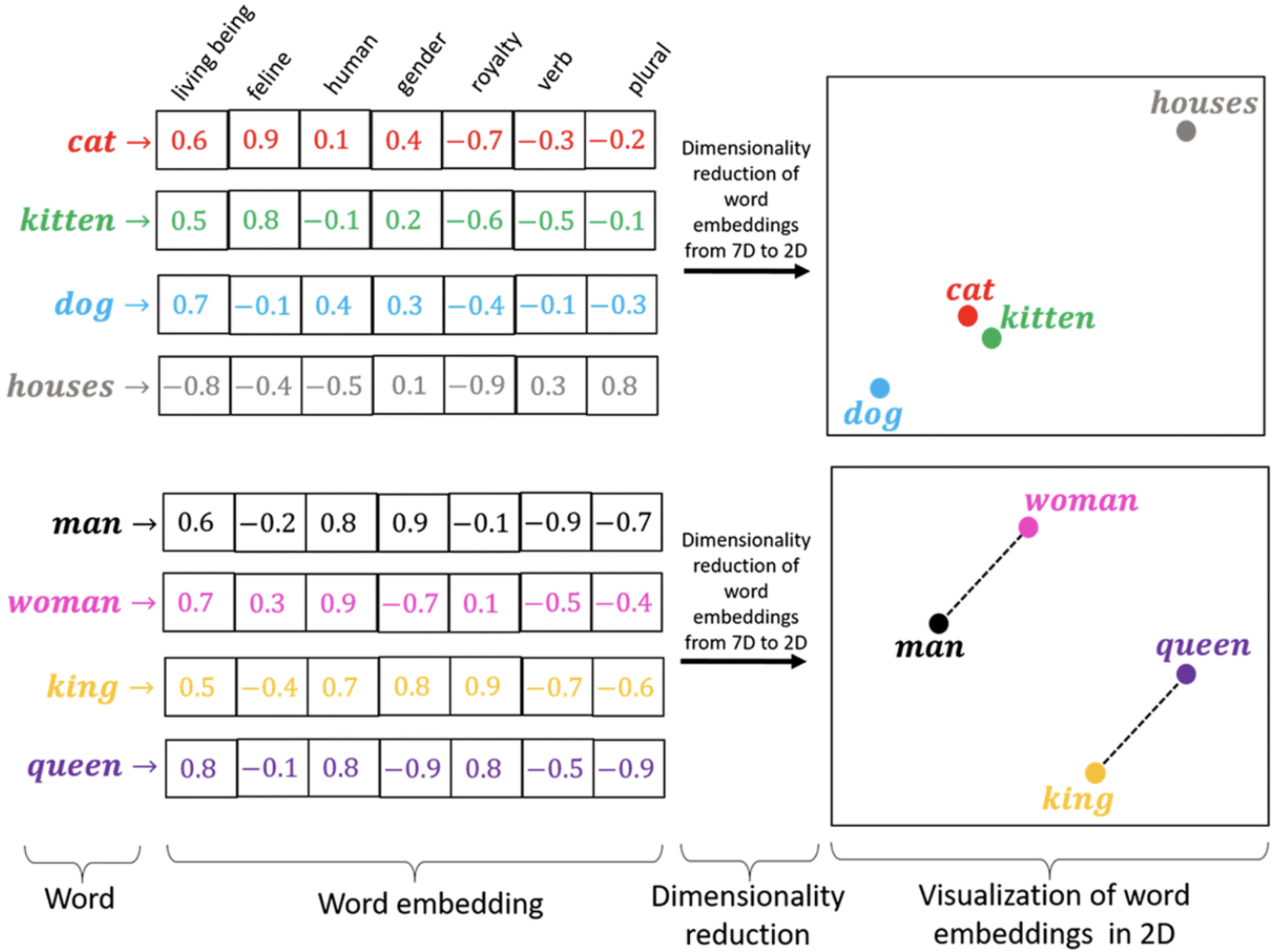


Layers, Heads, Dimension, Width

Word embedding

A word like bank, called a **token** when it represents a fundamental piece of text, is commonly encoded as a vector of real, continuous values: the **embedding vector**.

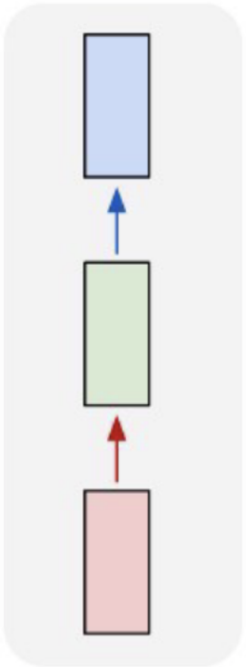
Determining the values inside the embedding vector of a token is a large part of the heavy lifting in text processing. Thankfully, with hundreds of dimensions available to organize the vocabulary of known tokens, embeddings can be pretrained to relate numerically in ways that reflect how their tokens relate in natural language.


living being
feline
human
gender
royalty
verb
plural
cat → 0.6 0.9 0.1 0.4 −0.7 −0.3 −0.2
kitten → 0.5 0.8 −0.1 0.2 −0.6 −0.5 −0.1
dog → 0.7 −0.1 0.4 0.3 −0.4 −0.1 −0.3
houses → −0.8 −0.4 −0.5 0.1 −0.9 0.3 0.8
Dimensionality reduction of word embeddings from 7D to 2D
houses
cat
kitten
dog
man → 0.6 −0.2 0.8 0.9 −0.1 −0.9 −0.7
woman → 0.7 0.3 0.9 −0.7 0.1 −0.5 −0.4
king → 0.5 −0.4 0.7 0.8 0.9 −0.7 −0.6
queen → 0.8 −0.1 0.8 −0.9 0.8 −0.5 −0.9
Dimensionality reduction of word embeddings from 7D to 2D
woman
man
queen
king
Word
Word embedding
Dimensionality reduction
Visualization of word embeddings in 2D

Sequence to sequence models

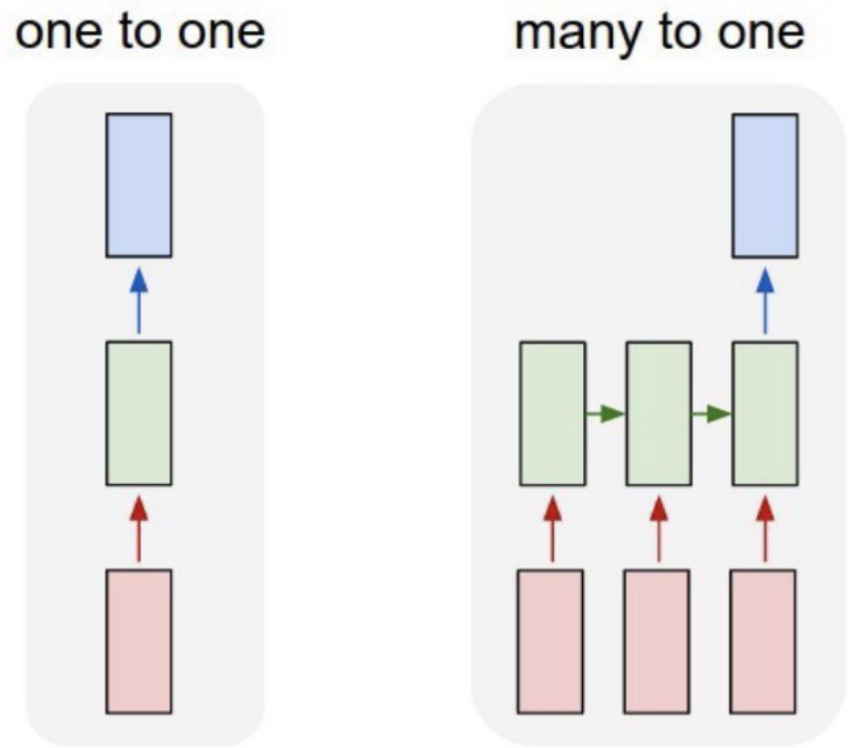
How to model sequential data?

one to one



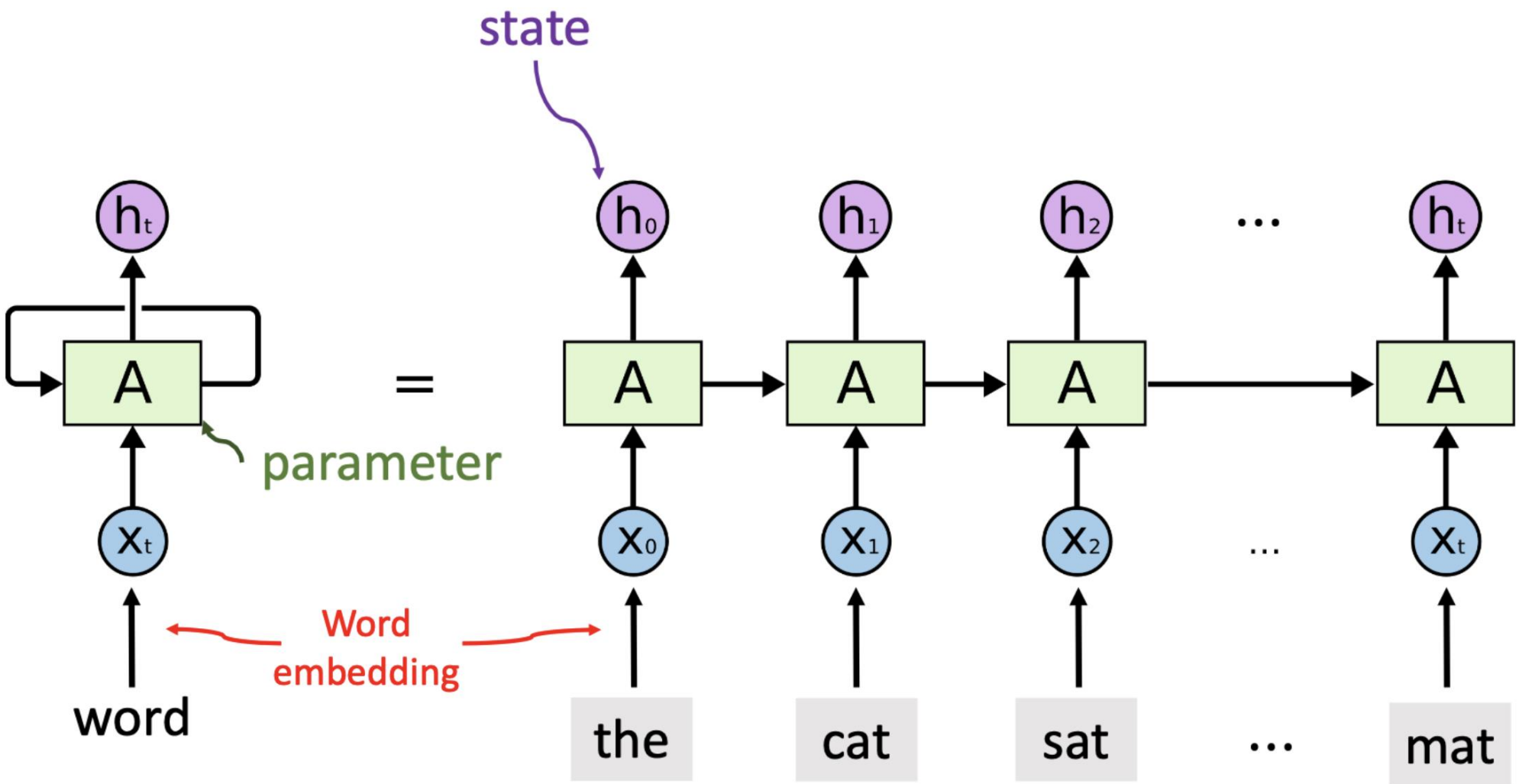
- Limitations of FC Nets and ConvNets:
  - Process a paragraph as a whole.
  - Fixed-size input (e.g., image).
  - Fixed-size output (e.g., predicted probabilities).

How to model sequential data?

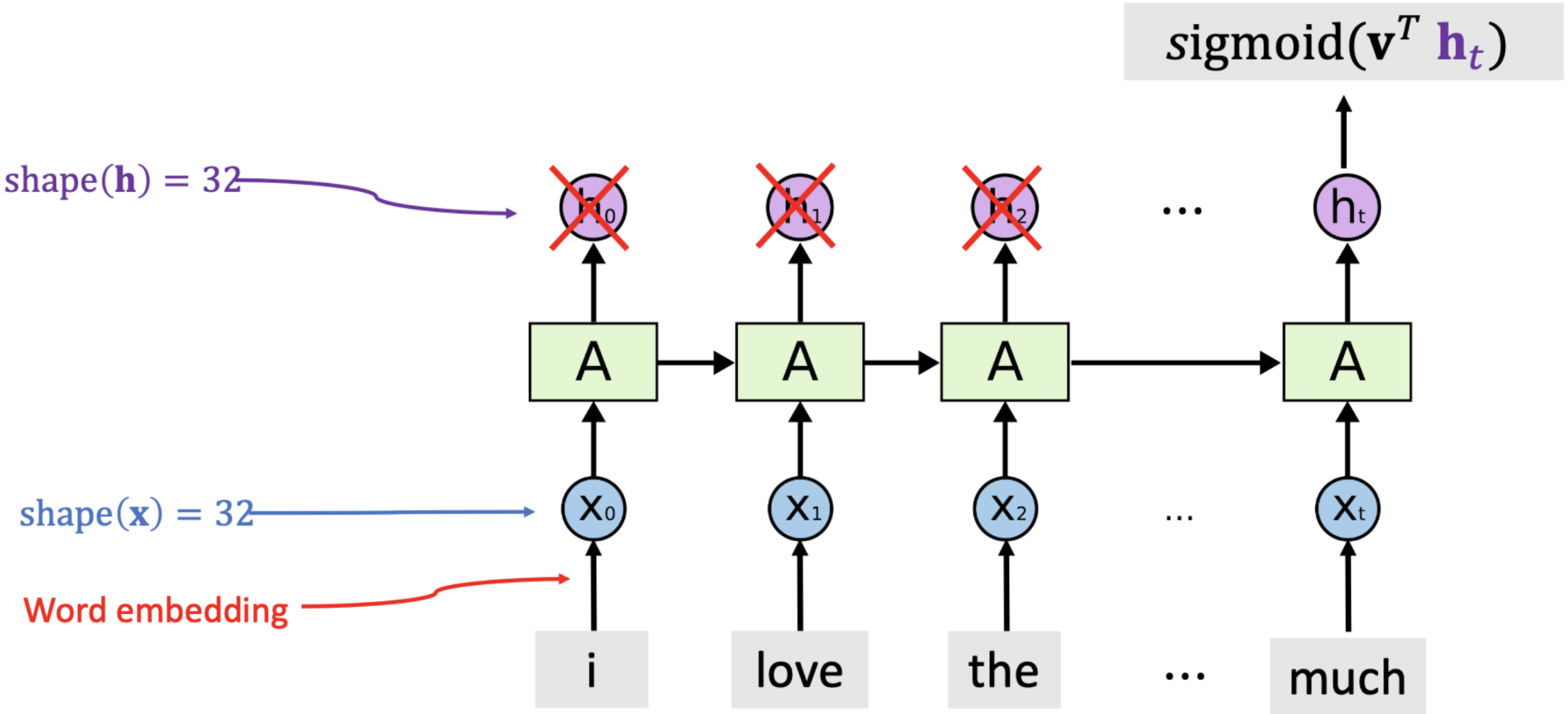


- Limitations of FC Nets and ConvNets:
  - Process a paragraph as a whole.
  - Fixed-size input (e.g., image).
  - Fixed-size output (e.g., predicted probabilities).
- RNNs are better ways to model the sequential data (e.g., text, speech, and time series).

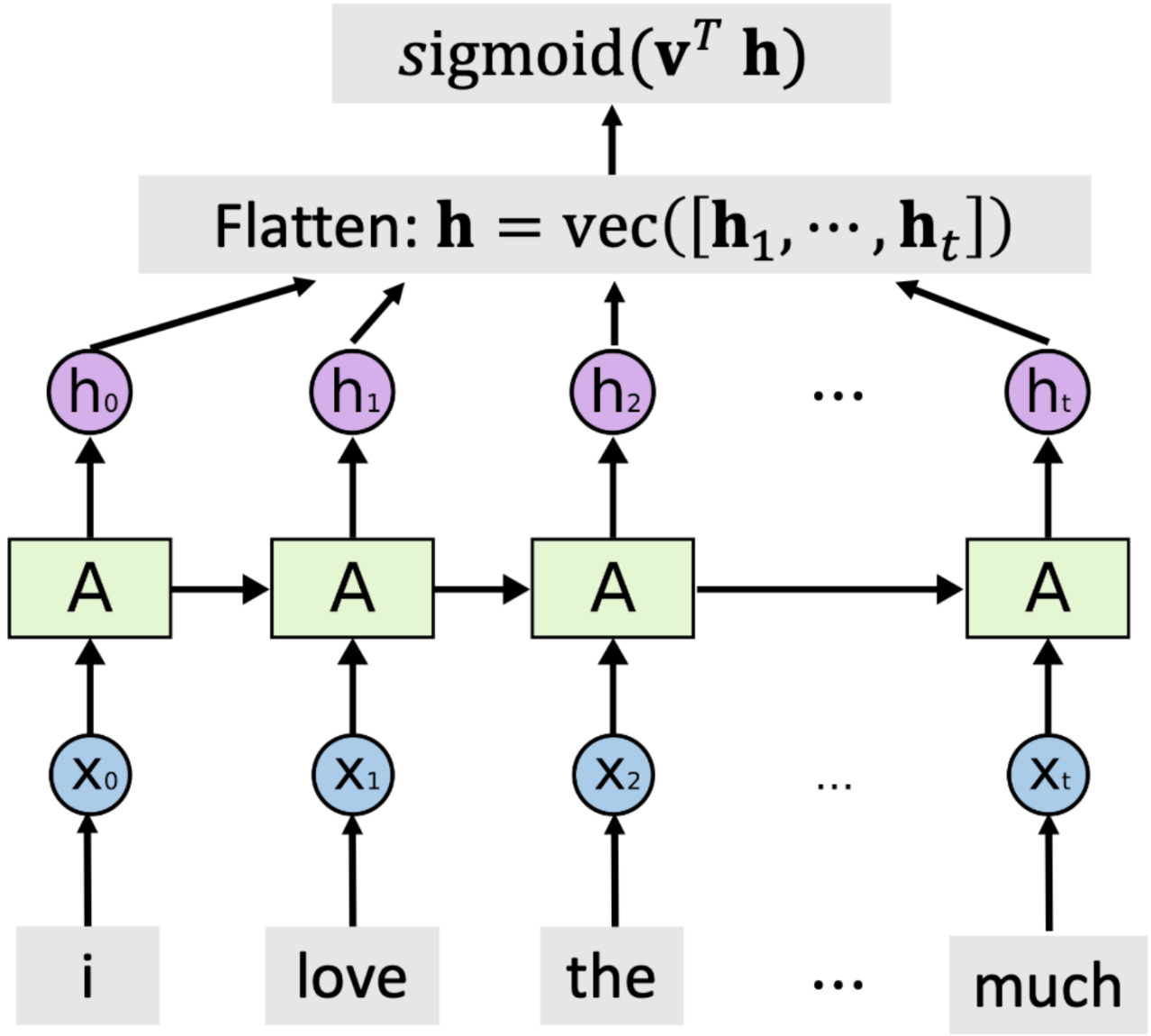
Recurrent Neural Networks (RNNs)



Simple RNN for IMDB Review

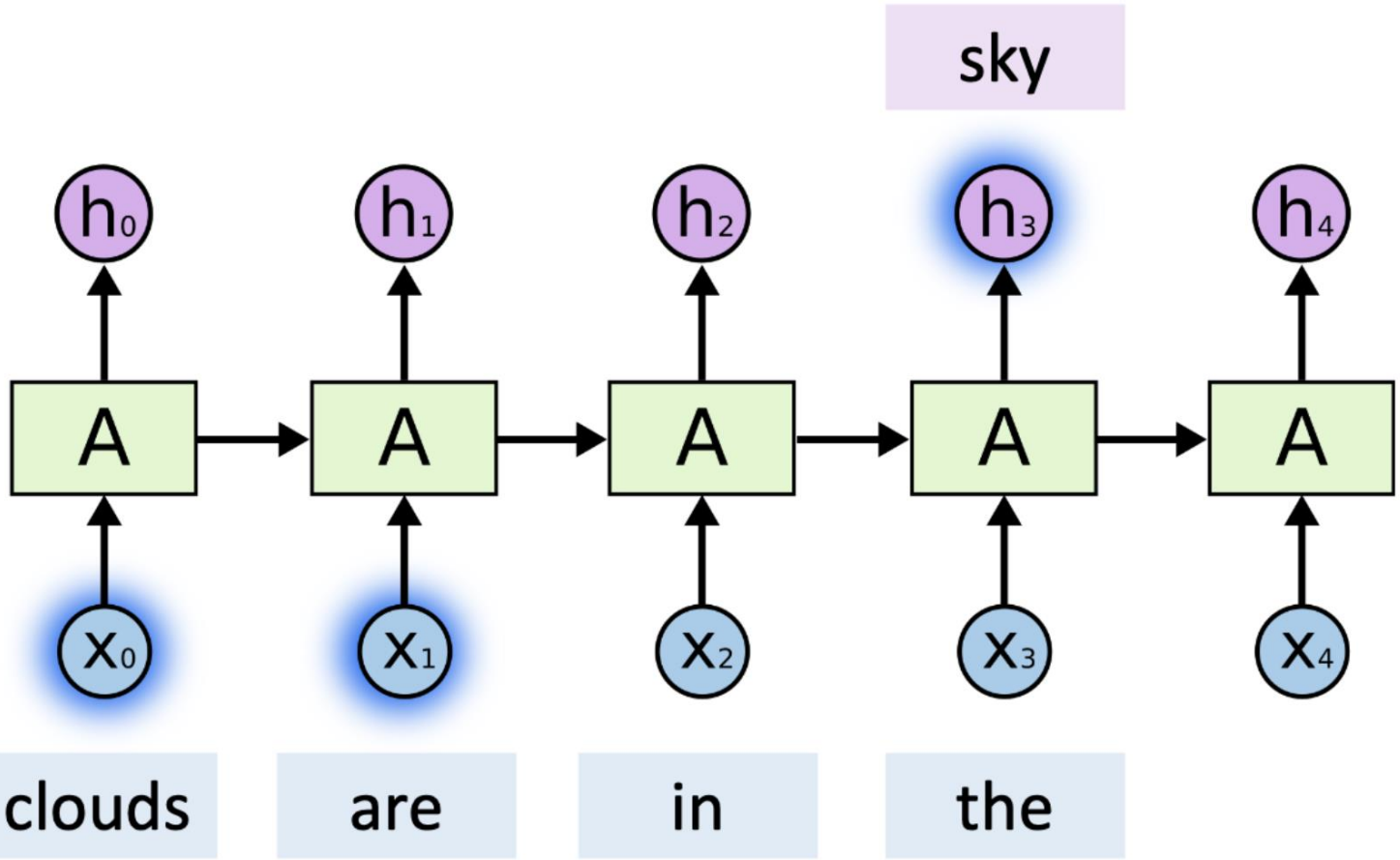


Simple RNN for IMDB Review



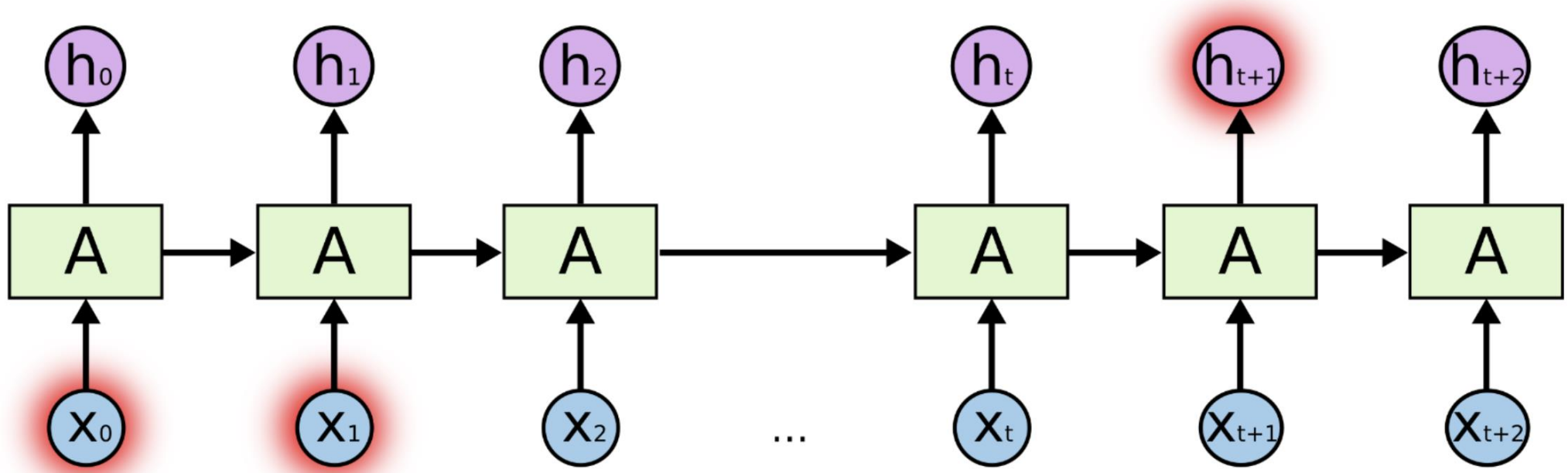
SimpleRNN is good at short-term dependence.

Predicted next words:



Figures are from Christopher Olah's blog: Understanding LSTM Networks.

SimpleRNN is bad at long-term dependence.

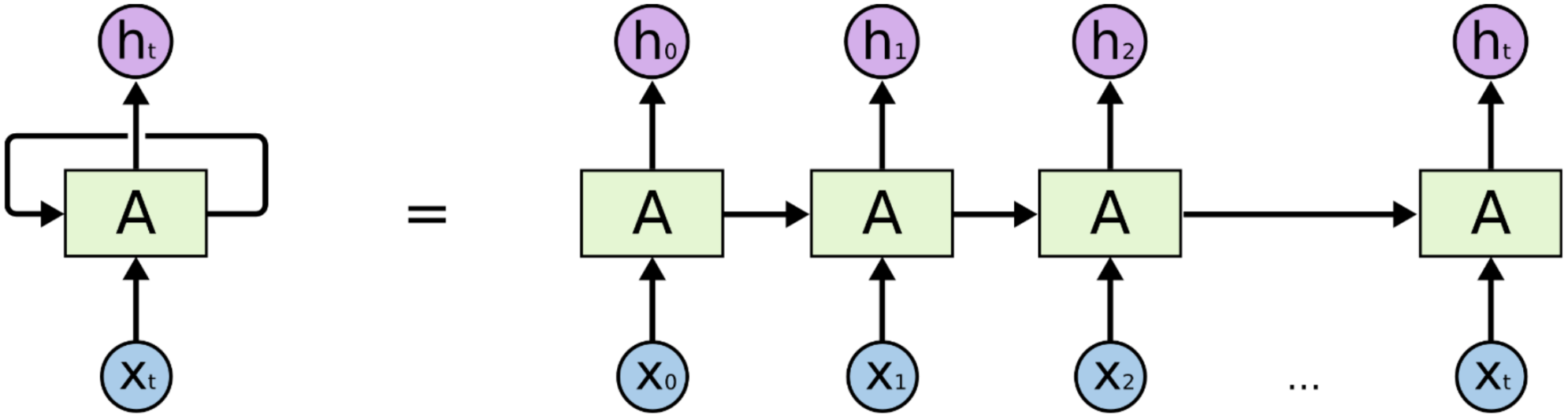


$\mathbf{h}_{100}$ is almost irrelevant to $\mathbf{x}_1$: $\frac{\partial \mathbf{h}_{100}}{\partial \mathbf{x}_1}$ is near zero.

Figures are from Christopher Olah's blog: Understanding LSTM Networks.

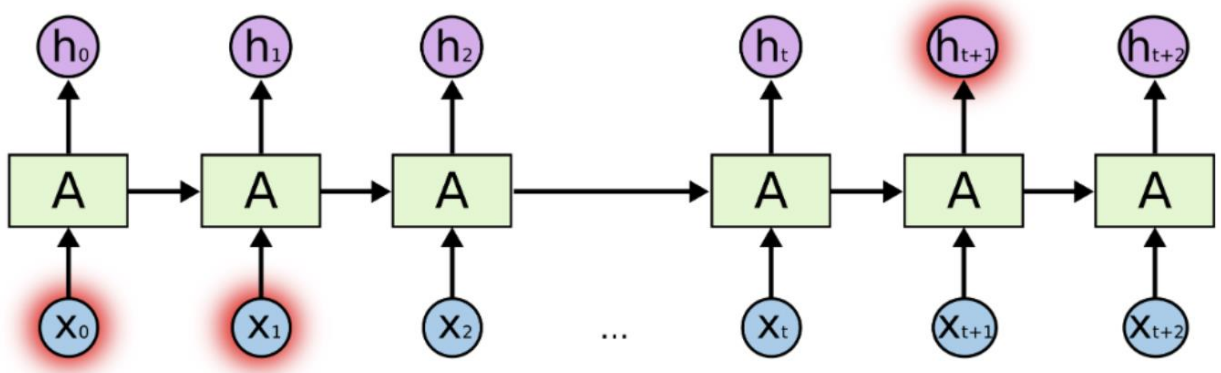
Summary

- RNN for text, speech, and time series data.
- Hidden state $\mathbf{h}_t$ aggregates information in the inputs $\mathbf{x}_0, \cdots, \mathbf{x}_t$.



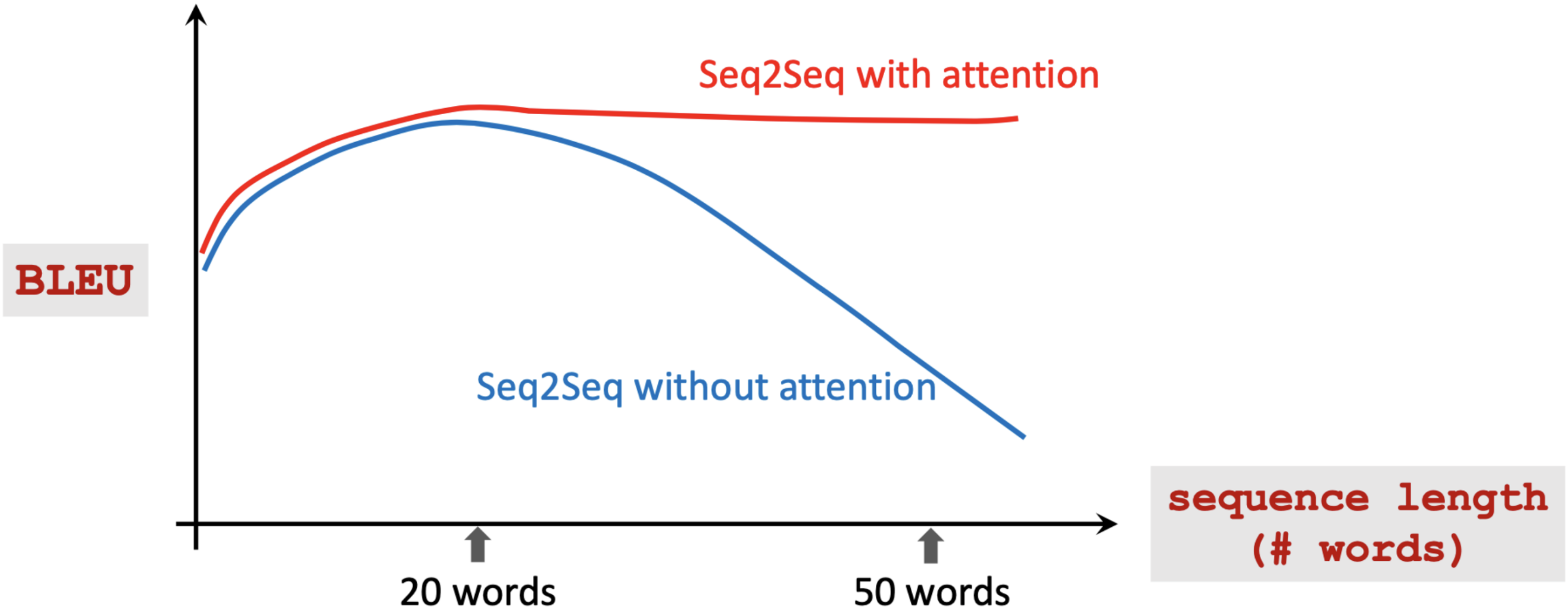
Summary

- RNN for text, speech, and time series data.
- Hidden state $\mathbf{h}_t$ aggregates information in the inputs $\mathbf{x}_0, \cdots, \mathbf{x}_t$.
- RNNs can forget early inputs.
  - It forgets what it has seen early on.
  - If t is large, $\mathbf{h}_t$ is almost irrelevant to $\mathbf{x}_0$.



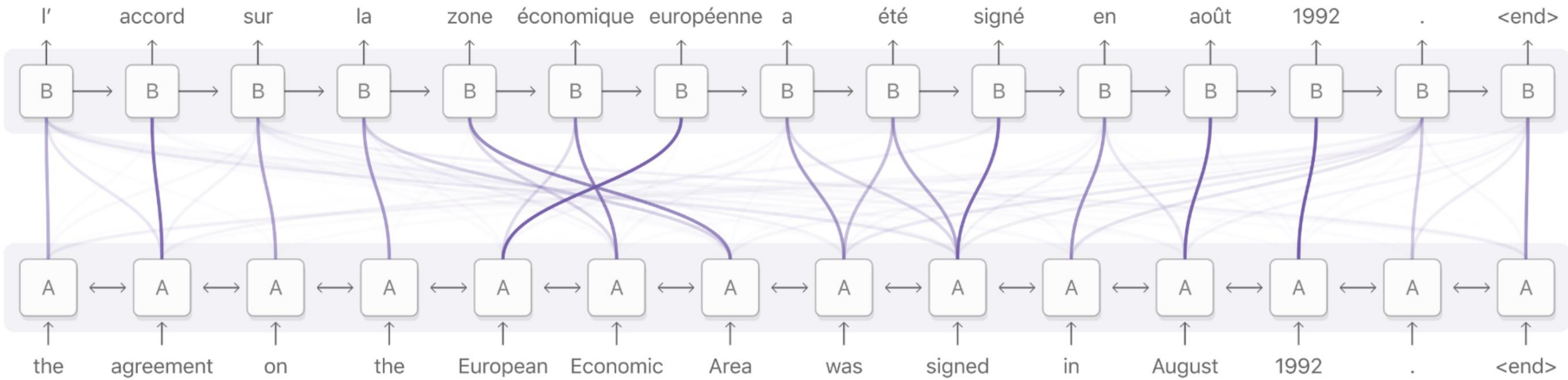
Seq2Seq Model

Shortcoming: The final state is incapable of remembering a **long** sequence.



Attention: Weights Visualization

Decoder RNN (target language: French)

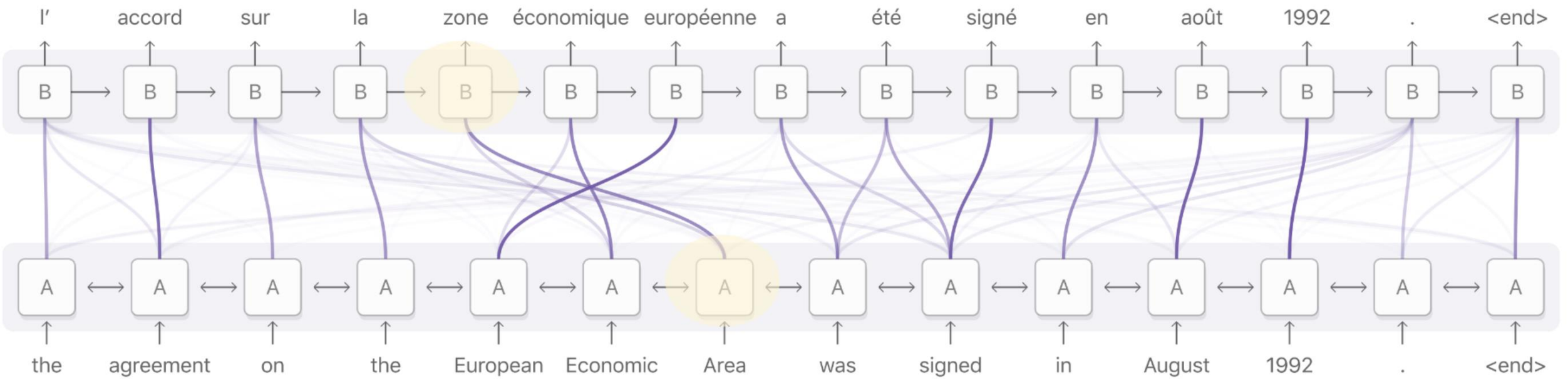


Encoder RNN (source language: English)

Figure is from https://distill.pub/2016/augmented-rnns/

Attention: Weights Visualization

Decoder RNN (target language: French)



Encoder RNN (source language: English)

Figure is from https://distill.pub/2016/augmented-rnns/

Seq2Seq Model with Attention

- Attention tremendously improves Seq2Seq model.
- With attention, Seq2Seq model does not forget source input.
- With attention, the decoder knows where to focus.
- Downside: much more computation.

Original paper:

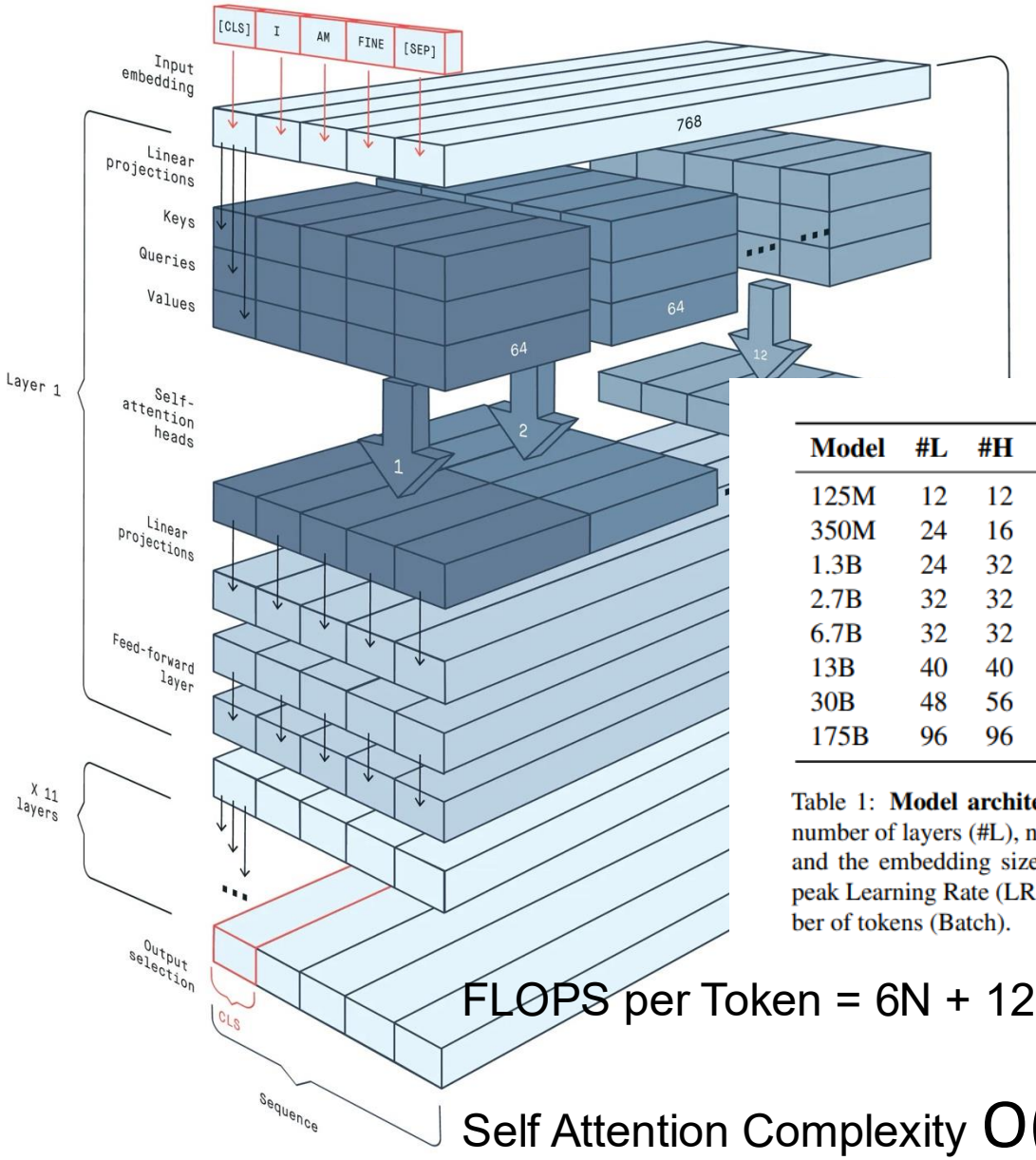
- Bahdanau, Cho, & Bengio. Neural machine translation by jointly learning to align and translate. In *ICLR*, 2015.

Summary

- Standard Seq2Seq model: the decoder looks at only its current state.
- Attention: decoder additionally looks at all the states of the encoder.
- Attention: decoder knows where to focus.

- Downside: higher time complexity.
  - m: source sequence length
  - t: target sequence length
  - Standard Seq2Seq: $O(m + t)$ time complexity
  - Seq2Seq + attention: $O(mt)$ time complexity

The Transformer with Self-Attention

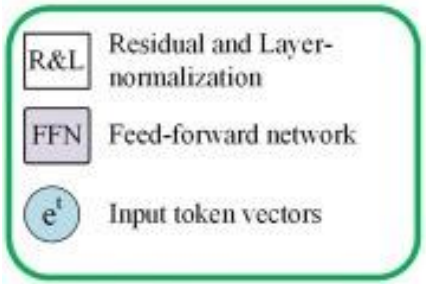


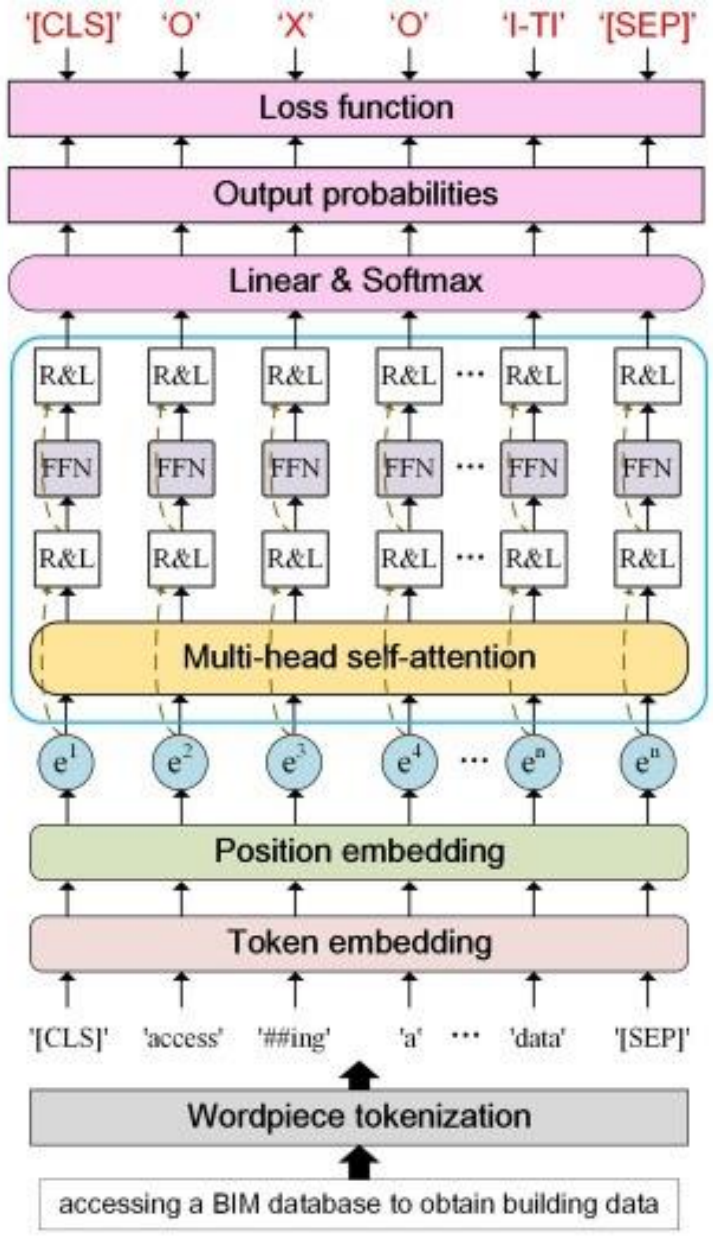
Model	#L	#H	d_{model}	LR	Batch
125M	12	12	768	$6.0e{-4}$	0.5M
350M	24	16	1024	$3.0e{-4}$	0.5M
1.3B	24	32	2048	$2.0e{-4}$	1M
2.7B	32	32	2560	$1.6e{-4}$	1M
6.7B	32	32	4096	$1.2e{-4}$	2M
13B	40	40	5120	$1.0e{-4}$	4M
30B	48	56	7168	$1.0e{-4}$	4M
175B	96	96	12288	$1.2e{-4}$	2M

Table 1: **Model architecture details.** We report the number of layers (#L), number of attention heads (#H), and the embedding size (d_{model}). We also report the peak Learning Rate (LR) and global batch size in number of tokens (Batch).

FLOPS per Token = 6N + 12LHQT

Self Attention Complexity $O(n^2d + nd^2)$





Layers, Heads, Dimension, Width

arXiv:1706.03762v5 [cs.CL] 6 Dec 2017

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

1 Introduction

Recurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

†Work performed while at Google Brain.

‡Work performed while at Google Research.

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

Transformer

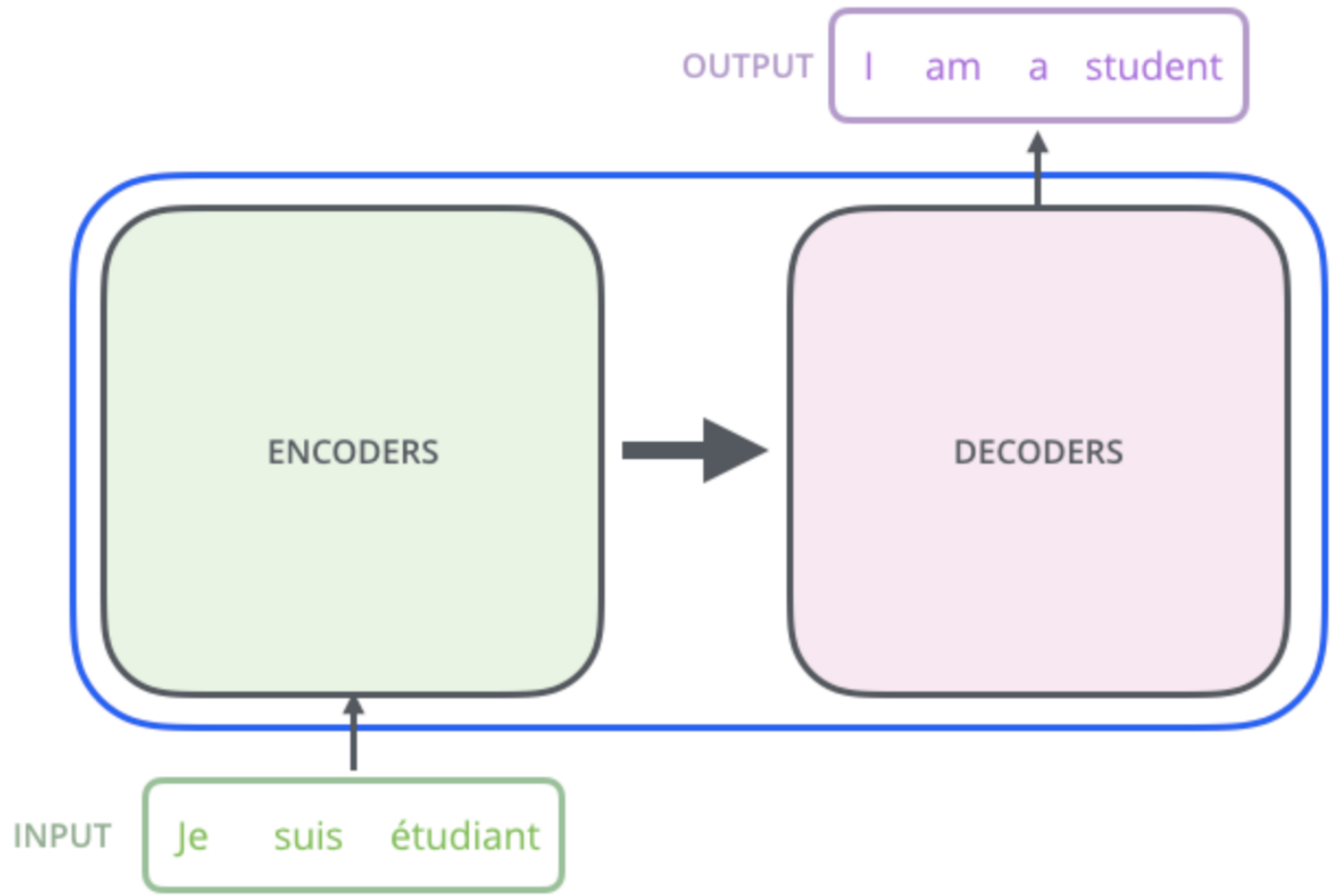
"Attention is All you Need" (Vaswani, et al., 2017), without a doubt, is one of the most impactful and interesting paper in 2017. It presented a lot of improvements to the soft attention and make it possible to do seq2seq modeling *without* recurrent network units. The proposed "**transformer**" model is entirely built on the self-attention mechanisms without using sequence-aligned recurrent architecture.

A High-Level Look

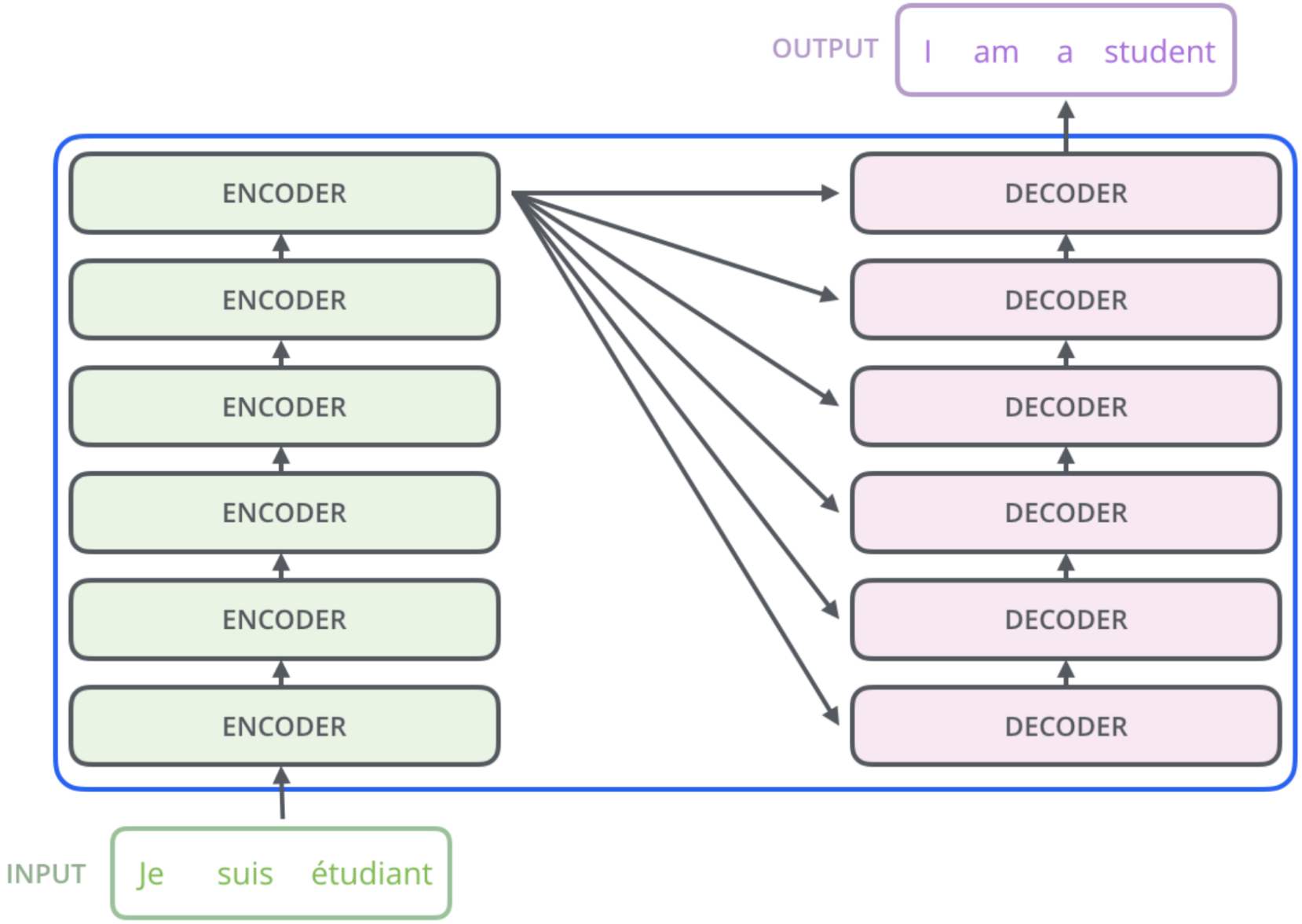
Let's begin by looking at the model as a single black box. In a machine translation application, it would take a sentence in one language, and output its translation in another.



Popping open that Optimus Prime goodness, we see an encoding component, a decoding component, and connections between them.



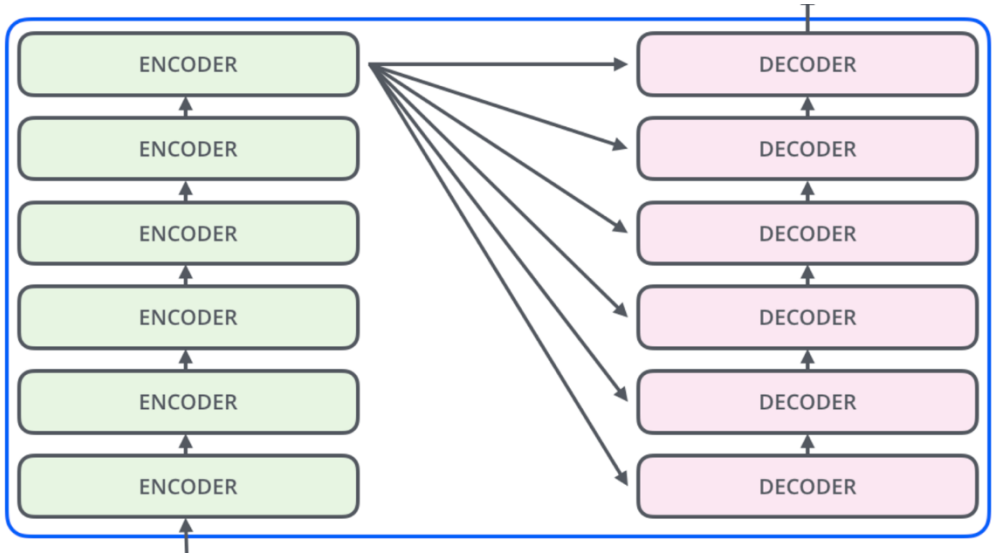
The encoding component is a stack of encoders (the paper stacks six of them on top of each other – there's nothing magical about the number six, one can definitely experiment with other arrangements). The decoding component is a stack of decoders of the same number.



Encoder-Decoder architecture

- Sequence to sequence models
- Encoder can see whole input (in the window)
- Decoder can see only to the left
- New sentences, summarization, translation, generative question answering

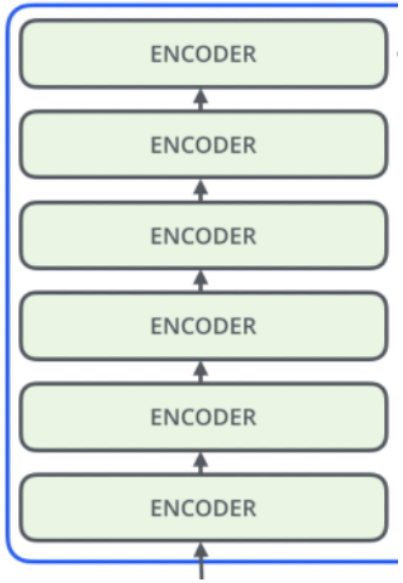
  - BART
  - mBART
  - Marian
  - T5



Encoder only, Decoder only

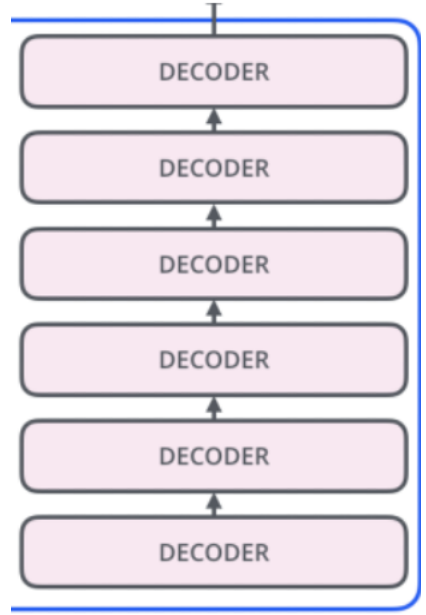
- Encoder only models
- Bidirectional
- Auto-encoding
- Classification, NER, Extractive QA

- ALBERT
- BERT
- DistilBERT
- ELECTRA
- RoBERTa

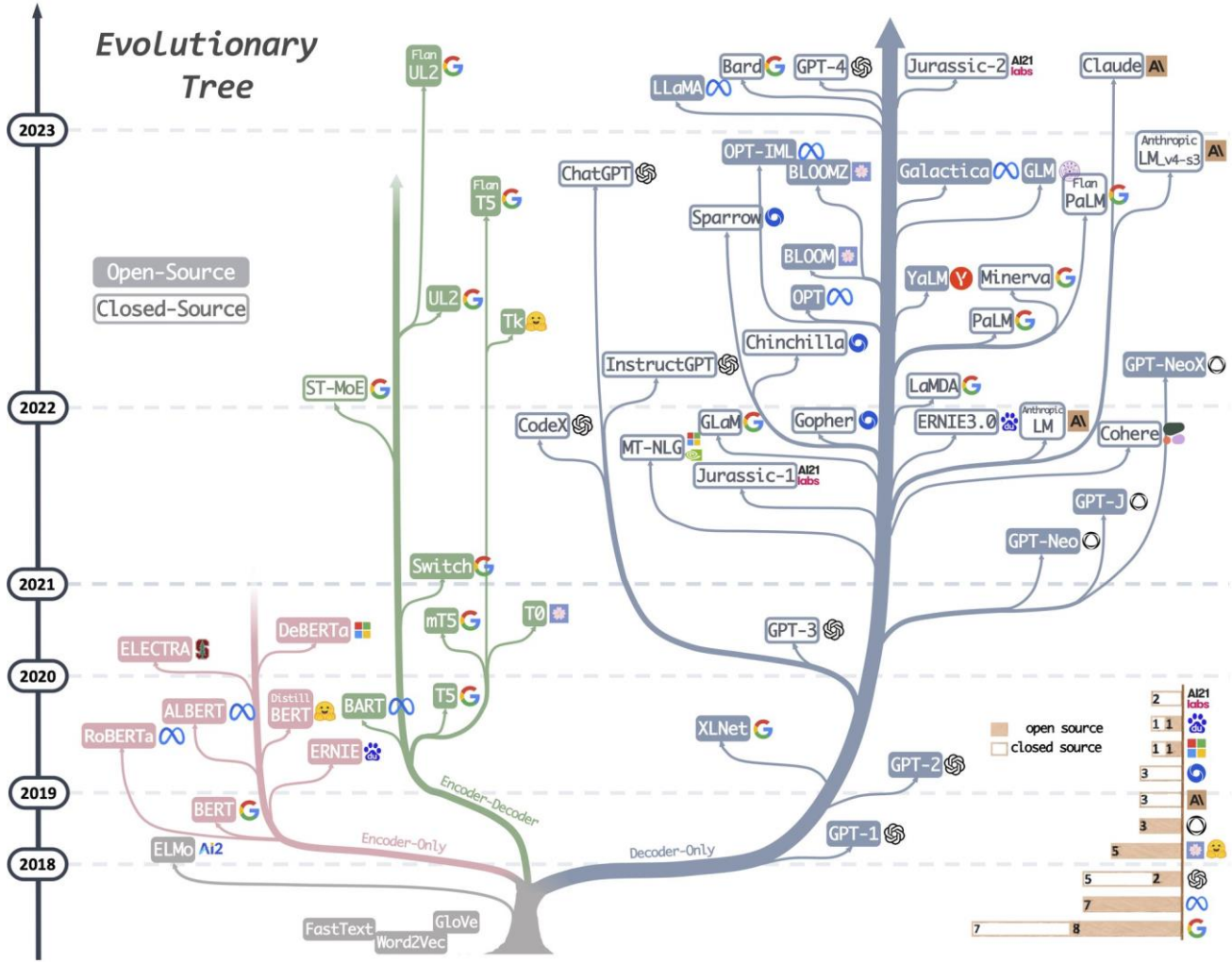


- Decoder only models
- Unidirectional
- Auto-regressive
- Text generation

- CTRL
- GPT
- GPT-2
- Transformer XL



Rapid Development of Large Language Models



Explosion of development of LLM based AI systems since 2019

Foundation Models are replacing narrow AI systems at a rapid pace

Foundation Models are the closest things that have yet been created that hint at the possibility of Artificial General Intelligence

Model	Examples	Tasks
Encoder	ALBERT, BERT, DistilBERT, ELECTRA, RoBERTa	Sentence classification, named entity recognition, extractive question answering
Decoder	CTRL, GPT, GPT-2, Transformer XL	Text generation
Encoder-decoder	BART, T5, Marian, mBART	Summarization, translation, generative question answering

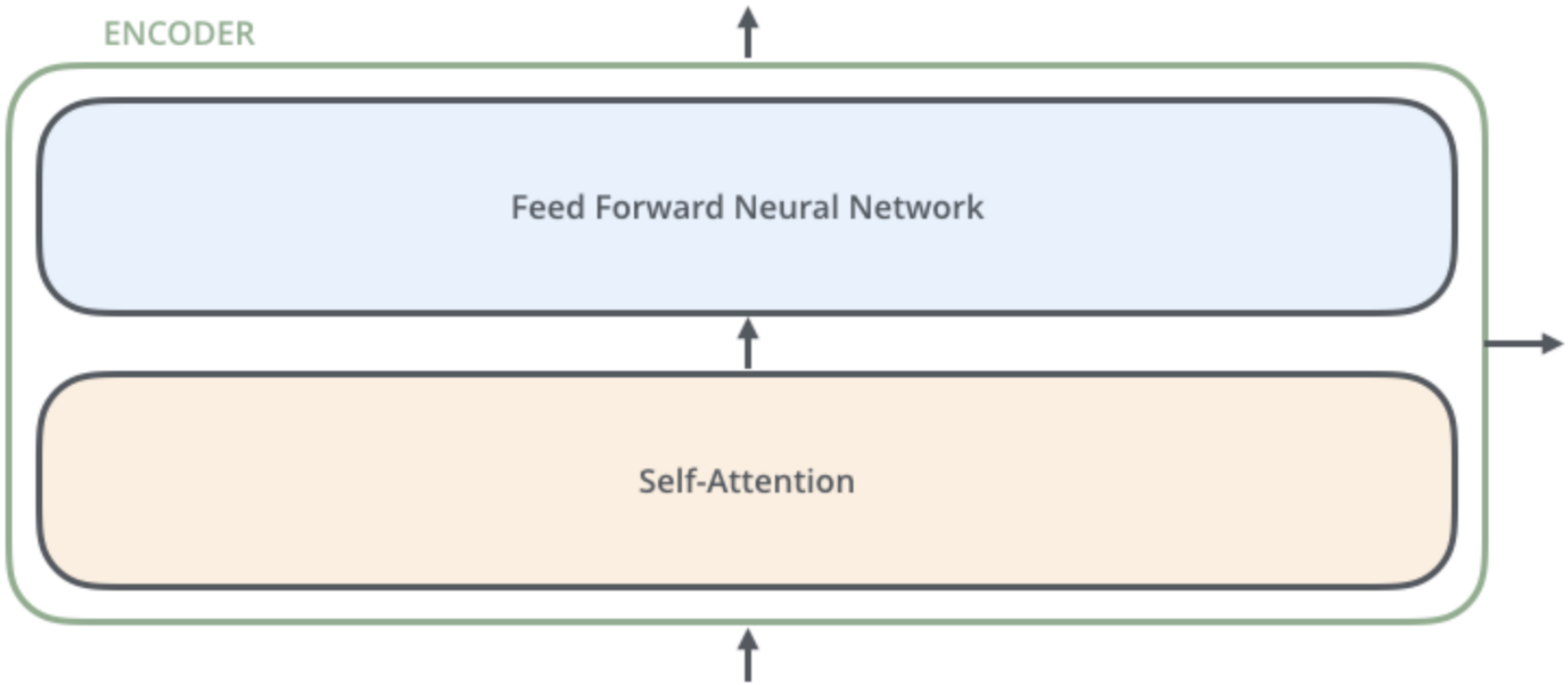
The rise of decoder-only Transformer models

Apart from the various interesting features of this model, one feature that catches the attention is its decoder-only architecture. In fact, not just PaLM, some of the most popular and widely used language models are decoder-only.

GPT-3 has demonstrated that large autoregressive language models can be used for few-shot predictions, and this class of models is generally trained with decoder-only architecture and a standard left-to-right language modelling objective on a large text corpus where the objective is to predict the next token, taking into account the previous tokens. Not just GPT-3, the previous versions, GPT and GPT-2, too, utilised a decoder only architecture.

Subsequently, researchers from Google Brain published a paper titled – Generating Wikipedia by Summarising Long Sequences. The study proposed another arrangement of transformer blocks for language modelling. This model omitted the encoder block. For this research, the team introduced a new decoder-only sequence transduction model for the abstractive stage. They demonstrated that the model is capable of handling very long input-output examples. This model outperformed traditional encoder-decoder architectures on long sequences, allowing the team to condition on many reference documents and generate coherent and informative Wikipedia articles.

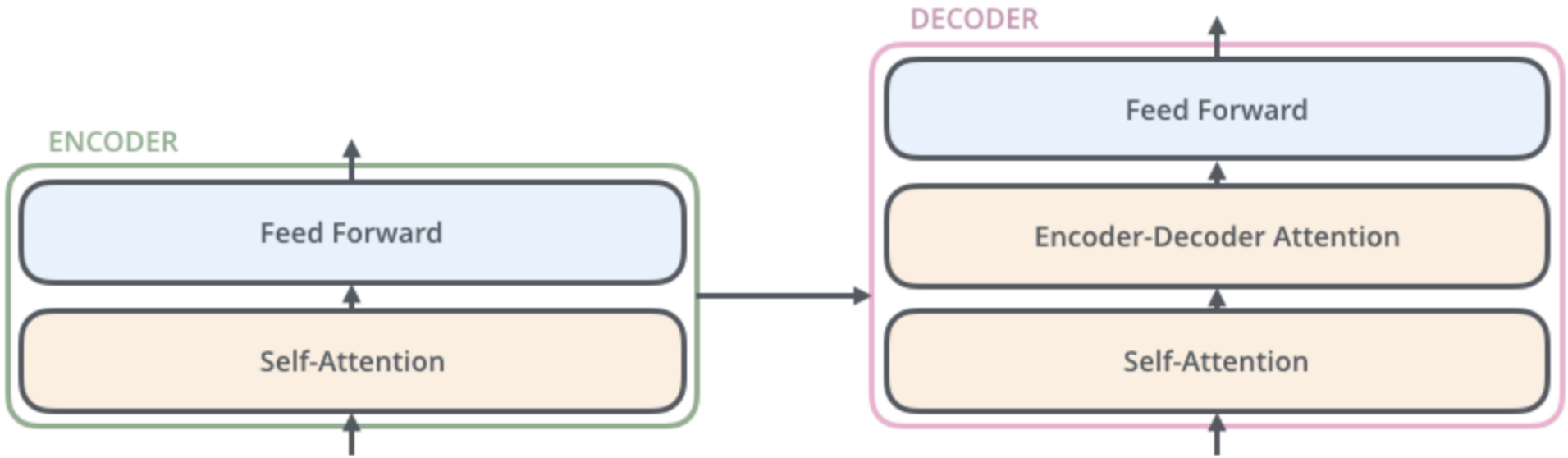
The encoders are all identical in structure (yet they do not share weights). Each one is broken down into two sub-layers:



The encoder's inputs first flow through a self-attention layer – a layer that helps the encoder look at other words in the input sentence as it encodes a specific word. We'll look closer at self-attention later in the post.

The outputs of the self-attention layer are fed to a feed-forward neural network. The exact same feed-forward network is independently applied to each position.

The decoder has both those layers, but between them is an attention layer that helps the decoder focus on relevant parts of the input sentence (similar what attention does in seq2seq models).



Full Architecture

Finally here is the complete view of the transformer's architecture:

- Both the source and target sequences first go through embedding layers to produce data of the same dimension $d_{\text{model}} = 512$.
- To preserve the position information, a sinusoid-wave-based positional encoding is applied and summed with the embedding output.
- A softmax and linear layer are added to the final decoder output.

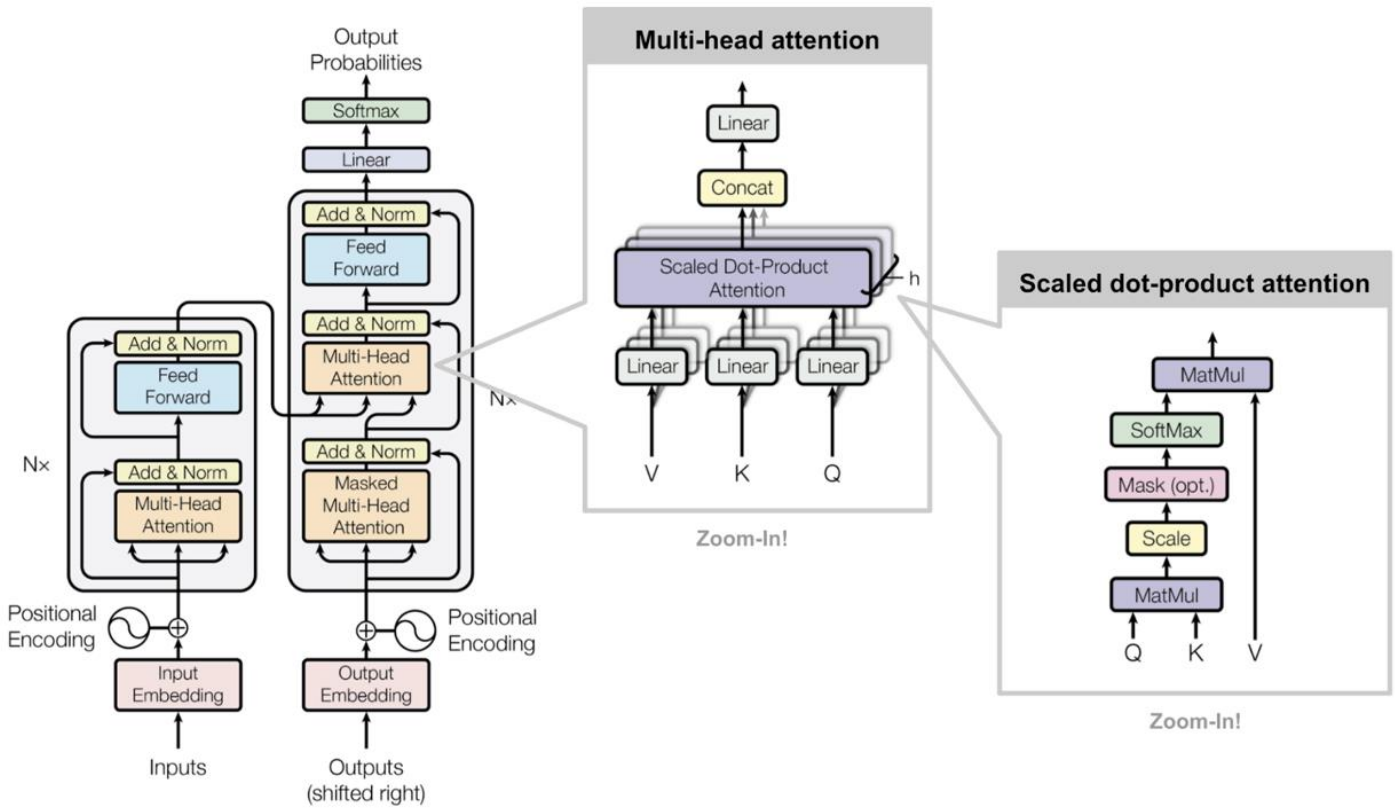
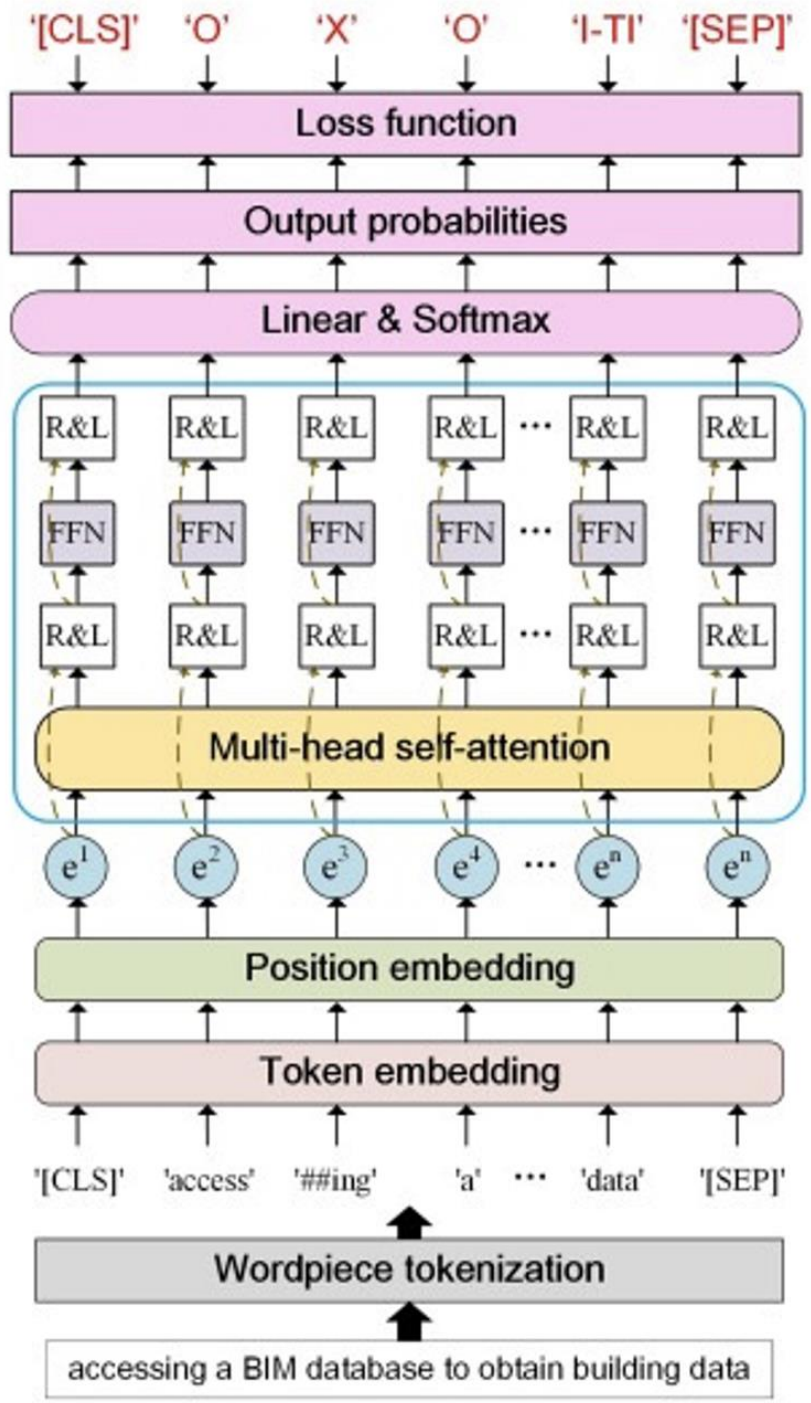
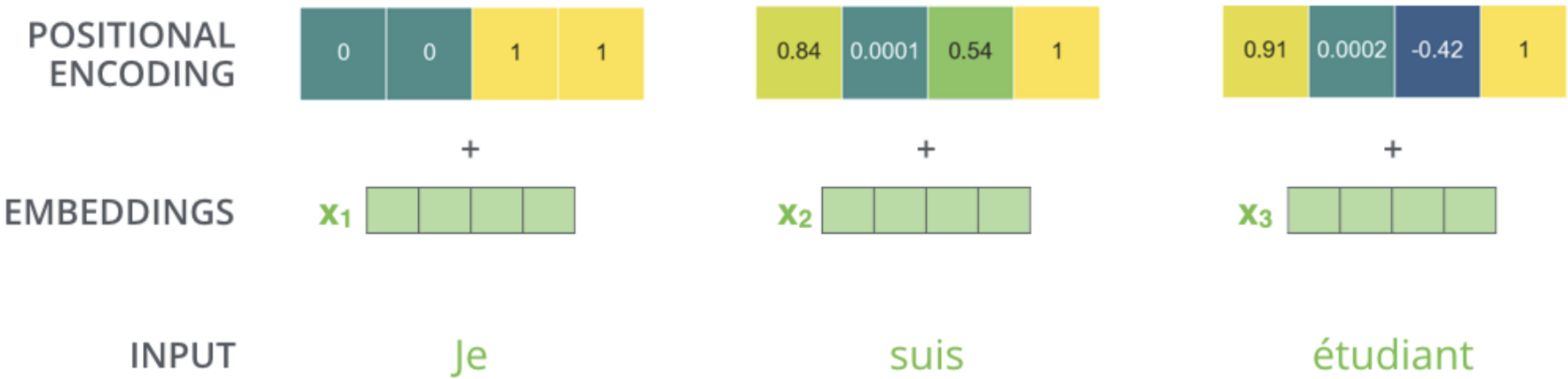


Fig. 17. The full model architecture of the transformer. (Image source: Fig 1 & 2 in Vaswani, et al., 2017.)


'[CLS]'
'O'
'X'
'O'
'I-TI'
'[SEP]'
Loss function
Output probabilities
Linear & Softmax
R&L
FFN
Multi-head self-attention
Position embedding
Token embedding
'[CLS]'
'access'
'##ing'
'a'
'data'
'[SEP]'
Wordpiece tokenization
accessing a BIM database to obtain building data

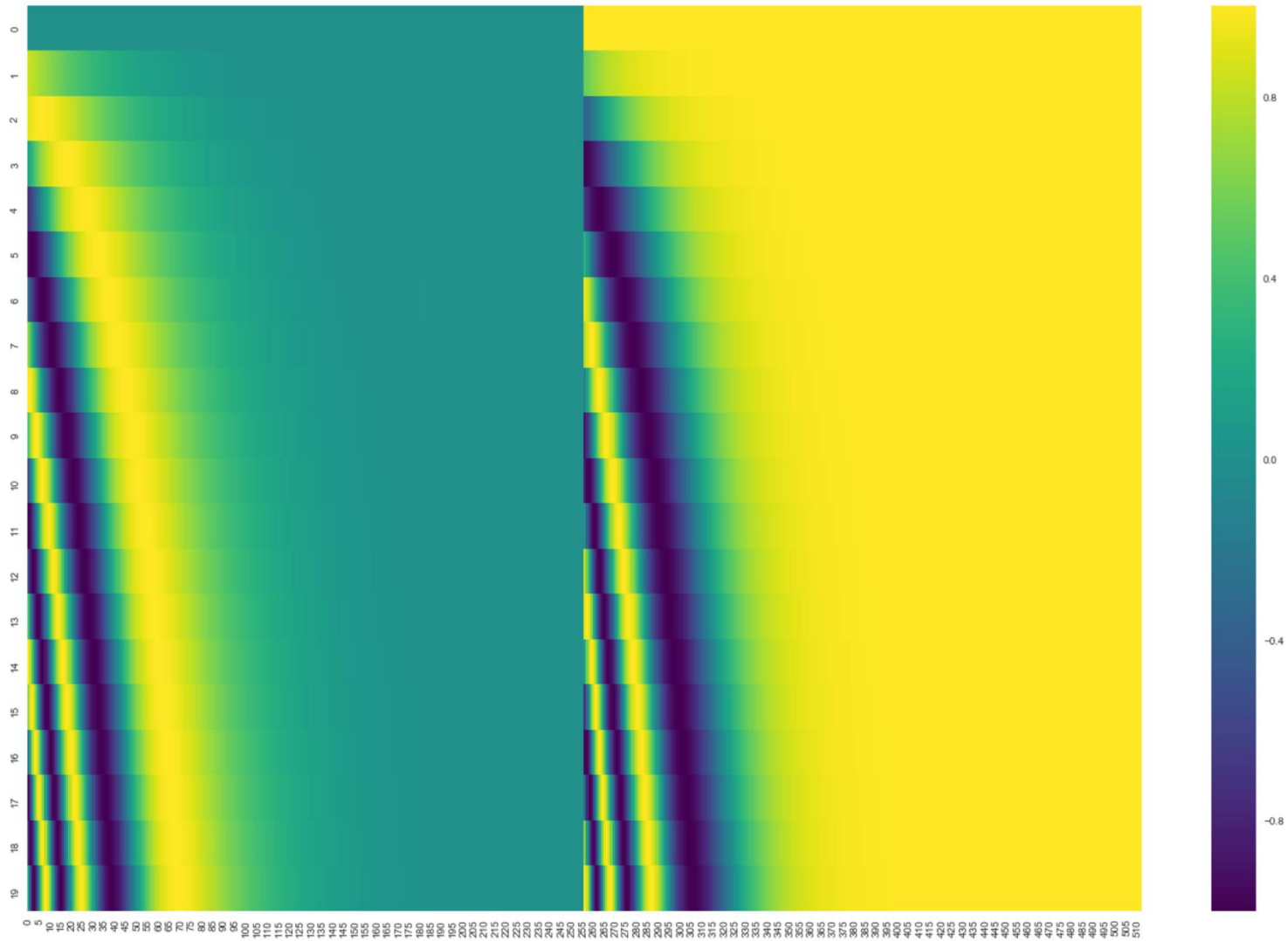
If we assumed the embedding has a dimensionality of 4, the actual positional encodings would look like this:



A real example of positional encoding with a toy embedding size of 4

What might this pattern look like?

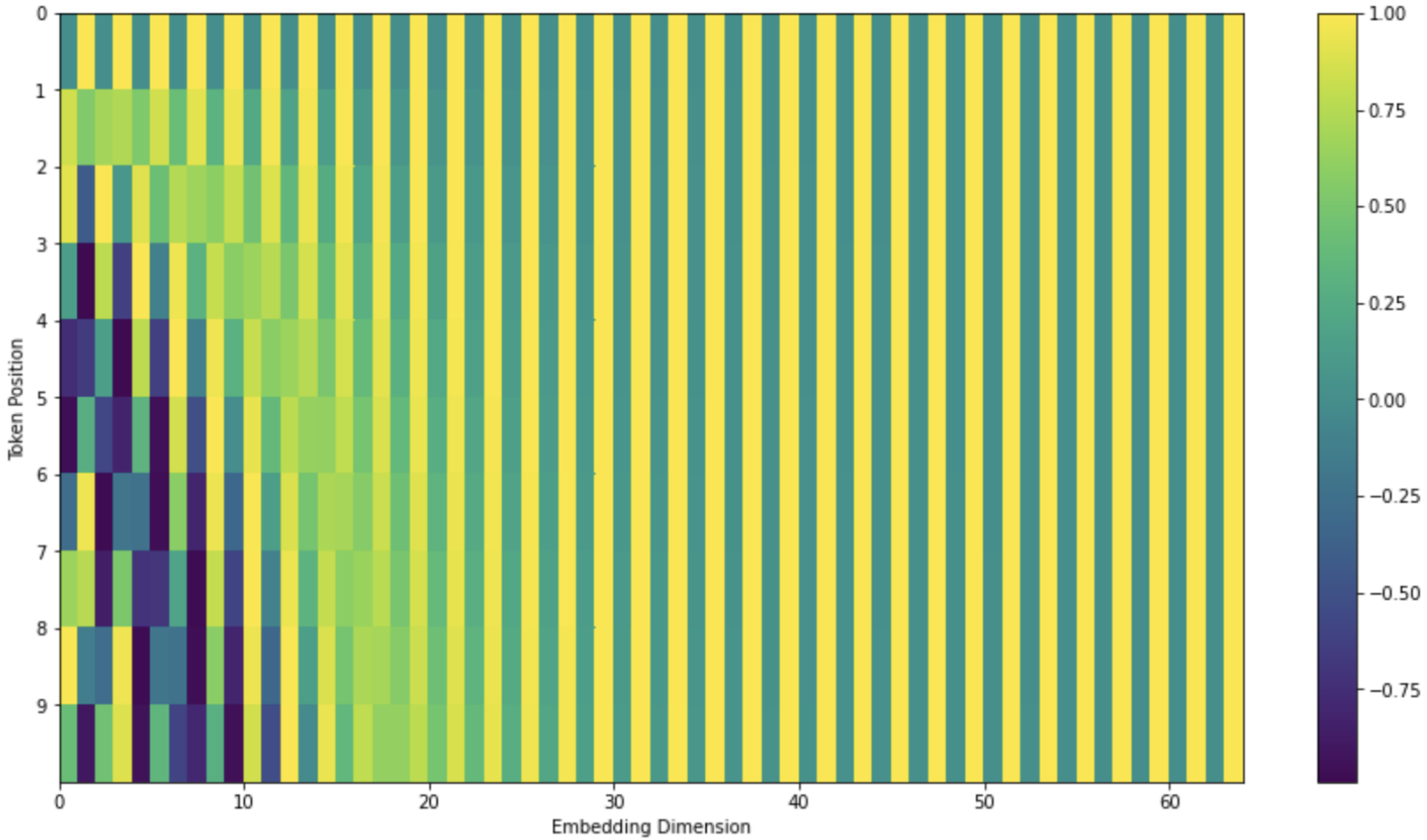
In the following figure, each row corresponds the a positional encoding of a vector. So the first row would be the vector we'd add to the embedding of the first word in an input sequence. Each row contains 512 values – each with a value between 1 and -1. We've color-coded them so the pattern is visible.

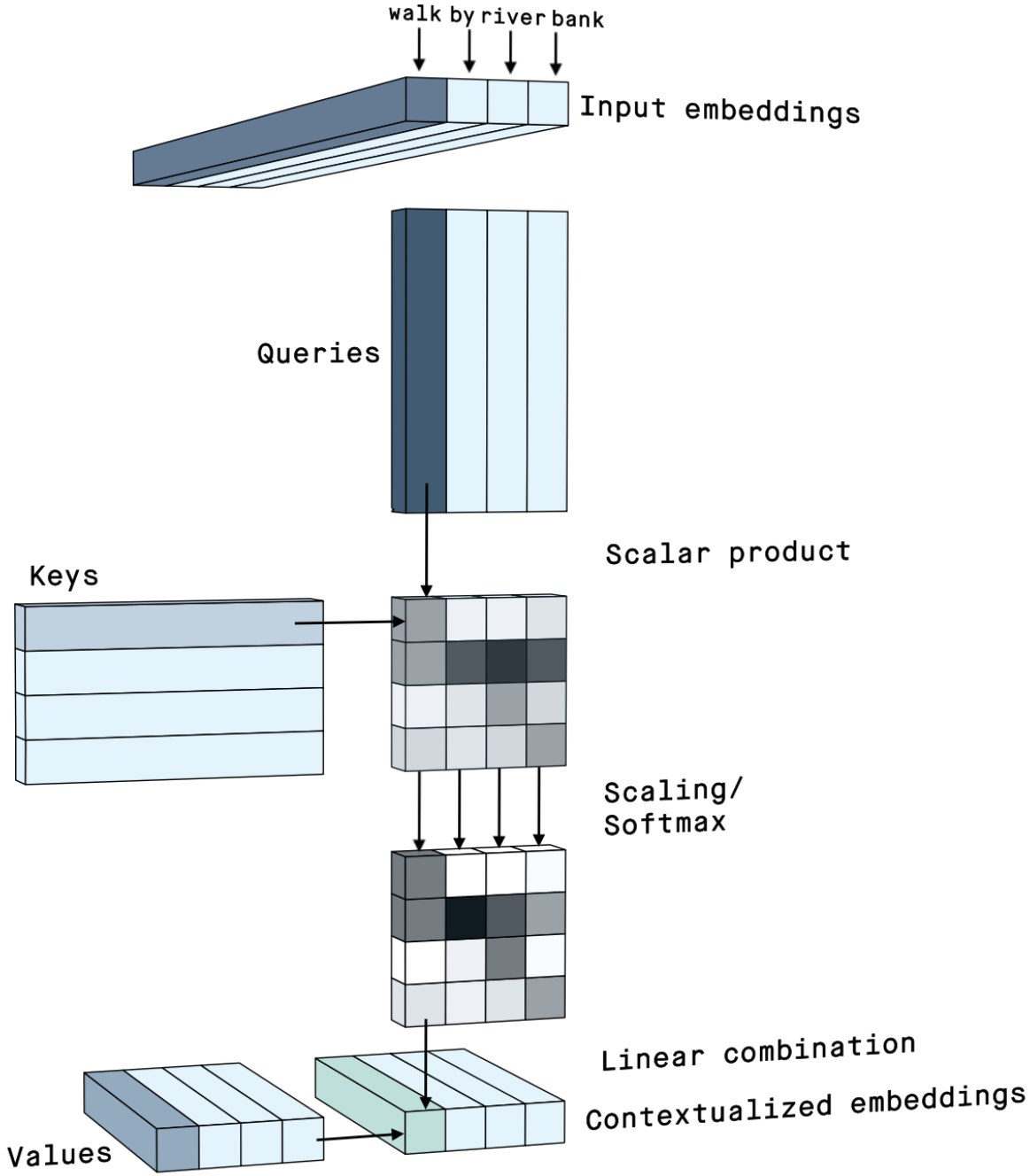


A real example of positional encoding for 20 words (rows) with an embedding size of 512 (columns). You can see that it appears split in half down the center. That's because the values of the left half are generated by one function (which uses sine), and the right half is generated by another function (which uses cosine). They're then concatenated to form each of the positional encoding vectors.

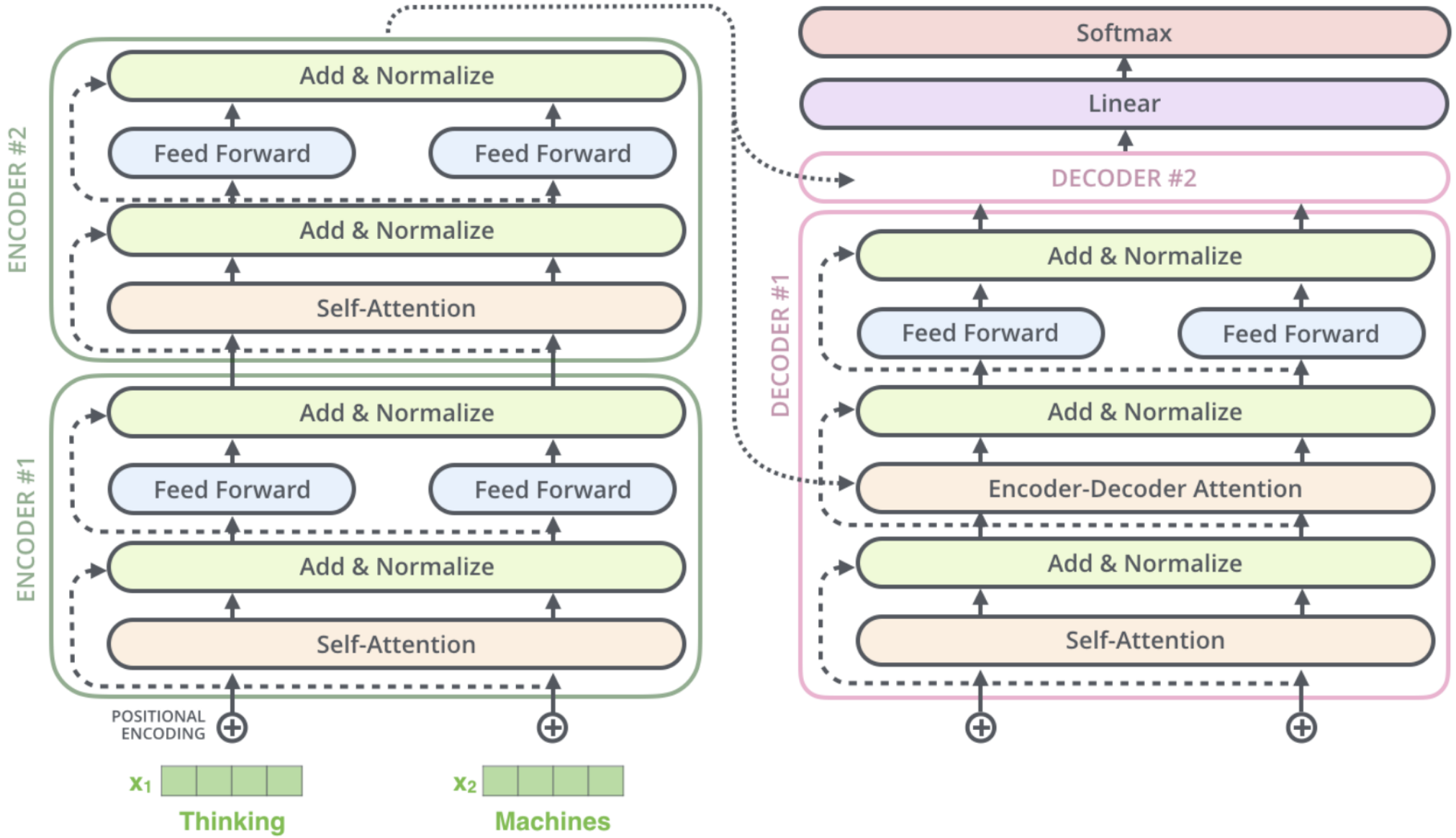
The formula for positional encoding is described in the paper (section 3.5). You can see the code for generating positional encodings in `get_timing_signal_1d()`. This is not the only possible method for positional encoding. It, however, gives the advantage of being able to scale to unseen lengths of sequences (e.g. if our trained model is asked to translate a sentence longer than any of those in our training set).

July 2020 Update: The positional encoding shown above is from the Tranformer2Transformer implementation of the Transformer. The method shown in the paper is slightly different in that it doesn't directly concatenate, but interweaves the two signals. The following figure shows what that looks like. Here's the code to generate it:



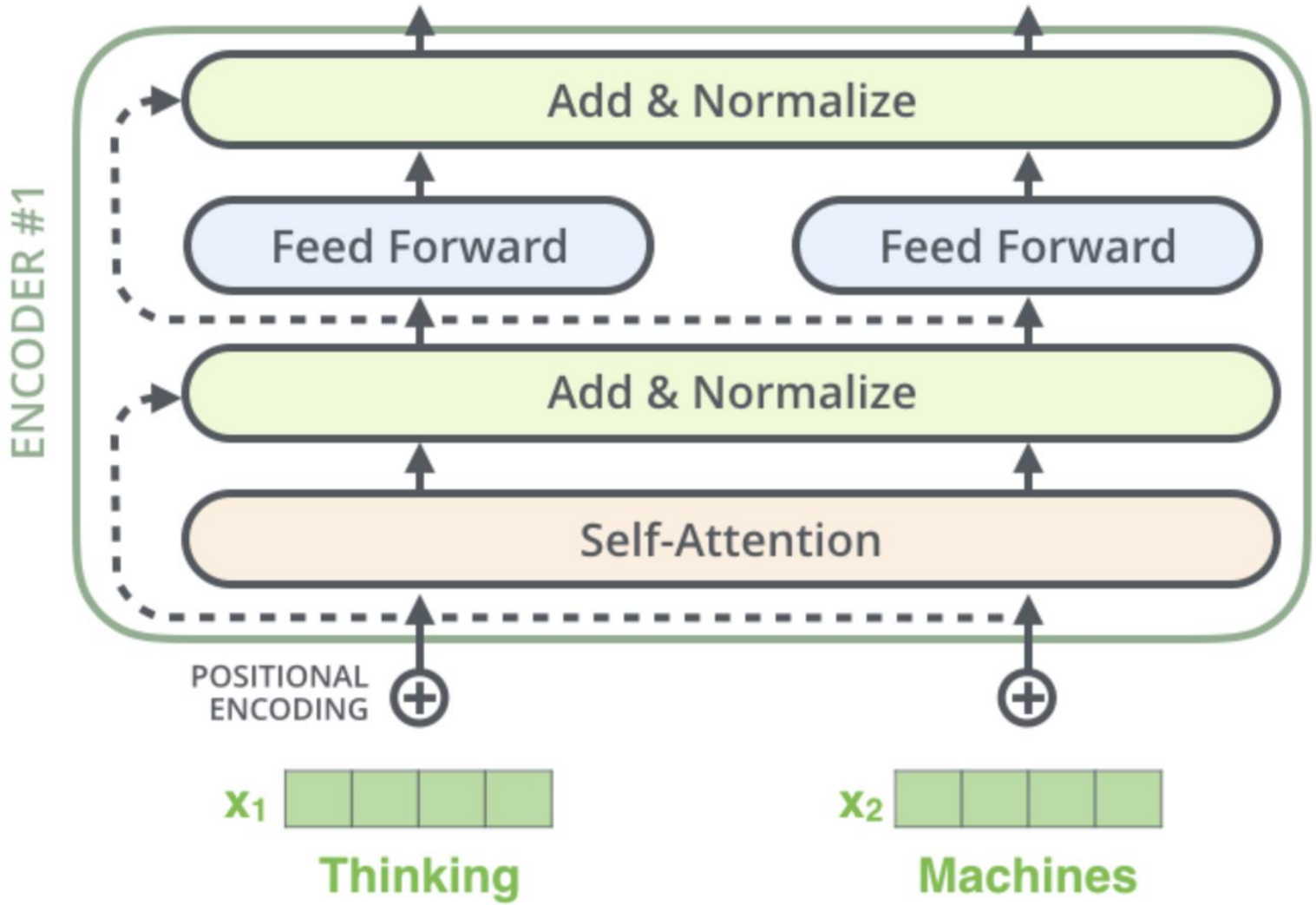

walk by river bank
Input embeddings
Queries
Keys
Scalar product
Scaling/
Softmax
Linear combination
Contextualized embeddings
Values

This goes for the sub-layers of the decoder as well. If we're to think of a Transformer of 2 stacked encoders and decoders, it would look something like this:

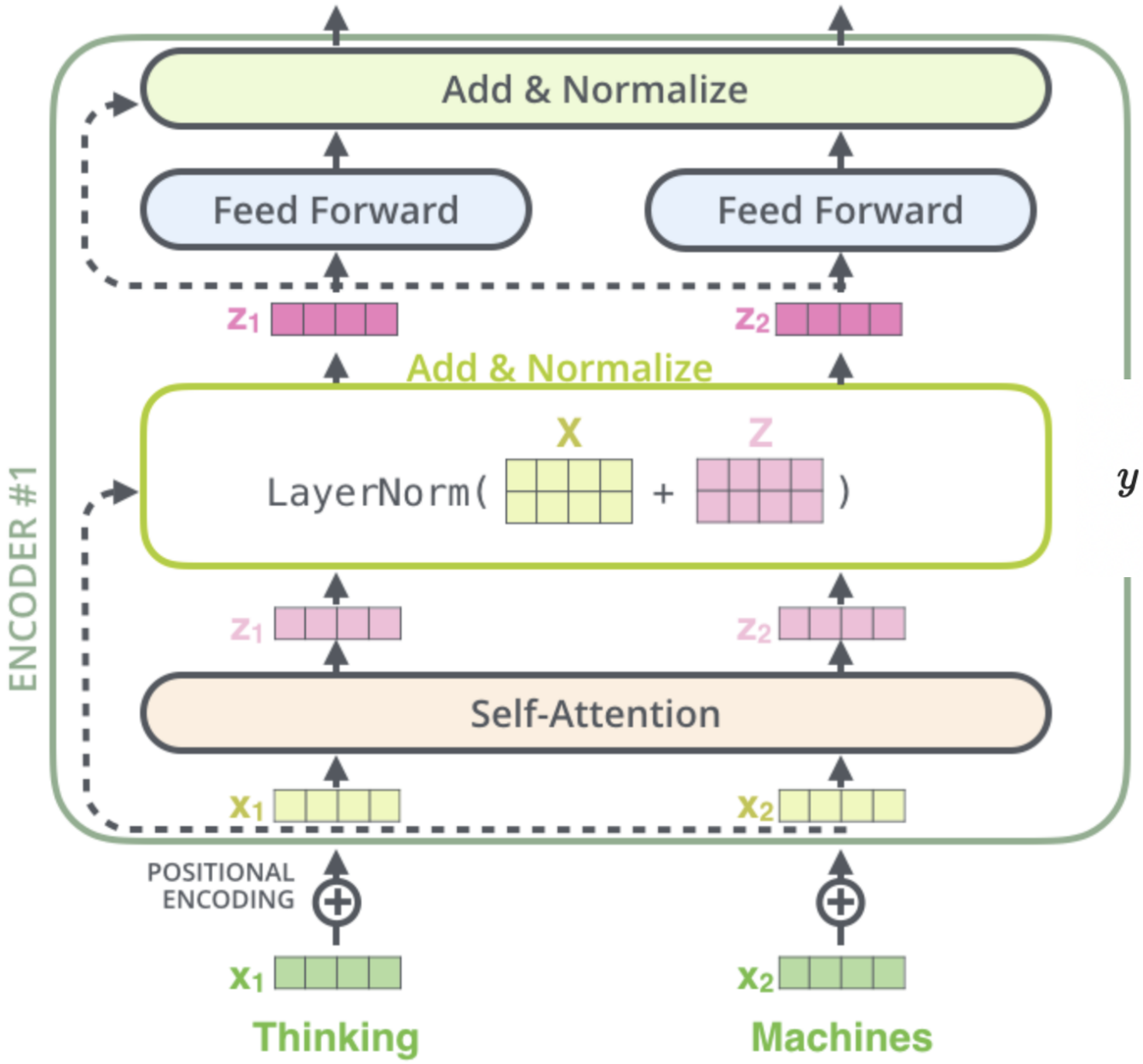


The Residuals

One detail in the architecture of the encoder that we need to mention before moving on, is that each sub-layer (self-attention, ffnn) in each encoder has a residual connection around it, and is followed by a layer-normalization step.



If we're to visualize the vectors and the layer-norm operation associated with self attention, it would look like this:



$$y = \frac{x - \mathbf{E}[x]}{\sqrt{\mathbf{Var}[x] + \epsilon}} * \gamma + \beta$$

What happens in the self-attention module?

A self-attention module takes in n inputs, and returns n outputs. What happens in this module? In layman's terms, the self-attention mechanism allows the inputs to interact with each other ("self") and find out who they should pay more attention to ("attention"). The outputs are aggregates of these interactions and attention scores.

1. Illustrations

The illustrations are divided into the following steps:

1. Prepare inputs
2. Initialise weights
3. Derive **key**, **query** and **value**
4. Calculate attention scores for Input 1
5. Calculate softmax
6. Multiply scores with **values**
7. Sum **weighted values** to get Output 1
8. Repeat steps 4–7 for Input 2 & Input 3

Step 1: Prepare inputs

Inputs are embeddings N positions by D deep in this case N = 3 and D = 4



Fig. 1.1: Prepare inputs

For this tutorial, we start with 3 inputs, each with dimension 4.

```
Input 1: [1, 0, 1, 0]
Input 2: [0, 2, 0, 2]
Input 3: [1, 1, 1, 1]
```

Self-Attention needs
Three representations

K (orange)
Q (red)
V (purple)

Recall N is 3
Embedding D is 4

These have shape 4 x 3

Step 2: Initialise weights

Every input must have three representations (see diagram below). These representations are called **key** (orange), **query** (red), and **value** (purple). For this example, let's take that we want these representations to have a dimension of 3. Because every input has a dimension of 4, this means each set of the weights must have a shape of 4×3.

> **Note**
> We'll see later that the dimension of **value** is also the dimension of the output.

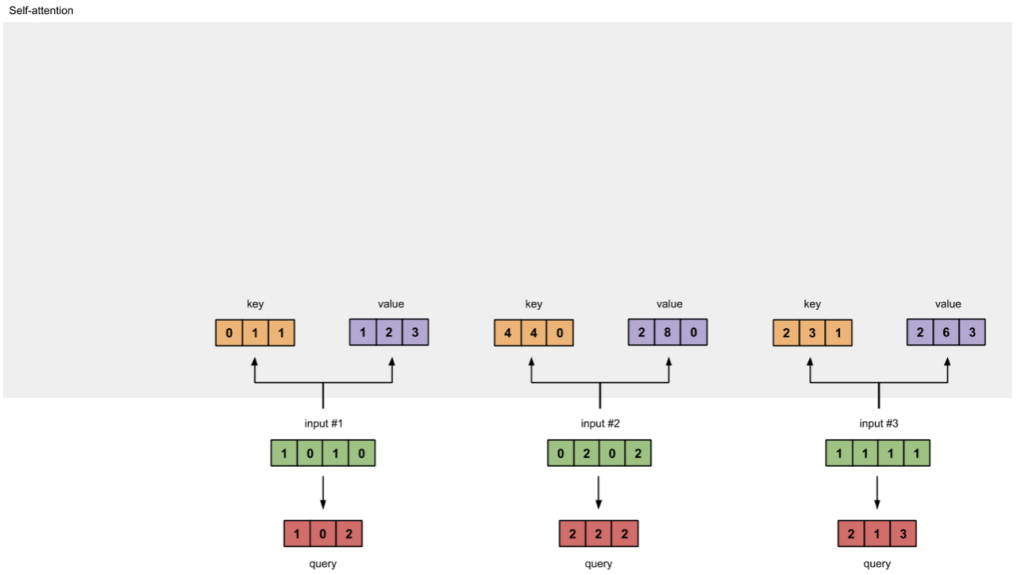


Fig. 1.2: Deriving **key**, **query** and **value** representations from each input

These weights are Randomly initialized

We are going to Multiply our input (green) by K, V and Q

In order to obtain these representations, every input (green) is multiplied with a set of weights for **keys**, a set of weights for **querys** (I know that's not the right spelling), and a set of weights for **values**. In our example, we initialise the three sets of weights as follows.

Weights for **key**:

```
[[0, 0, 1],
 [1, 1, 0],
 [0, 1, 0],
 [1, 1, 0]]
```

Weights for **query**:

```
[[1, 0, 1],
 [1, 0, 0],
 [0, 0, 1],
 [0, 1, 1]]
```

Weights for **value**:

```
[[0, 2, 0],
 [0, 3, 0],
 [1, 0, 3],
 [1, 1, 0]]
```

> **Notes**
>
> In a neural network setting, these weights are usually small numbers, initialised randomly using an appropriate random distribution like Gaussian, Xavier and Kaiming distributions. This initialisation is done once before training.

$Inputs_i \times K = K_i$

In practice this will
Be vectorized for performance

Step 3: Derive key, query and value

Now that we have the three sets of weights, let's actually obtain the **key**, **query** and **value** representations for every input.

Key representation for Input 1:

```
                [0, 0, 1]
[1, 0, 1, 0] x [1, 1, 0] = [0, 1, 1]
                [0, 1, 0]
                [1, 1, 0]
```

Use the same set of weights to get the **key** representation for Input 2:

```
                [0, 0, 1]
[0, 2, 0, 2] x [1, 1, 0] = [4, 4, 0]
                [0, 1, 0]
                [1, 1, 0]
```

Use the same set of weights to get the **key** representation for Input 3:

```
                [0, 0, 1]
[1, 1, 1, 1] x [1, 1, 0] = [2, 3, 1]
                [0, 1, 0]
                [1, 1, 0]
```

A faster way is to vectorise the above operations:

A faster way is to vectorise the above operations:

```
                  [0, 0, 1]
[1, 0, 1, 0]      [1, 1, 0]     [0, 1, 1]
[0, 2, 0, 2]  x   [0, 1, 0]  =  [4, 4, 0]
[1, 1, 1, 1]      [1, 1, 0]     [2, 3, 1]
```

Vectorized so we do
The whole input set
In one x K operation
Giving us our K_is

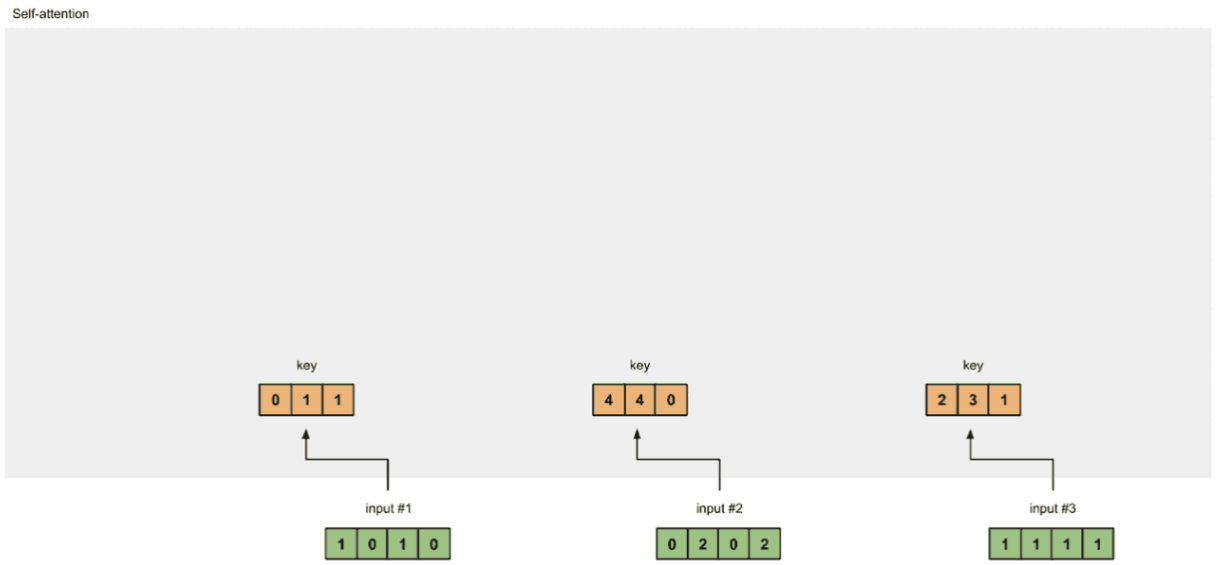


Fig. 1.3a: Derive **key** representations from each input

Same thing
For the V_is

Let's do the same to obtain the **value** representations for every input:

```
                  [0, 2, 0]
[1, 0, 1, 0]      [0, 3, 0]      [1, 2, 3]
[0, 2, 0, 2]  x   [1, 0, 3]  =   [2, 8, 0]
[1, 1, 1, 1]      [1, 1, 0]      [2, 6, 3]
```

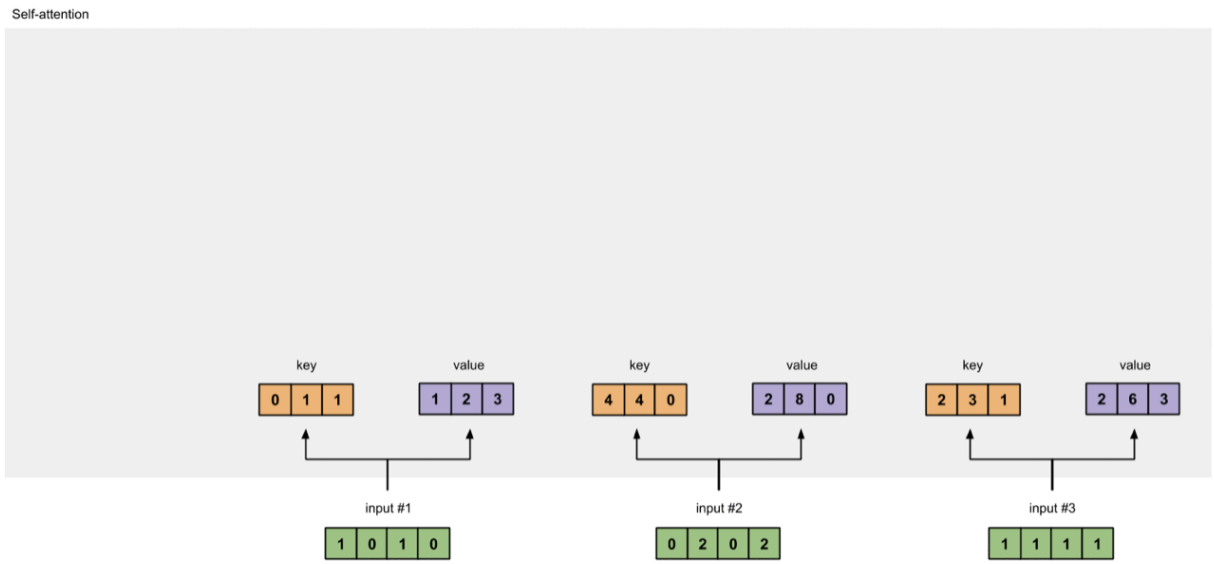


Fig. 1.3b: Derive **value** representations from each input

and finally the **query** representations:

```
                  [1, 0, 1]
[1, 0, 1, 0]      [1, 0, 0]     [1, 0, 2]
[0, 2, 0, 2]  x   [0, 0, 1]  =  [2, 2, 2]
[1, 1, 1, 1]      [0, 1, 1]     [2, 1, 3]
```

Same thing
For the Q_is

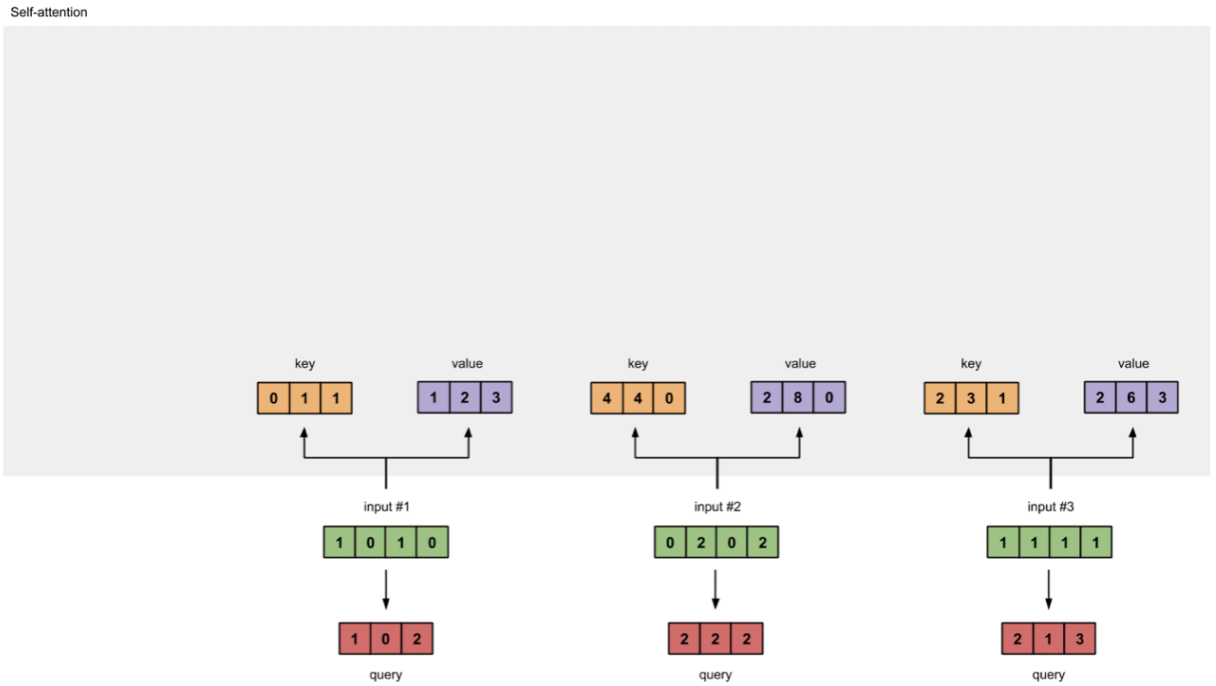


Fig. 1.3c: Derive **query** representations from each input

> **Notes**
>
> In practice, a *bias vector* may be added to the product of matrix multiplication.

Step 4: Calculate attention scores for Input 1

Now we calculate the Attention scores for Input 1

Dot product of Input1's Q_1 with all Ks including Self

Dot product = scalar

Giving three attention scores

We do this for each of The other inputs next

Scaling is also done but Not represented here.

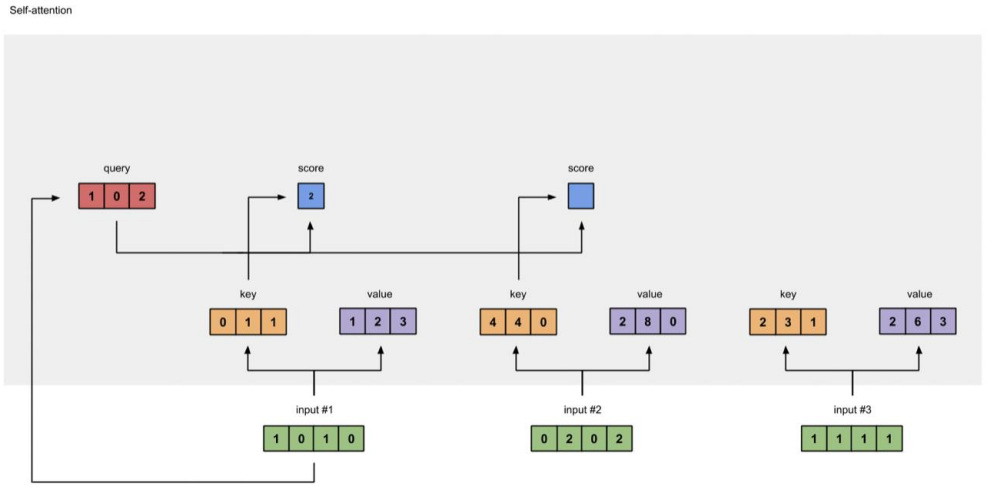


Fig. 1.4: Calculating attention scores (blue) from query 1

To obtain *attention scores*, we start off with taking a dot product between Input 1's **query** (red) with all **keys** (orange), including itself. Since there are 3 **key** representations (because we have 3 inputs), we obtain 3 attention scores (blue).

```
                [0, 4, 2]
[1, 0, 2] x     [1, 4, 3] = [2, 4, 4]
                [1, 0, 1]
```

Notice that we only use the **query** from Input 1. Later we'll work on repeating this same step for the other **querys**.

> **Note**
>
> The above operation is known as *dot product attention*, one of the several **score functions**. Other score functions include *scaled dot product* and *additive/concat*.

Step 5: Calculate softmax

Next calculate softmax
Across the scores

Softmax sums to 1.0

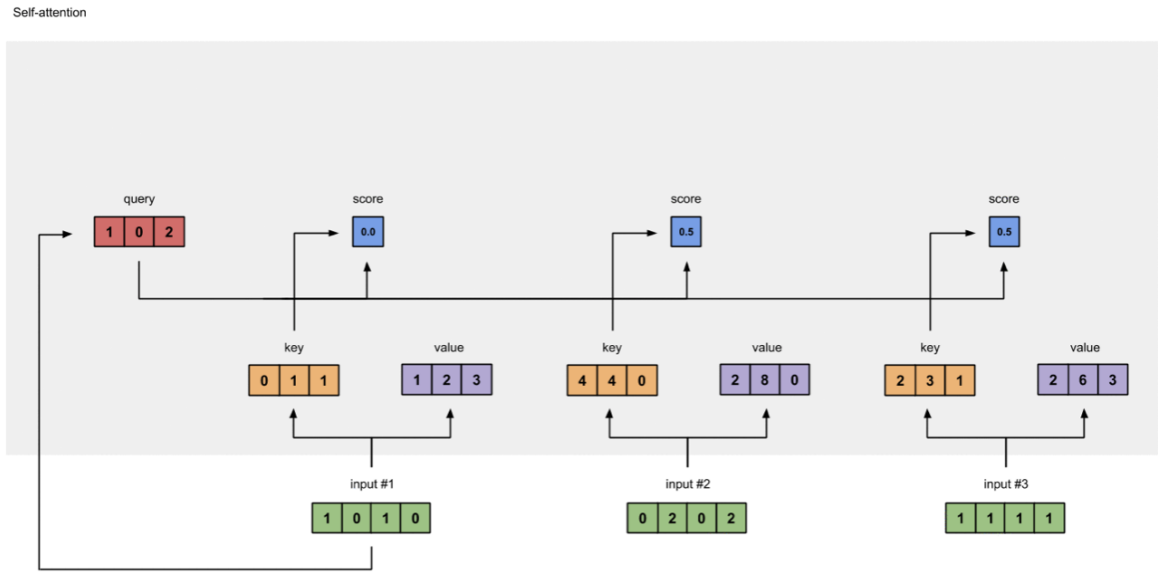


Fig. 1.5: Softmax the attention scores (blue)

Take the softmax across these attention scores (blue).

```
softmax([2, 4, 4]) = [0.0, 0.5, 0.5]
```

Note that we round of to 1 decimal place here for readability.

Step 6: Multiply scores with values

Multiply the softmax Scores with values V

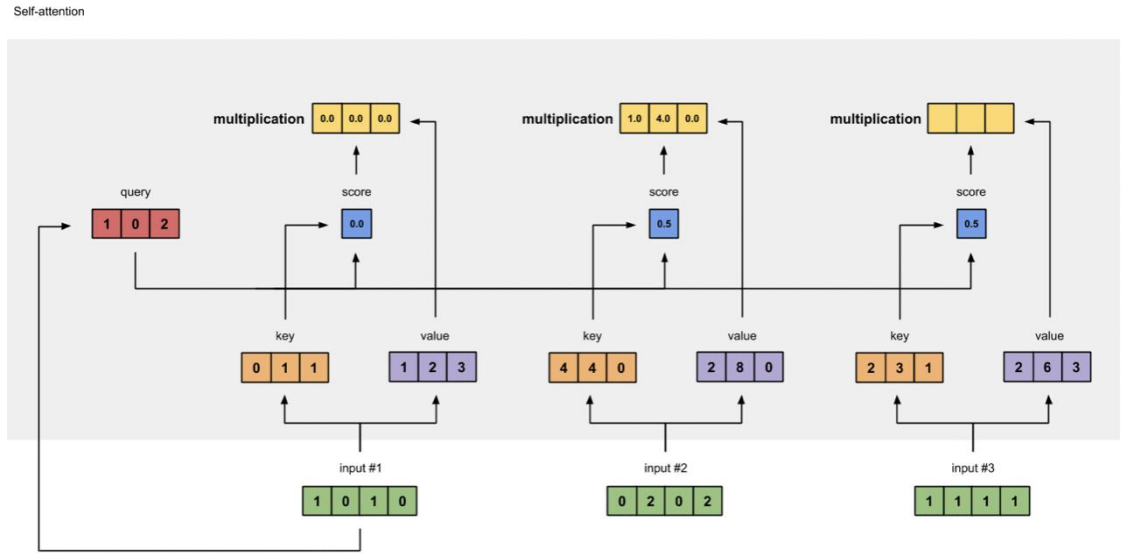


Fig. 1.6: Derive **weighted value** representation (yellow) from multiply **value** (purple) and score (blue)

The softmaxed attention scores for each input (blue) is multiplied with its corresponding **value** (purple). This results in 3 *alignment vectors* (yellow). In this tutorial, we'll refer to them as **weighted values**.

```
1: 0.0 * [1, 2, 3] = [0.0, 0.0, 0.0]
2: 0.5 * [2, 8, 0] = [1.0, 4.0, 0.0]
3: 0.5 * [2, 6, 3] = [1.0, 3.0, 1.5]
```

Step 7: Sum weighted values to get Output 1

Sum the weighted values
To get output for position 1

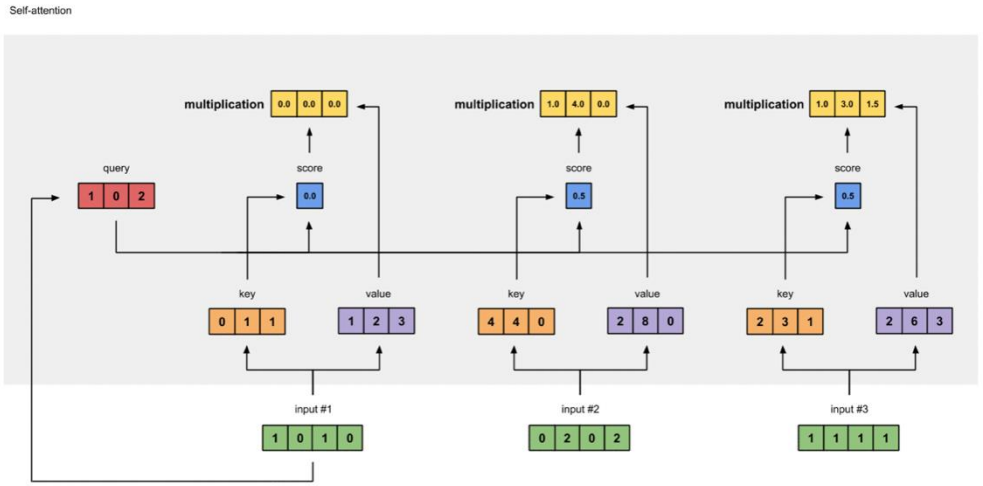


Fig. 1.7: Sum all **weighted values** (yellow) to get Output 1 (dark green)

Take all the **weighted values** (yellow) and sum them element-wise:

```
  [0.0, 0.0, 0.0]
+ [1.0, 4.0, 0.0]
+ [1.0, 3.0, 1.5]
-----------------
= [2.0, 7.0, 1.5]
```

The resulting vector [2.0, 7.0, 1.5] (dark green) is Output 1, which is based on the **query representation from Input 1** interacting with all other keys, including itself.

Step 8: Repeat for Input 2 & Input 3

Now that we're done with Output 1, we repeat Steps 4 to 7 for Output 2 and Output 3. I trust that I can leave you to work out the operations yourself 👍🏼.

Repeat for input 2 and 3

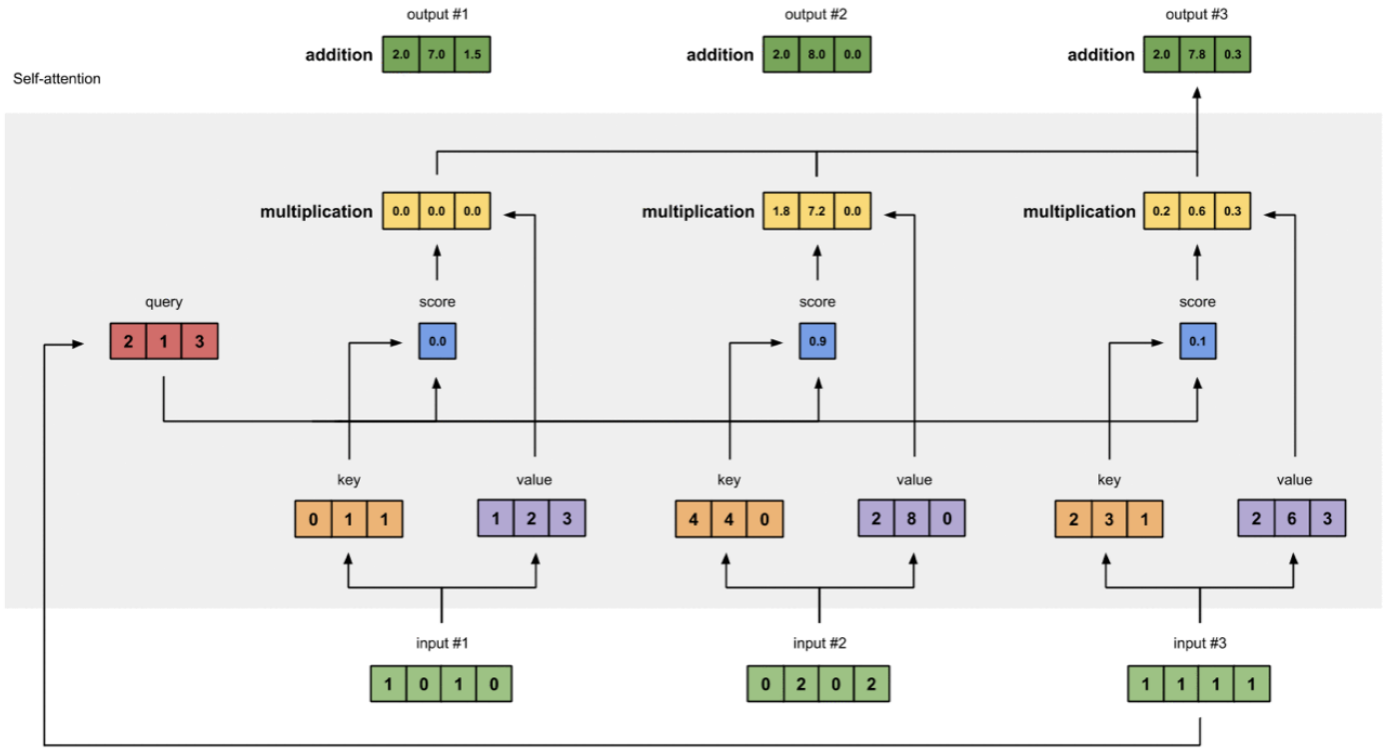
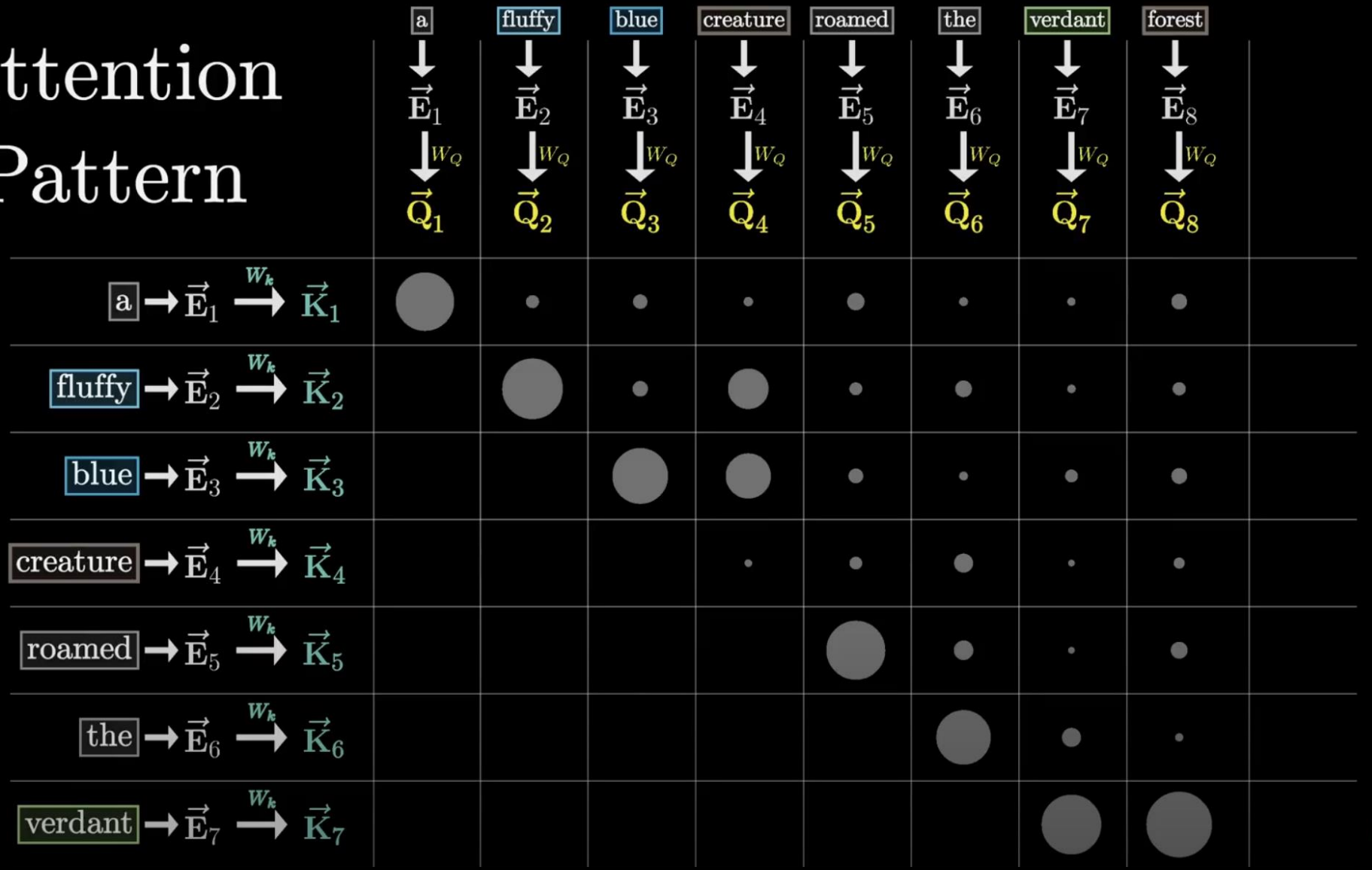
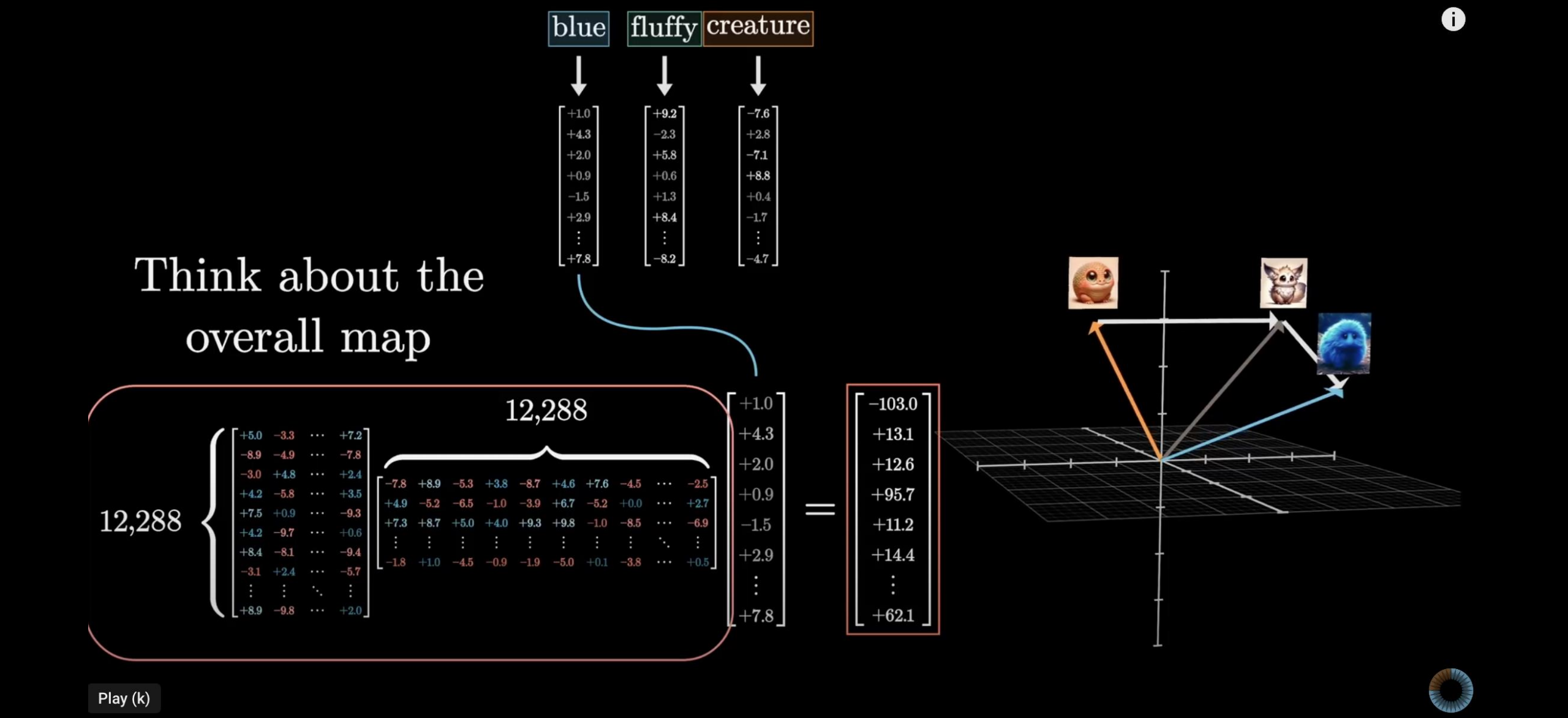


Fig. 1.8: Repeat previous steps for Input 2 & Input 3

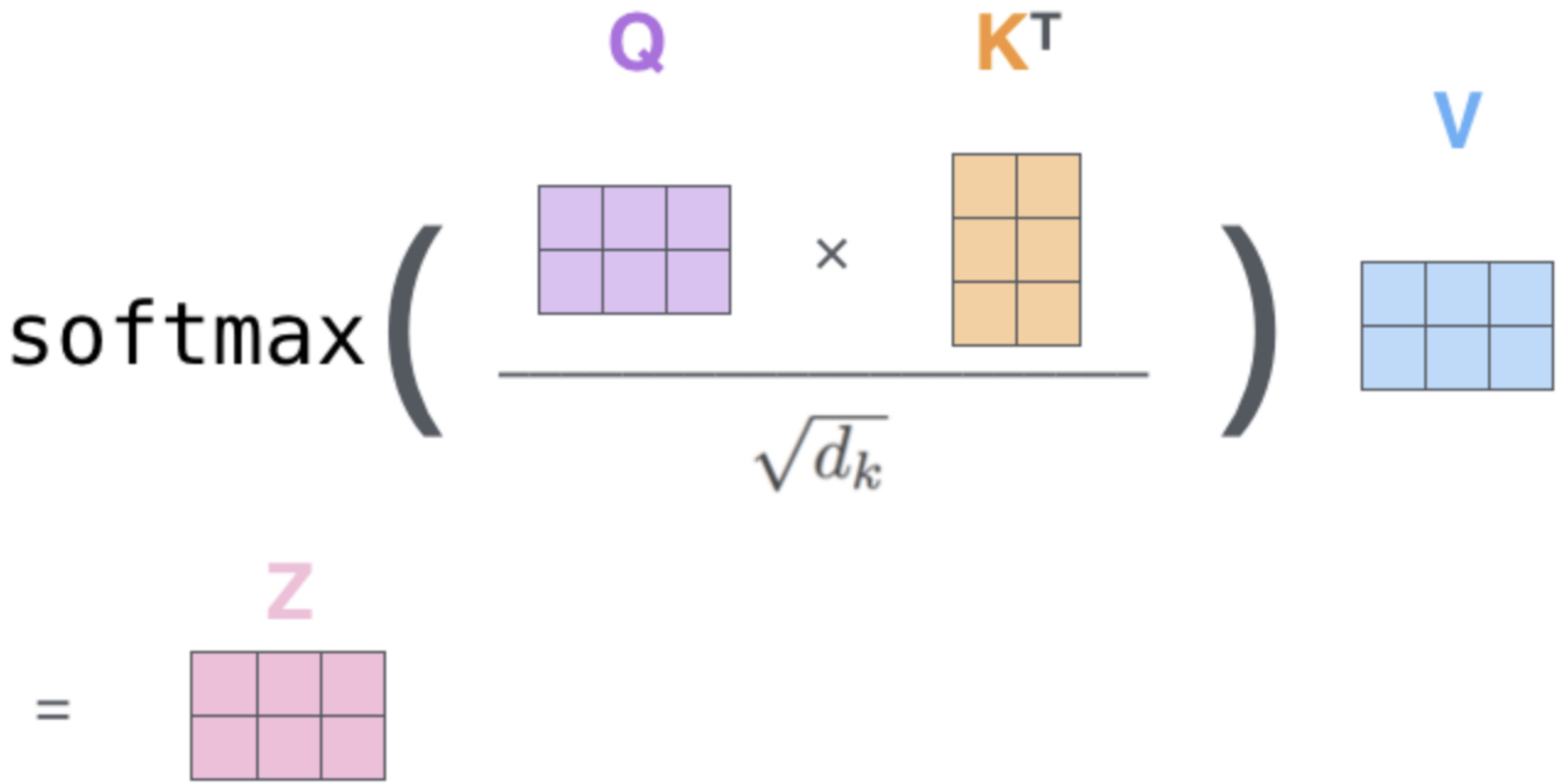
Attention Pattern




blue
fluffy
creature
Think about the overall map
12,288
12,288
Play (k)

> **Notes**
>
> The dimension of **query** and **key** must always be the same because of the dot product score function. However, the dimension of **value** may be different from **query** and **key**. The resulting output will consequently follow the dimension of **value**.

Finally, since we're dealing with matrices, we can condense steps two through six in one formula to calculate the outputs of the self-attention layer.



The self-attention calculation in matrix form

Multihead attention

Multiple representation Subspaces

Think of it as multiple Contexts

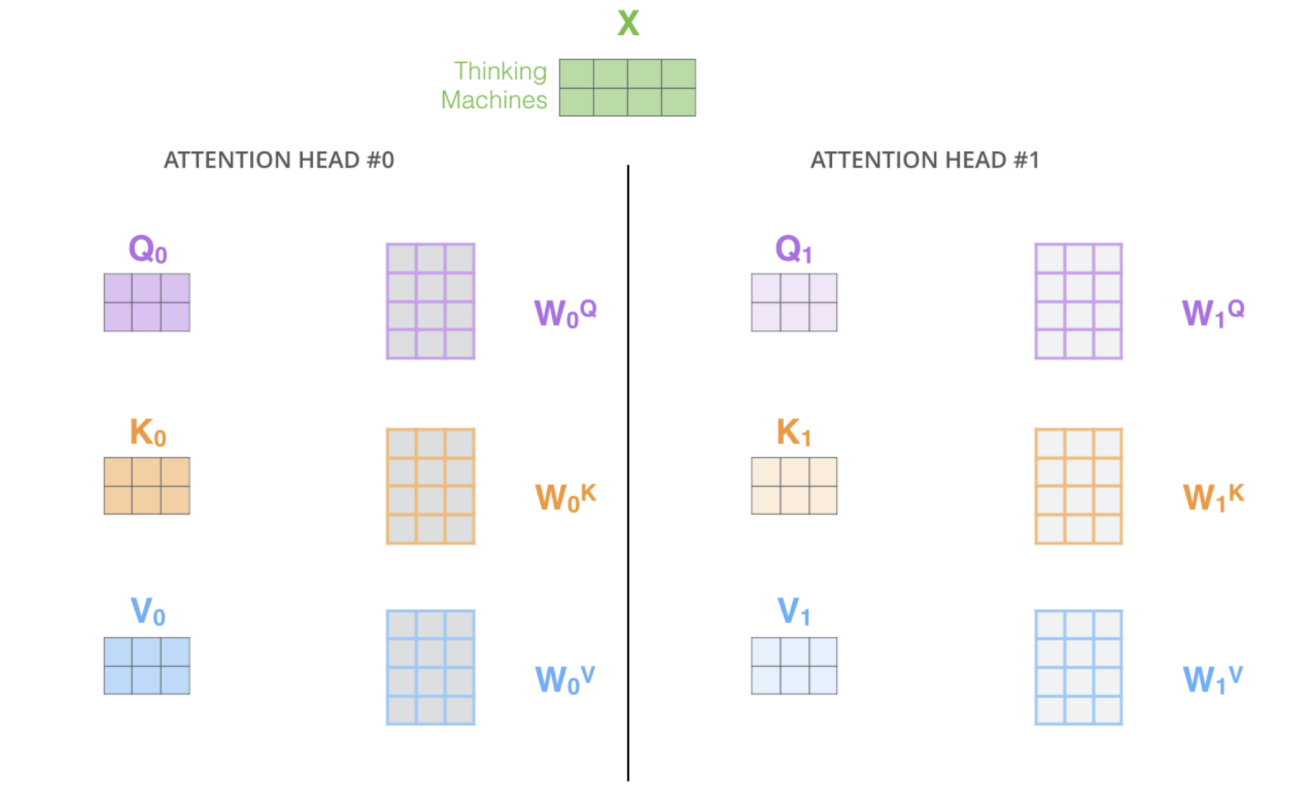
#heads in GPT-3 175B = 96

Same as number of layers

The Beast With Many Heads

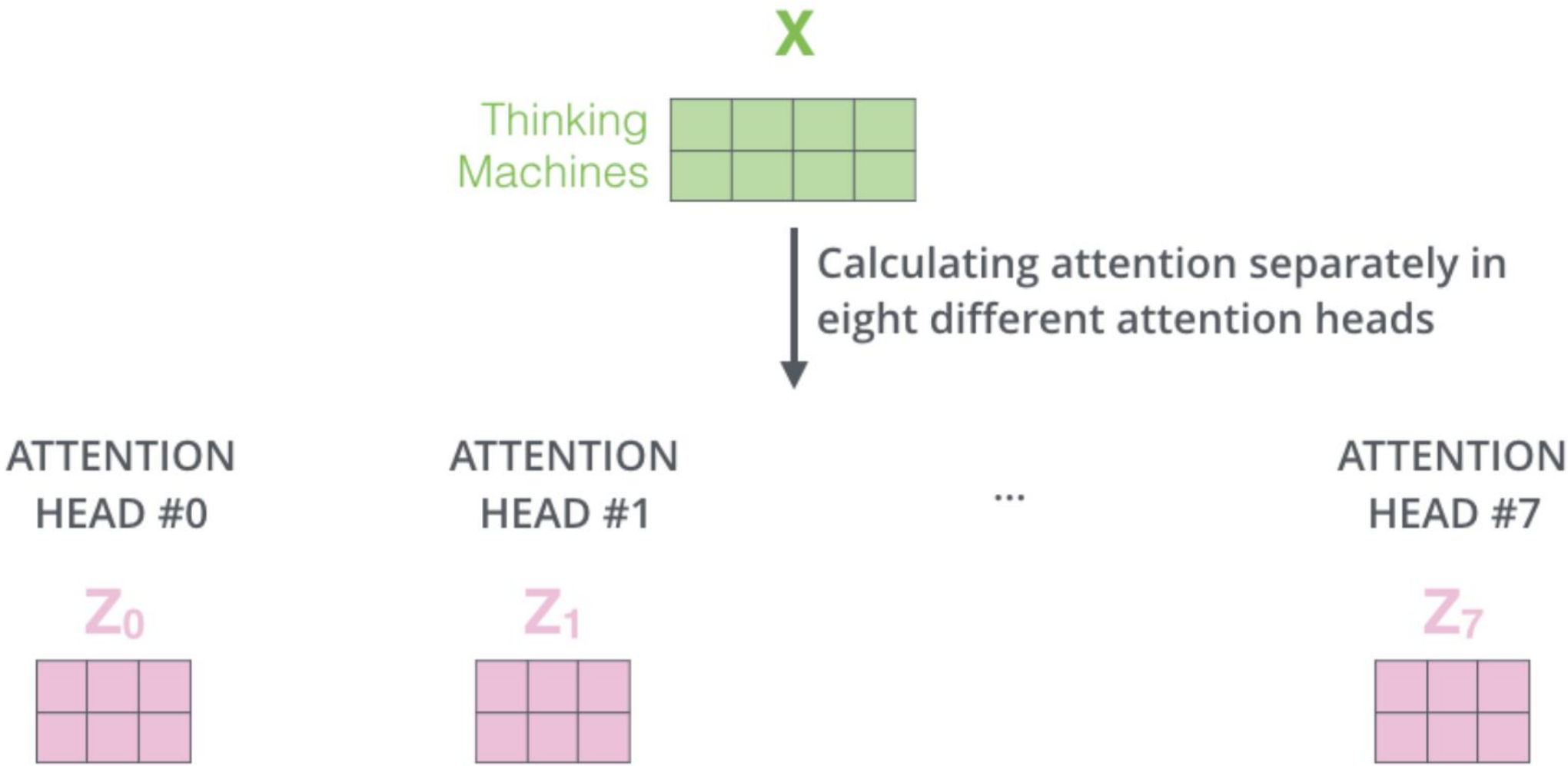
The paper further refined the self-attention layer by adding a mechanism called “multi-headed” attention. This improves the performance of the attention layer in two ways:

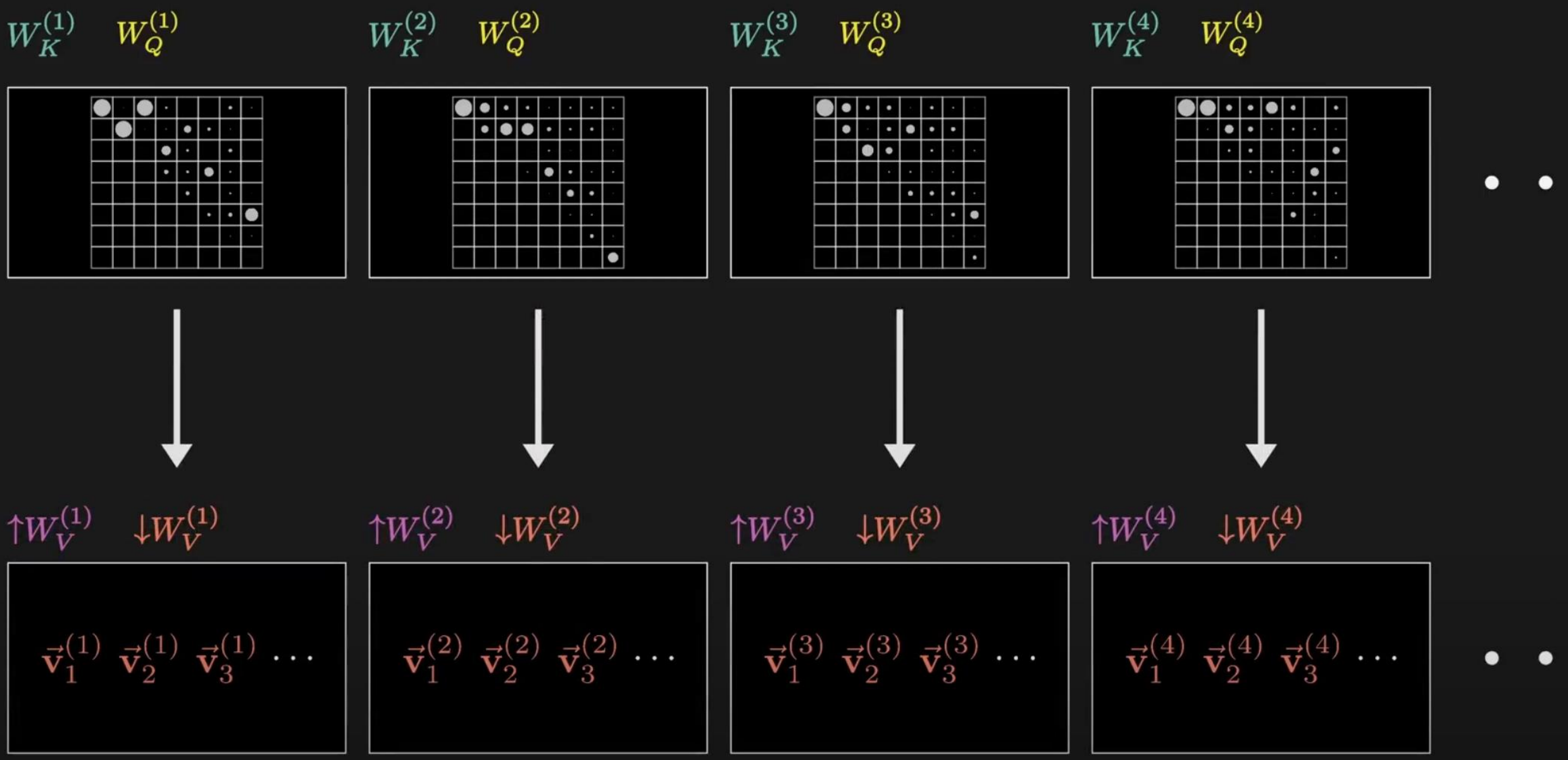
1. It expands the model’s ability to focus on different positions. Yes, in the example above, z1 contains a little bit of every other encoding, but it could be dominated by the the actual word itself. It would be useful if we’re translating a sentence like “The animal didn’t cross the street because it was too tired”, we would want to know which word “it” refers to.
2. It gives the attention layer multiple “representation subspaces”. As we’ll see next, with multi-headed attention we have not only one, but multiple sets of Query/Key/Value weight matrices (the Transformer uses eight attention heads, so we end up with eight sets for each encoder/decoder). Each of these sets is randomly initialized. Then, after training, each set is used to project the input embeddings (or vectors from lower encoders/decoders) into a different representation subspace.



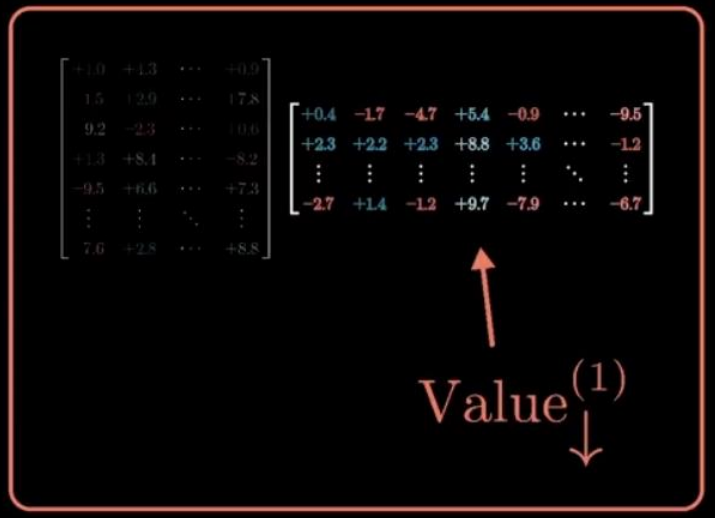
With multi-headed attention, we maintain separate Q/K/V weight matrices for each head resulting in different Q/K/V matrices. As we did before, we multiply X by the WQ/WK/WV matrices to produce Q/K/V matrices.

If we do the same self-attention calculation we outlined above, just eight different times with different weight matrices, we end up with eight different Z matrices

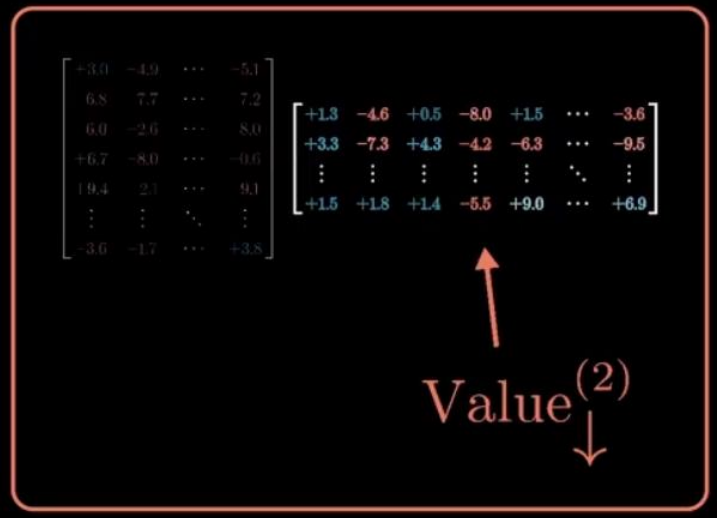



$W_K^{(1)}$ $W_Q^{(1)}$
$W_K^{(2)}$ $W_Q^{(2)}$
$W_K^{(3)}$ $W_Q^{(3)}$
$W_K^{(4)}$ $W_Q^{(4)}$
$\uparrow W_V^{(1)}$ $\downarrow W_V^{(1)}$
$\uparrow W_V^{(2)}$ $\downarrow W_V^{(2)}$
$\uparrow W_V^{(3)}$ $\downarrow W_V^{(3)}$
$\uparrow W_V^{(4)}$ $\downarrow W_V^{(4)}$
$\vec{\mathbf{v}}_1^{(1)}$ $\vec{\mathbf{v}}_2^{(1)}$ $\vec{\mathbf{v}}_3^{(1)}$ $\cdots$
$\vec{\mathbf{v}}_1^{(2)}$ $\vec{\mathbf{v}}_2^{(2)}$ $\vec{\mathbf{v}}_3^{(2)}$ $\cdots$
$\vec{\mathbf{v}}_1^{(3)}$ $\vec{\mathbf{v}}_2^{(3)}$ $\vec{\mathbf{v}}_3^{(3)}$ $\cdots$
$\vec{\mathbf{v}}_1^{(4)}$ $\vec{\mathbf{v}}_2^{(4)}$ $\vec{\mathbf{v}}_3^{(4)}$ $\cdots$

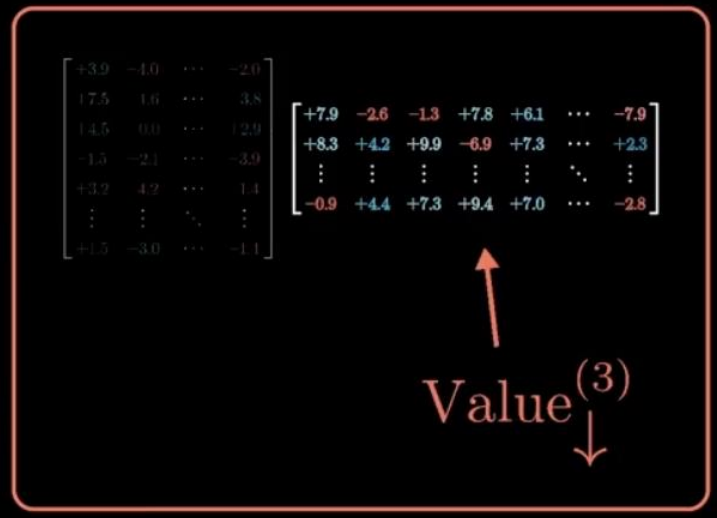
Head 1
Value map



Head 2
Value map



Head 3
Value map



Output matrix

$$\begin{bmatrix} \text{Value}^{(1)} & & & & \text{Value}^{(2)} & & & & \text{Value}^{(3)} & & & & \\ +1.0 & +4.3 & \cdots & +0.9 & +3.0 & -4.9 & \cdots & -5.1 & +3.9 & -4.0 & \cdots & -2.0 & \\ -1.5 & +2.9 & \cdots & +7.8 & -6.8 & -7.7 & \cdots & -7.2 & +7.5 & +1.6 & \cdots & +3.8 & \\ +9.2 & -2.3 & \cdots & +0.6 & -6.0 & -2.6 & \cdots & -8.0 & +4.5 & +0.0 & \cdots & +2.9 & \\ +1.3 & +8.4 & \cdots & -8.2 & +6.7 & -8.0 & \cdots & -0.6 & -1.5 & +2.1 & \cdots & -3.9 & \cdots \\ -9.5 & +6.6 & \cdots & +7.3 & +9.4 & +2.1 & \cdots & -9.1 & +3.2 & -4.2 & \cdots & -1.4 & \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \\ -7.6 & +2.8 & \cdots & +8.8 & -3.6 & -1.7 & \cdots & +3.8 & +1.5 & +3.0 & \cdots & -1.4 & \end{bmatrix}$$

This leaves us with a bit of a challenge. The feed-forward layer is not expecting eight matrices – it's expecting a single matrix (a vector for each word). So we need a way to condense these eight down into a single matrix.

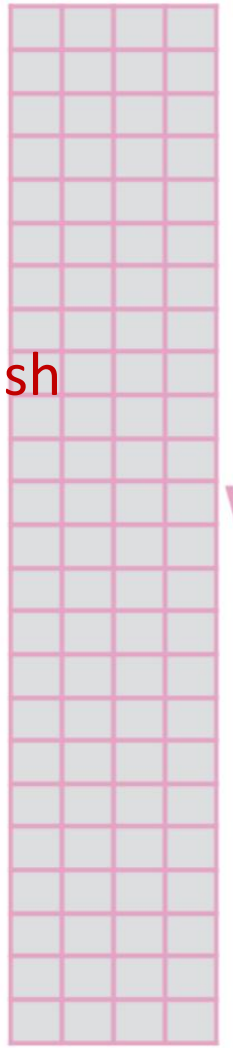
How do we do that? We concat the matrices then multiple them by an additional weights matrix WO.

1) Concatenate all the attention heads



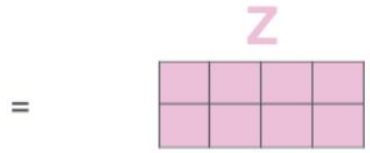
2) Multiply with a weight matrix W^O that was trained jointly with the model

X

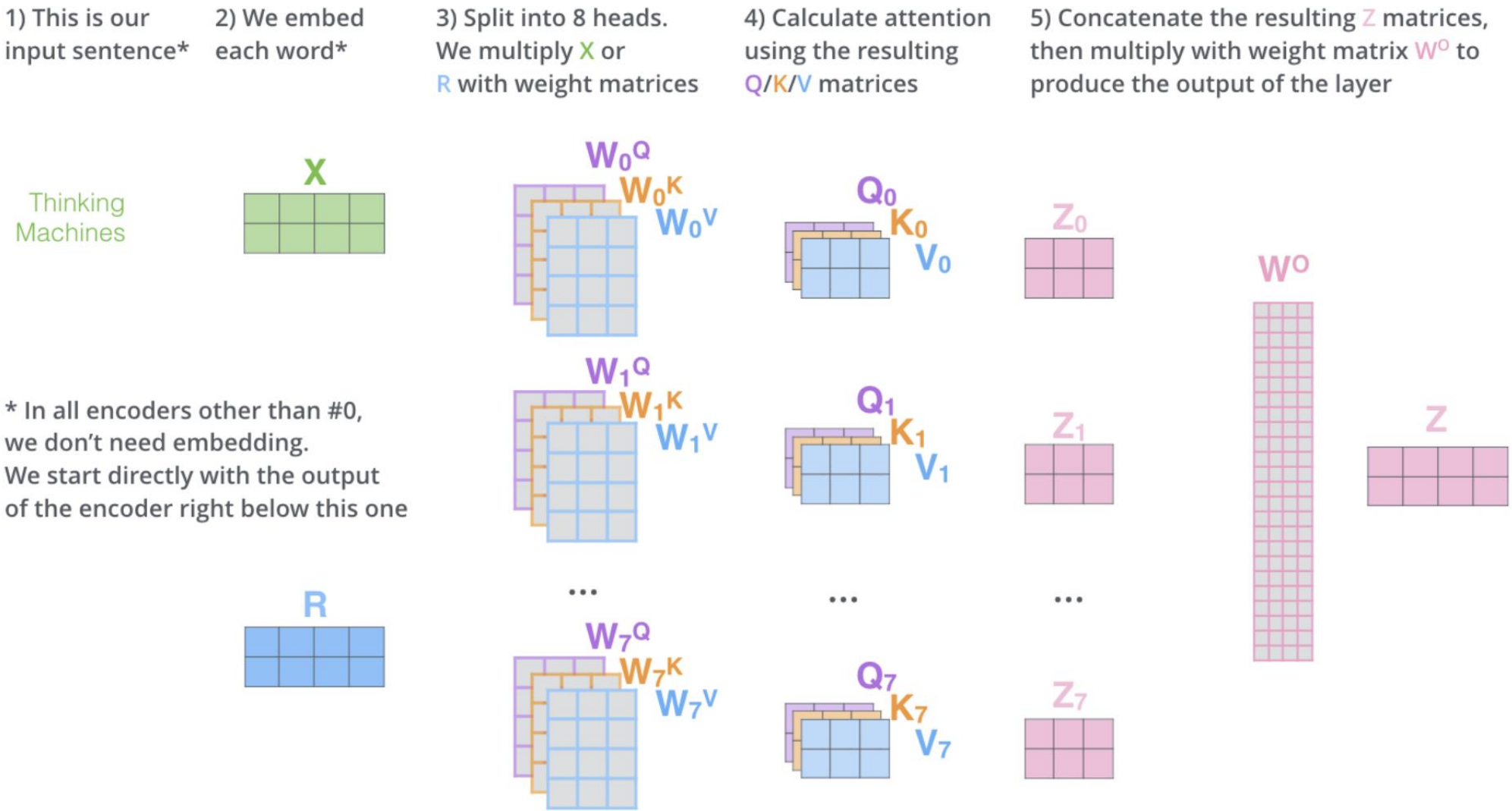


Seems odd we calculate them separately then concatenate and then squish

3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN

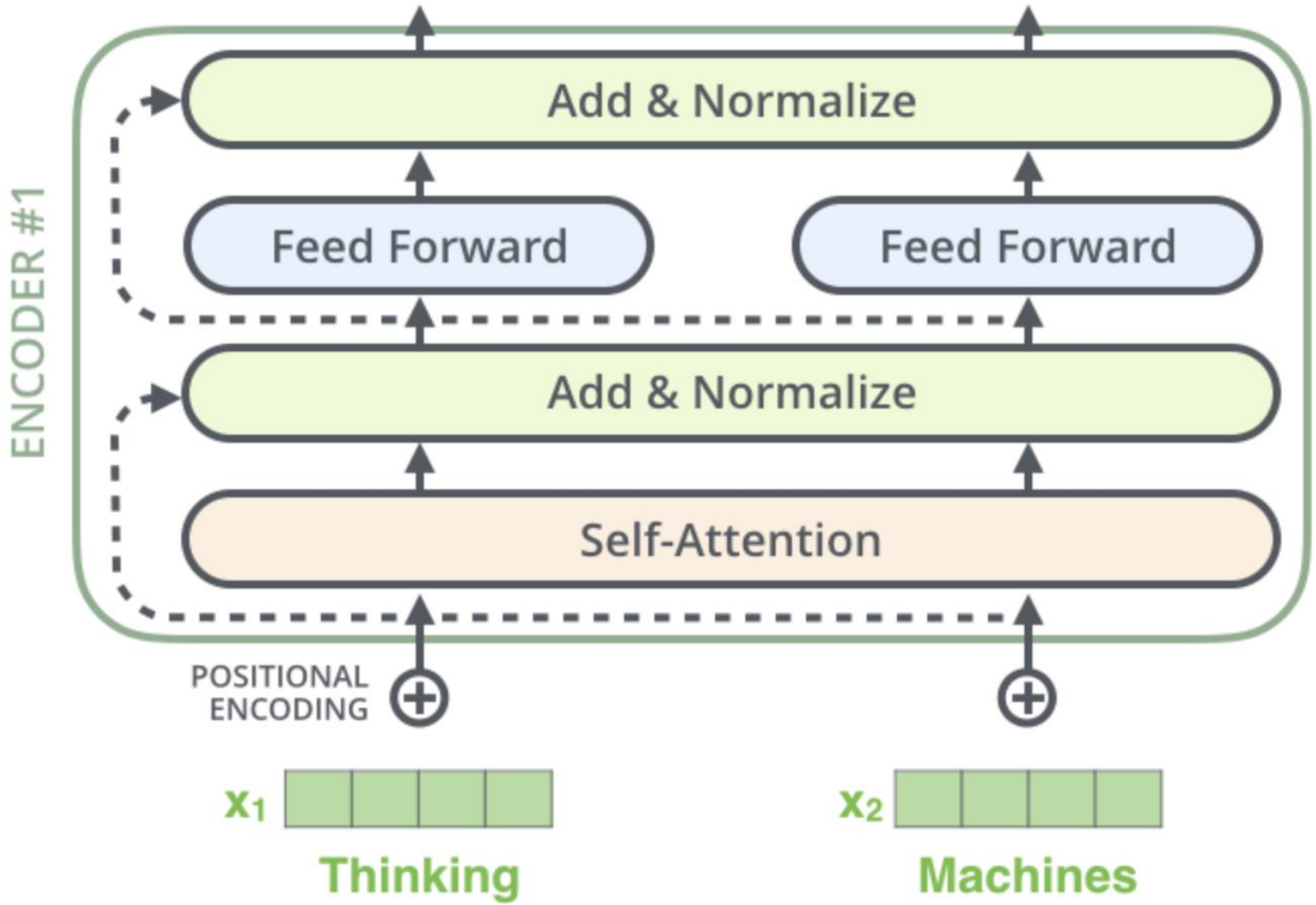


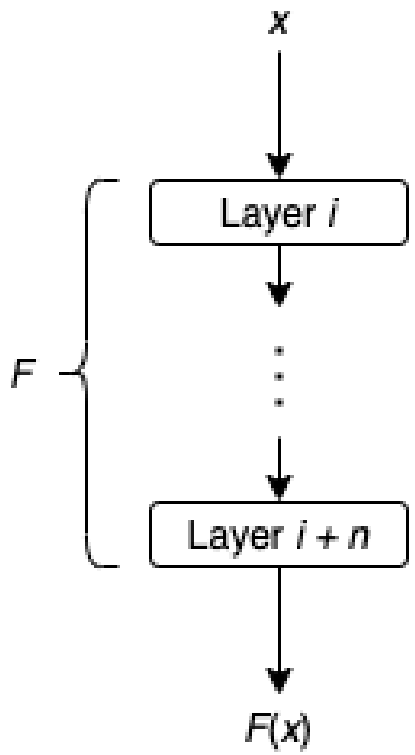
That's pretty much all there is to multi-headed self-attention. It's quite a handful of matrices, I realize. Let me try to put them all in one visual so we can look at them in one place



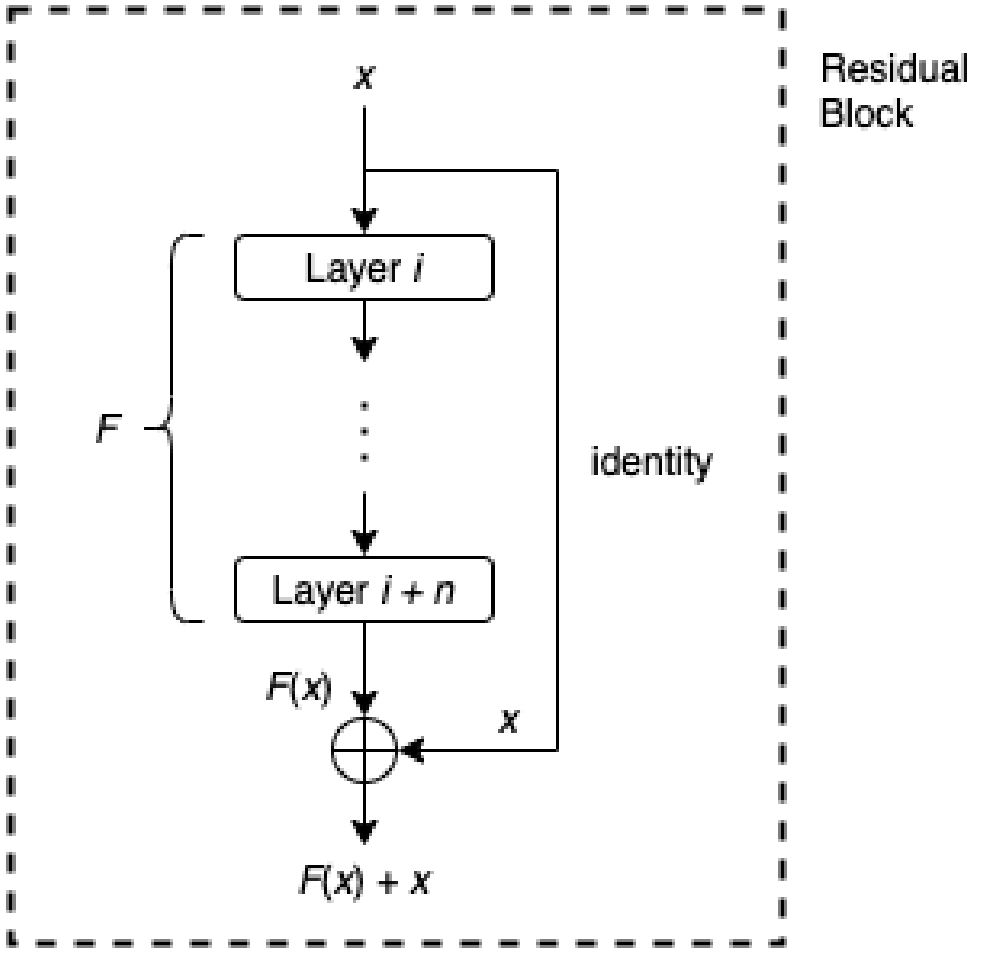
The Residuals

One detail in the architecture of the encoder that we need to mention before moving on, is that each sub-layer (self-attention, ffnn) in each encoder has a residual connection around it, and is followed by a layer-normalization step.

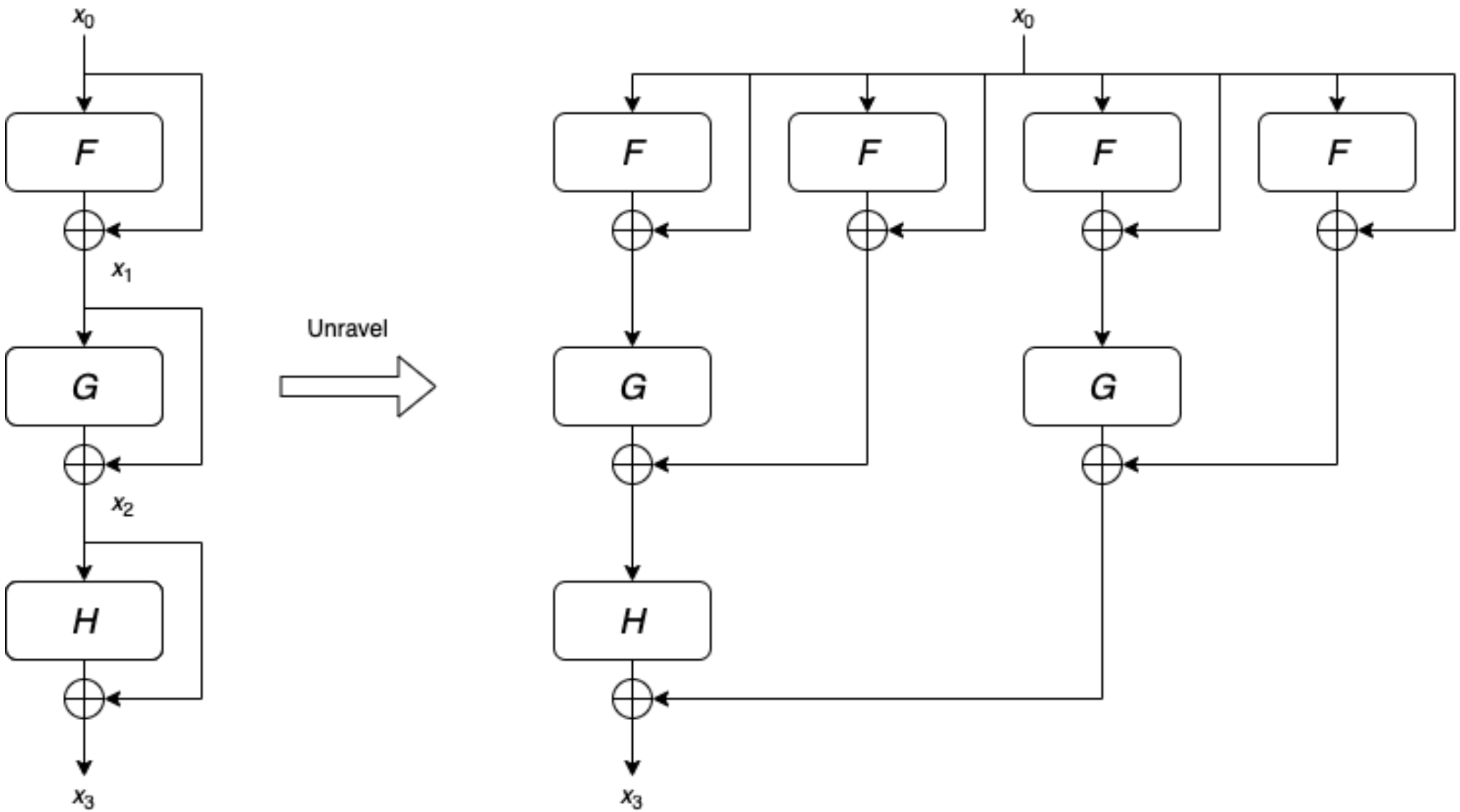




Traditional Feedforward
without Residual Connection



With Residual Connection


x_0
F
x_1
G
x_2
H
x_3
Unravel
x_0
F
F
F
F
G
G
H
x_3

Ways residual connections help

Residual connections, introduced in ResNet (Residual Networks), play a vital role in enabling deep neural networks to **incrementally learn** by addressing problems like **vanishing gradients** and **degraded learning** that often plague very deep mod

1. **Eases Optimization in Deeper Networks:**
• Without residual connections, deep networks need to learn complex transformations all at once.
• Residual connections allow the intermediate layers to focus on learning **incremental changes** to the input. They can treat the output of each block as an enhancement of the input instead of transforming it entirely.
Analogy: Imagine writing a long essay—rather than starting over from scratch every time, you make incremental edits to improve what already exists.

2. **Addressing Vanishing Gradients:**
• In deep networks, gradients can shrink as they propagate backward through layers, leading to poor weight updates.
• Residual connections allow **gradients to flow more freely** through the skip connections, ensuring that learning signals reach the earlier layers effectively.

3. **Facilitates Layer-Wise Refinement:**
• Each residual block can learn to **fine-tune** the output from the previous layers, ensuring that the network gradually improves.
• Even if some blocks learn nothing useful initially, the residual connection ensures that the identity function (input = output) is preserved, so the model can build on already-learned features without degradation.

Ways residual networks help

4. **Encourages Modular Learning:**
• Incremental learning benefits from modularity, where individual components of a system can refine different aspects over time.
• Residual blocks function as modular units that **independently refine or improve features** in isolation without disrupting previously learned representations.

5. **Resilience to Overfitting and Better Generalization:**
• Since residual blocks can "skip" learning unnecessary transformations when the data does not require them, the network is less likely to overfit to noise.
• The model effectively learns the **necessary corrections** to existing features rather than over-complicating its transformations.

6. **Supports Smooth Gradient Flow in Transfer Learning:**
• In incremental learning tasks, such as transfer learning or continual learning, pre-trained networks often need to adapt to new tasks gradually.
• Residual connections make it easier to **adapt existing representations incrementally** because the network doesn't discard previously learned features—it refines them through residual updates.

The Feed Forward Network in Transformers

In the Transformer algorithm, **Feed-Forward Networks (FFNs)** are used within each layer to apply **non-linear transformations** to the data, helping the model learn complex relationships. They are crucial for processing the output from the **self-attention mechanism**.

1. **Transformation and Feature Extraction:**

• The FFN takes the output of the self-attention block and applies a series of **non-linear transformations** to extract richer representations.

• This helps the model capture intricate patterns in the input data beyond what the attention mechanism identifies.

2. **Structure:**

• Each FFN consists of two linear layers with a non-linear activation function (usually **ReLU or GELU**) in between:

$$\mathrm{FFN}(x) = \mathrm{ReLU}(xW_1 + b_1)W_2 + b_2$$

- W_1 and W_2 are learned weight matrices, and b_1, b_2 are biases.

3. **Independence Across Positions:**

• Unlike self-attention, the FFN operates **independently on each position** (token) in the sequence. It applies the same transformation across all tokens in parallel.

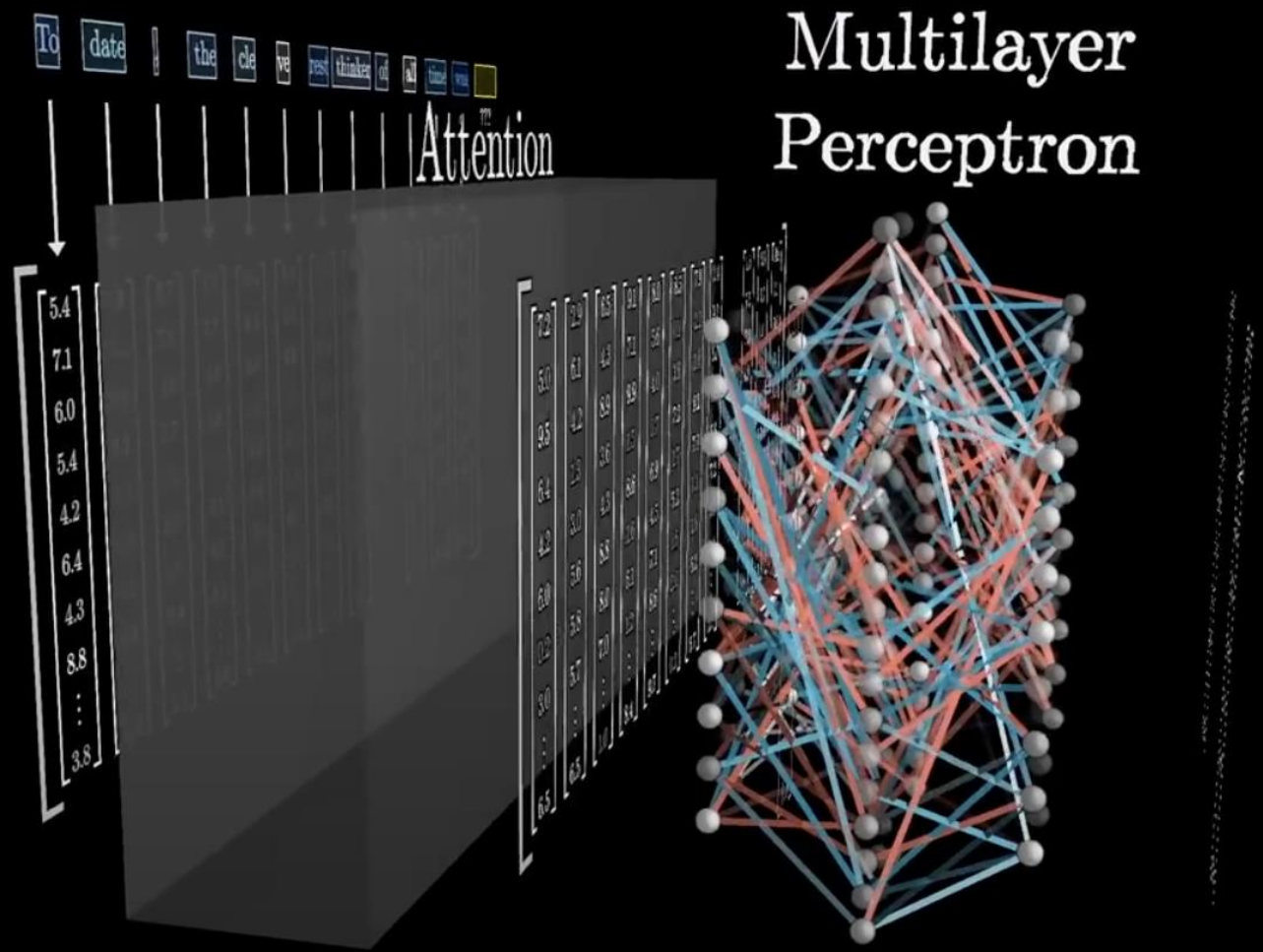
The Feed Forward Networks in Transformers

4. **Adding Depth to Learning:**
• FFNs enable deeper feature extraction within each Transformer layer, complementing the **contextual dependencies** learned by attention.

5. **Contribution to Residual Learning:**
• The FFN output is added to the input of the block via **residual connections**, helping the model **incrementally learn** and prevent gradient issues.

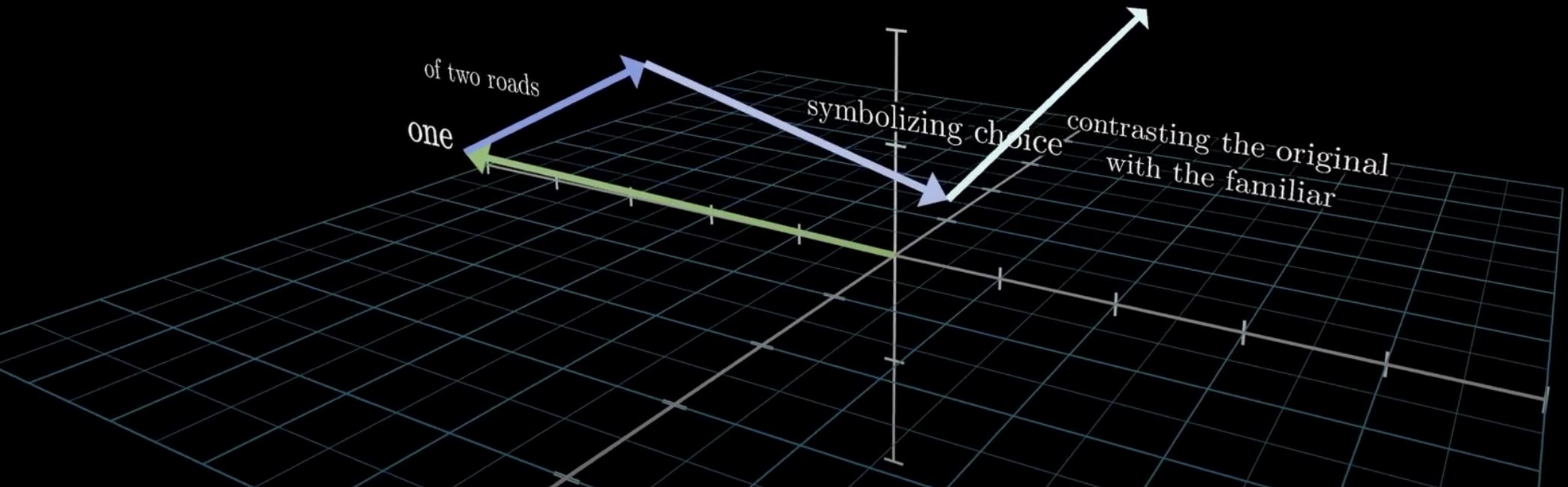
About 1/3 of What

Attention is ~~All~~ You Need

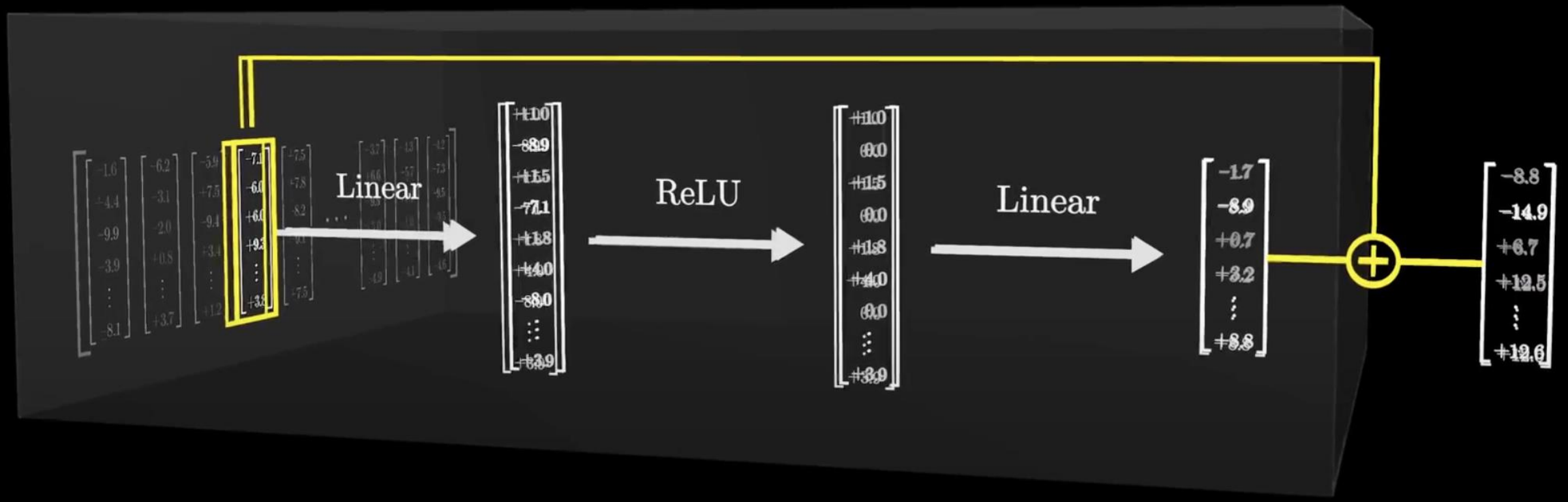

To
date
the
thinker
of
Attention
Multilayer
Perceptron

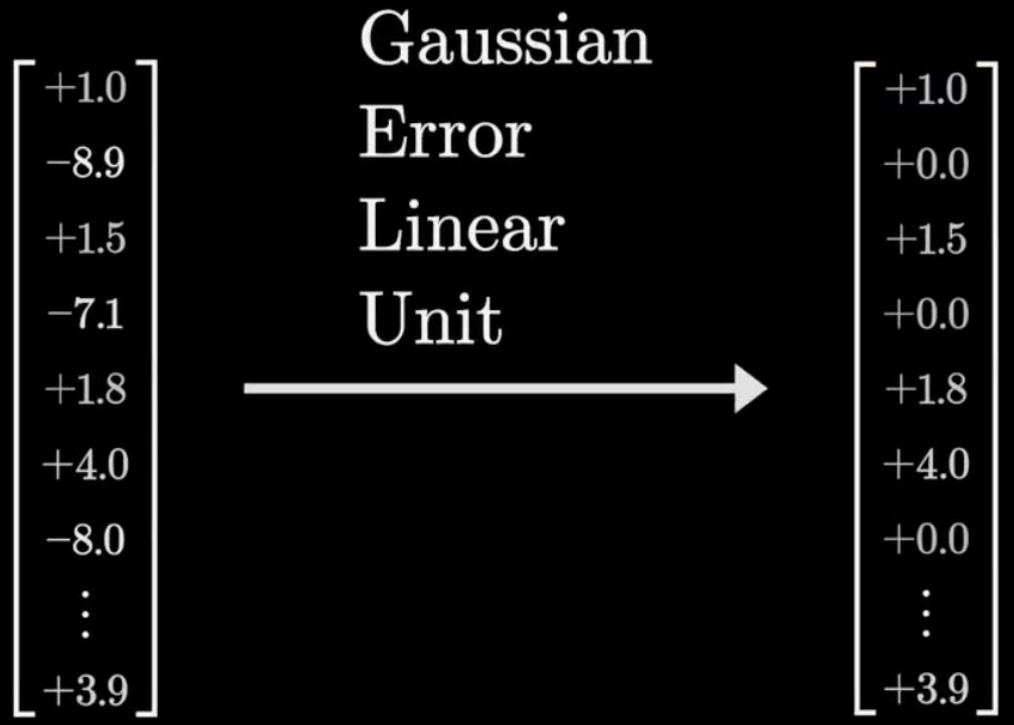
⋮

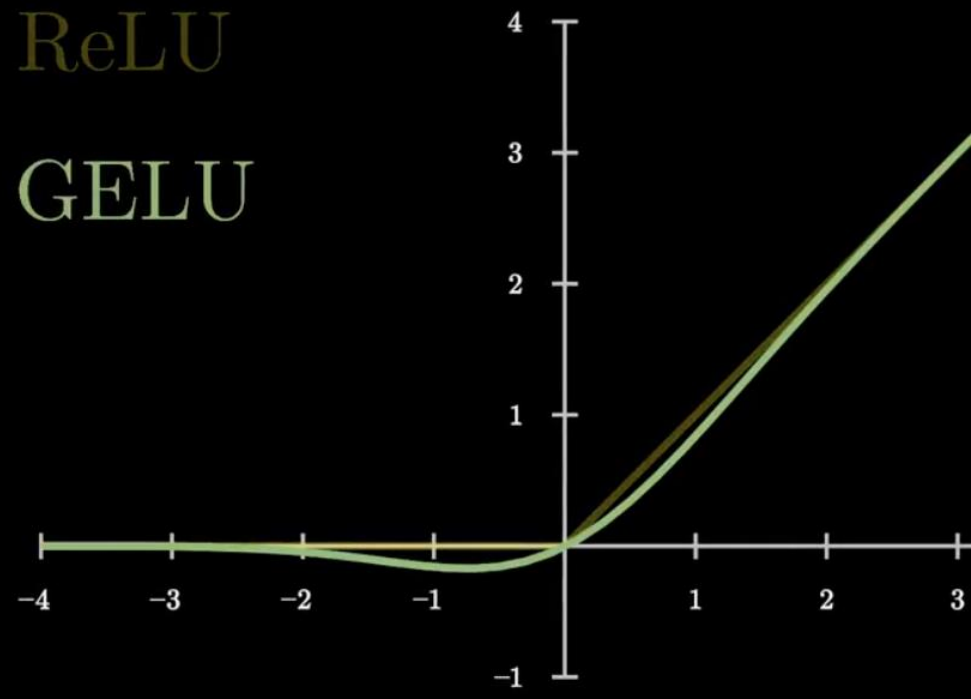
Two roads diverged in a wood, and I—
I took the one less traveled by,

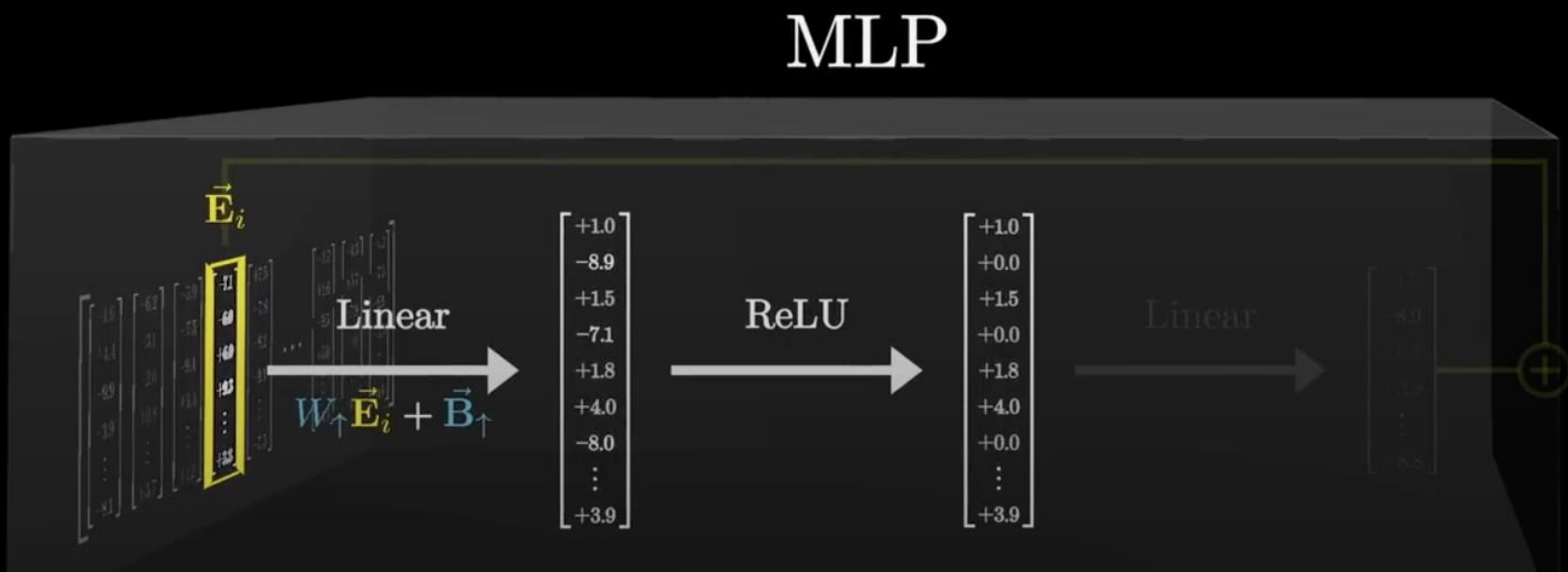


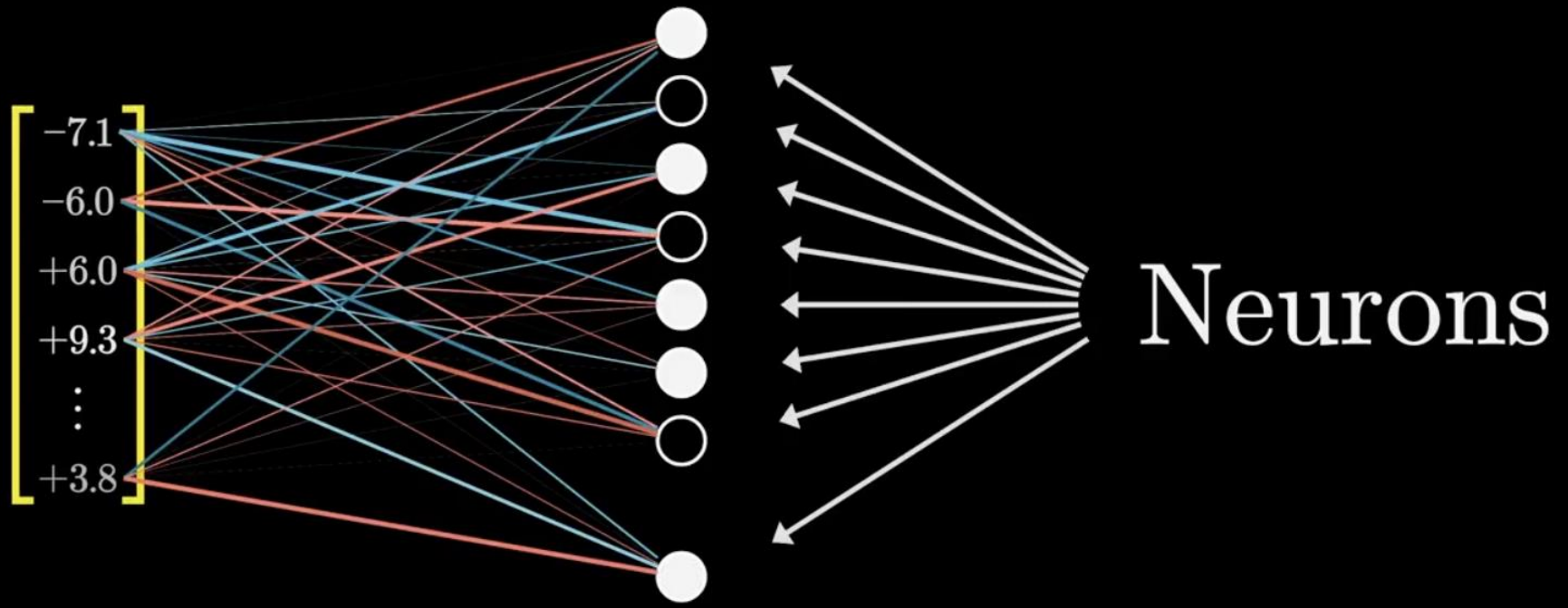
MLP

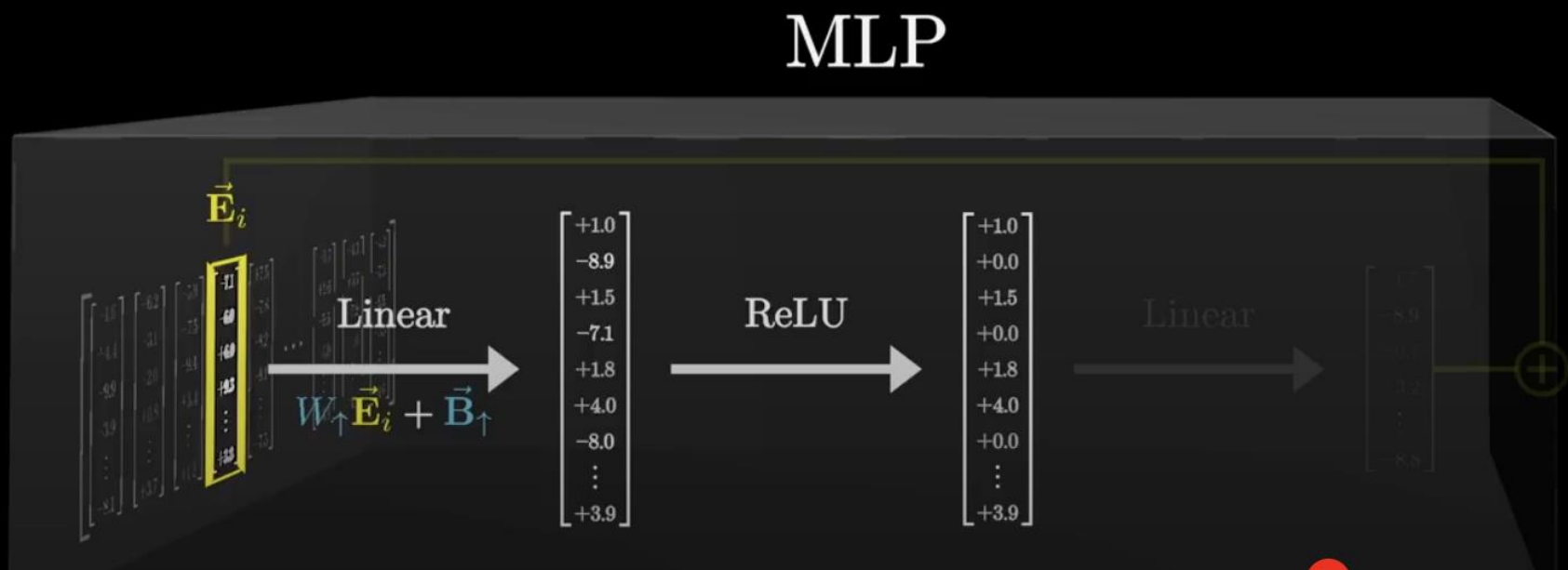


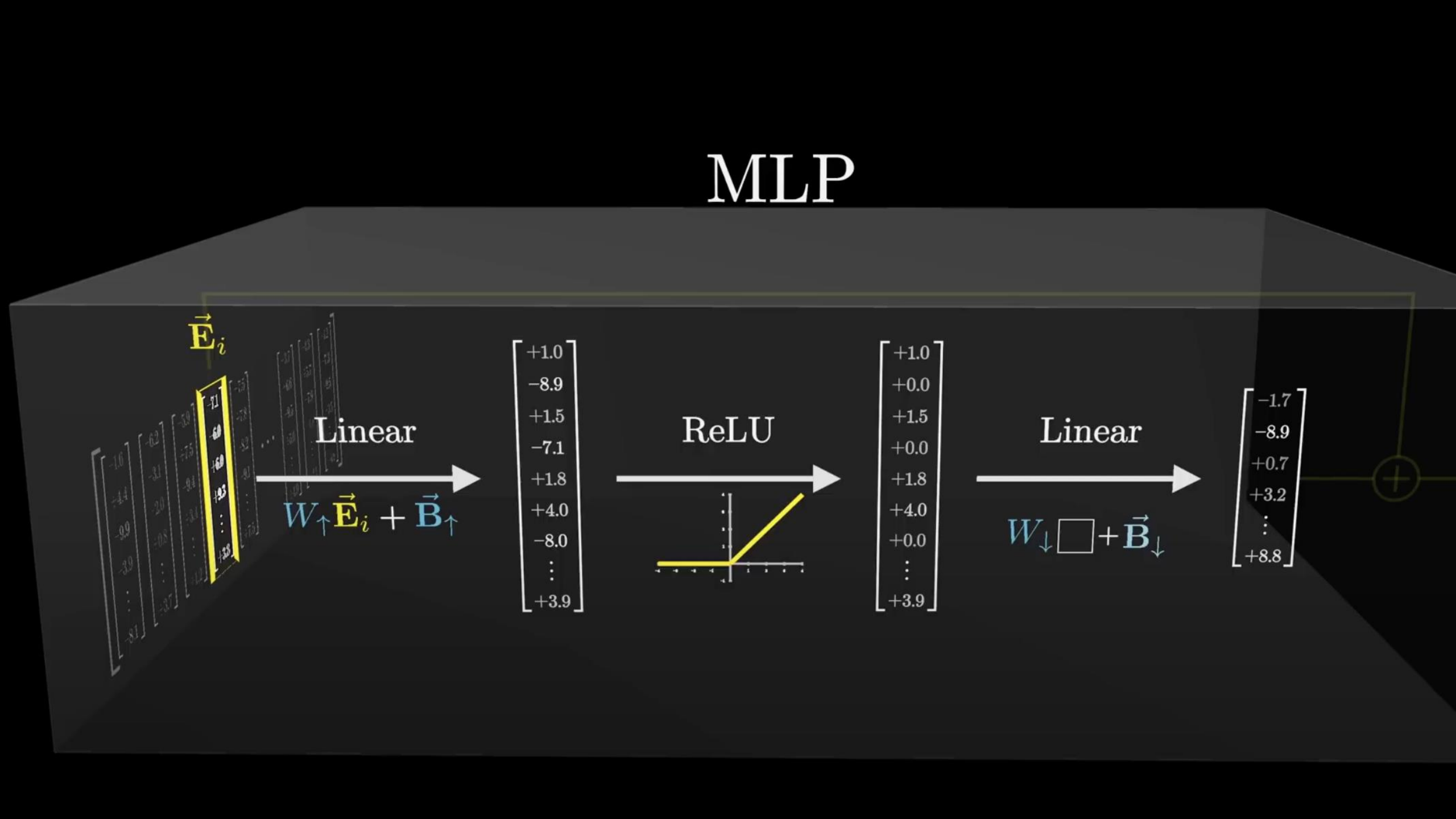

+1.0
−8.9
+1.5
−7.1
+1.8
+4.0
−8.0
+3.9
Gaussian
Error
Linear
Unit
+1.0
+0.0
+1.5
+0.0
+1.8
+4.0
+0.0
+3.9

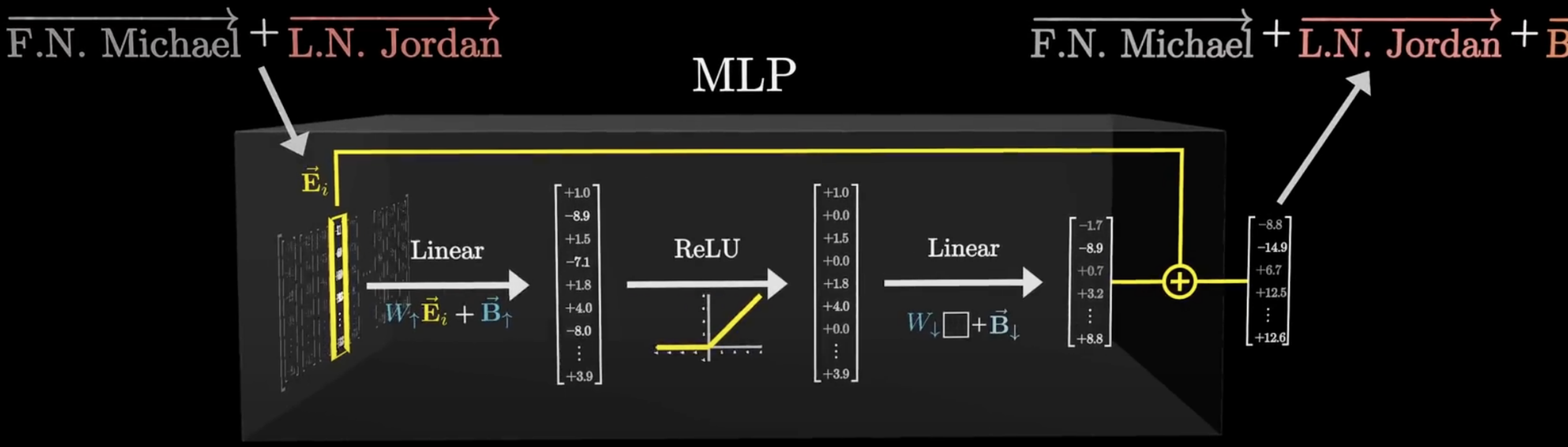

ReLU
GELU
4
3
2
1
−1
−4
−3
−2
−1
1
2
3

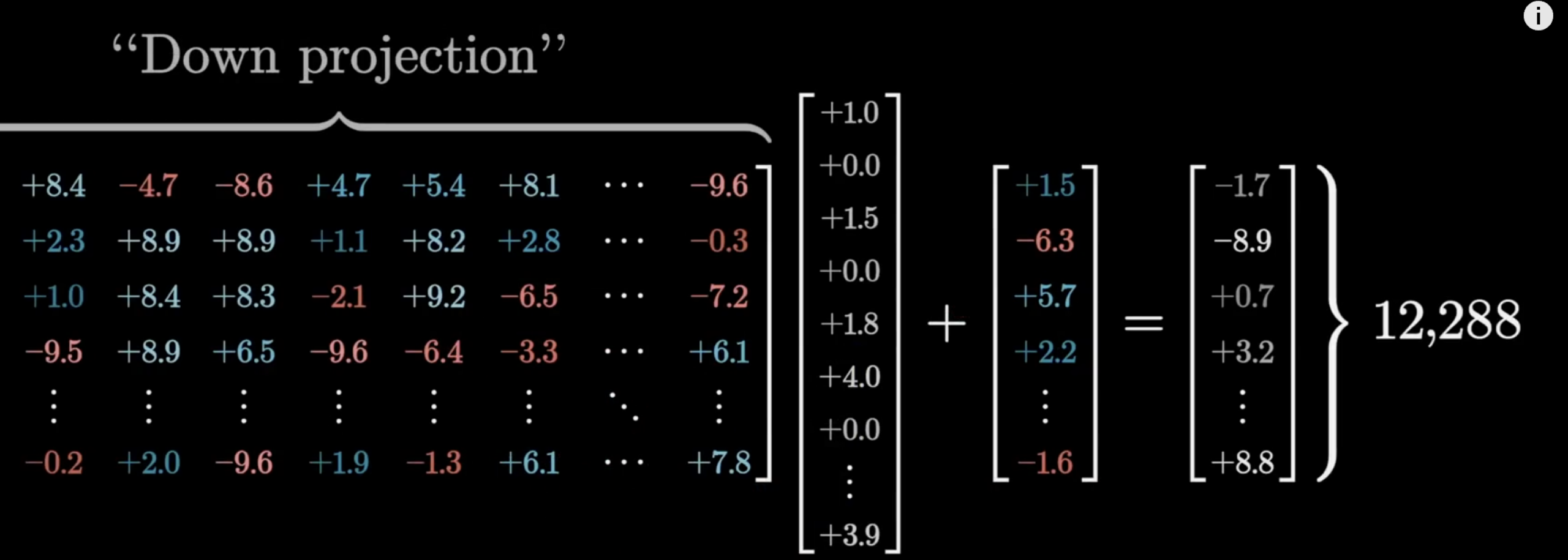

MLP
$\vec{E}_i$
Linear
$W_\uparrow \vec{E}_i + \vec{B}_\uparrow$
+1.0
−8.9
+1.5
−7.1
+1.8
+4.0
−8.0
+3.9
ReLU
+1.0
+0.0
+1.5
+0.0
+1.8
+4.0
+0.0
+3.9
Linear


−7.1
−6.0
+6.0
+9.3
+3.8
Neurons


MLP
$\vec{E}_i$
Linear
$W_\uparrow \vec{E}_i + \vec{B}_\uparrow$
+1.0
−8.9
+1.5
−7.1
+1.8
+4.0
−8.0
+3.9
ReLU
+1.0
+0.0
+1.5
+0.0
+1.8
+4.0
+0.0
+3.9
Linear


MLP
$\vec{\mathbf{E}}_i$
Linear
$W_\uparrow \vec{\mathbf{E}}_i + \vec{\mathbf{B}}_\uparrow$
+1.0
−8.9
+1.5
−7.1
+1.8
+4.0
−8.0
+3.9
ReLU
+1.0
+0.0
+1.5
+0.0
+1.8
+4.0
+0.0
+3.9
Linear
$W_\downarrow \square + \vec{\mathbf{B}}_\downarrow$
−1.7
−8.9
+0.7
+3.2
+8.8


F.N. Michael + L.N. Jordan
MLP
$\vec{E}_i$
Linear
$W_\uparrow \vec{E}_i + \vec{B}_\uparrow$
+1.0
−8.9
+1.5
−7.1
+1.8
+4.0
−8.0
+3.9
ReLU
+1.0
+0.0
+1.5
+0.0
+1.8
+4.0
+0.0
+3.9
Linear
$W_\downarrow \square + \vec{B}_\downarrow$
−1.7
−8.9
+0.7
+3.2
+8.8
−8.8
−14.9
+6.7
+12.5
+12.6
F.N. Michael + L.N. Jordan +



rojection	$\overset{49{,}152}{\text{n_neurons}} * \overset{12{,}288}{\text{d_embed}} = 603{,}979{,}776$ per layer
n-projection	$\overset{12{,}288}{\text{d_embed}} * \overset{49{,}152}{\text{n_neurons}} = 603{,}979{,}776$ per layer
mbedding	$\overset{50{,}257}{\text{n_vocab}} * \overset{12{,}288}{\text{d_embed}} = 617{,}558{,}016$

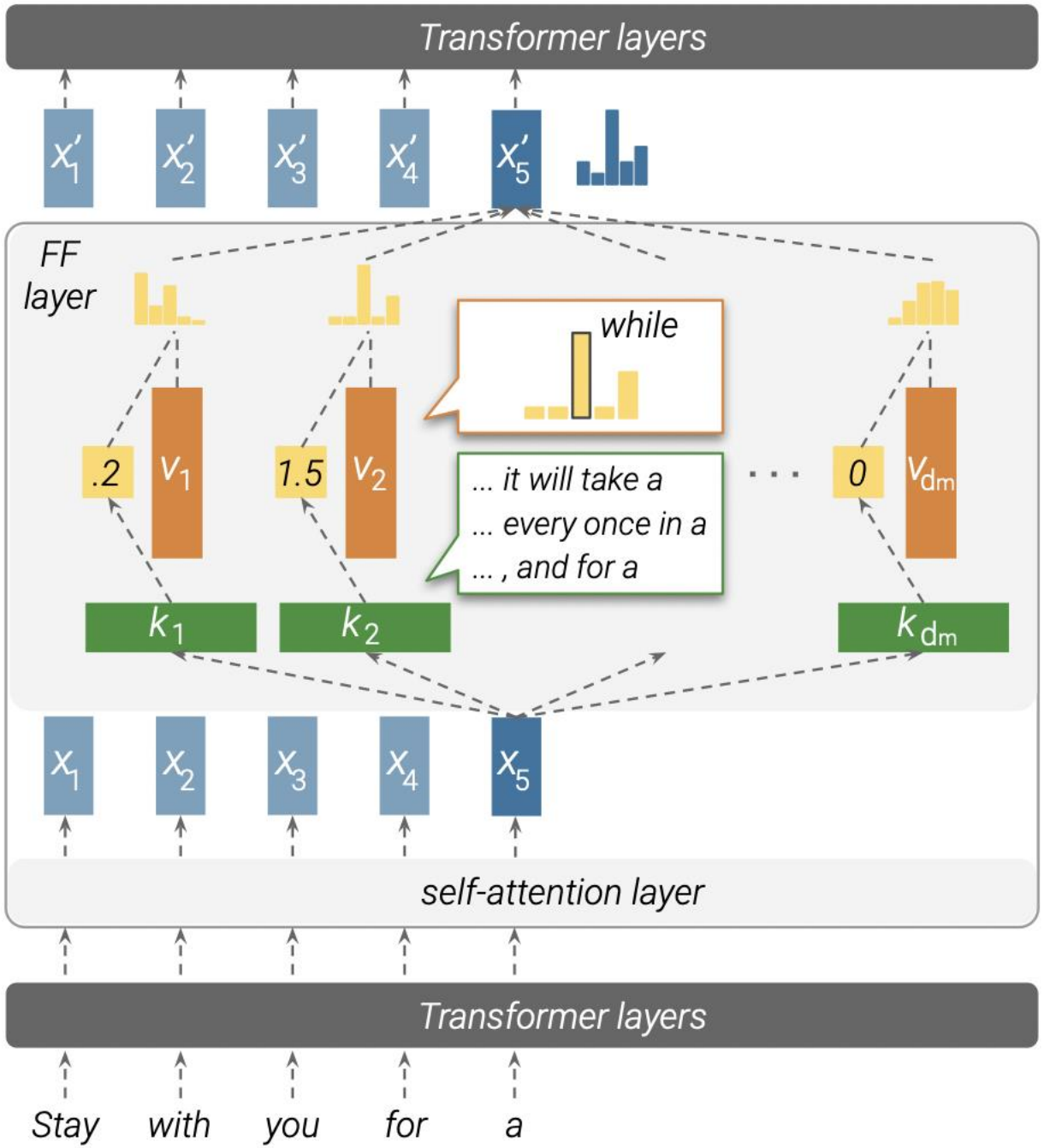
GPT-3

Total weights:

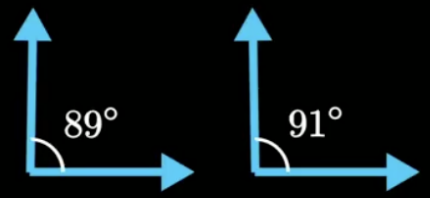
175,181,291,520

Embedding	$\overset{12,288}{\text{d_embed}} * \overset{50,257}{\text{n_vocab}}$	$= 617,558,016$
Key	$\overset{128}{\text{d_query}} * \overset{12,288}{\text{d_embed}} * \overset{96}{\text{n_heads}} * \overset{96}{\text{n_layers}}$	$= 14,495,514,624$
Query	$\overset{128}{\text{d_query}} * \overset{12,288}{\text{d_embed}} * \overset{96}{\text{n_heads}} * \overset{96}{\text{n_layers}}$	$= 14,495,514,624$
Value	$\overset{128}{\text{d_value}} * \overset{12,288}{\text{d_embed}} * \overset{96}{\text{n_heads}} * \overset{96}{\text{n_layers}}$	$= 14,495,514,624$
Output	$\overset{12,288}{\text{d_embed}} * \overset{128}{\text{d_value}} * \overset{96}{\text{n_heads}} * \overset{96}{\text{n_layers}}$	$= 14,495,514,624$
Up-projection	$\overset{49,152}{\text{n_neurons}} * \overset{12,288}{\text{d_embed}} * \overset{96}{\text{n_layers}}$	$= 57,982,058,496$
Down-projection	$\overset{12,288}{\text{d_embed}} * \overset{49,152}{\text{n_neurons}} * \overset{96}{\text{n_layers}}$	$= 57,982,058,496$
Unembedding	$\overset{50,257}{\text{n_vocab}} * \overset{12,288}{\text{d_embed}}$	$= 617,558,016$

Feed-forward layers constitute two-thirds of a transformer model's parameters, yet their role in the network remains under-explored. We show that feed-forward layers in transformer- based language models operate as key-value memories, where each key correlates with textual patterns in the training examples, and each value induces a distribution over the output vocabulary. Our experiments show that the learned patterns are human-interpretable, and that lower layers tend to capture shallow patterns, while upper layers learn more semantic ones. The values complement the keys' input patterns by inducing output distributions that concentrate probability mass on tokens likely to appear immediately after each pattern, particularly in the upper layers. Finally, we demonstrate that the output of a feed-forward layer is a composition of its memories, which is subsequently refined throughout the model's layers via residual connections to produce the final output distribution.



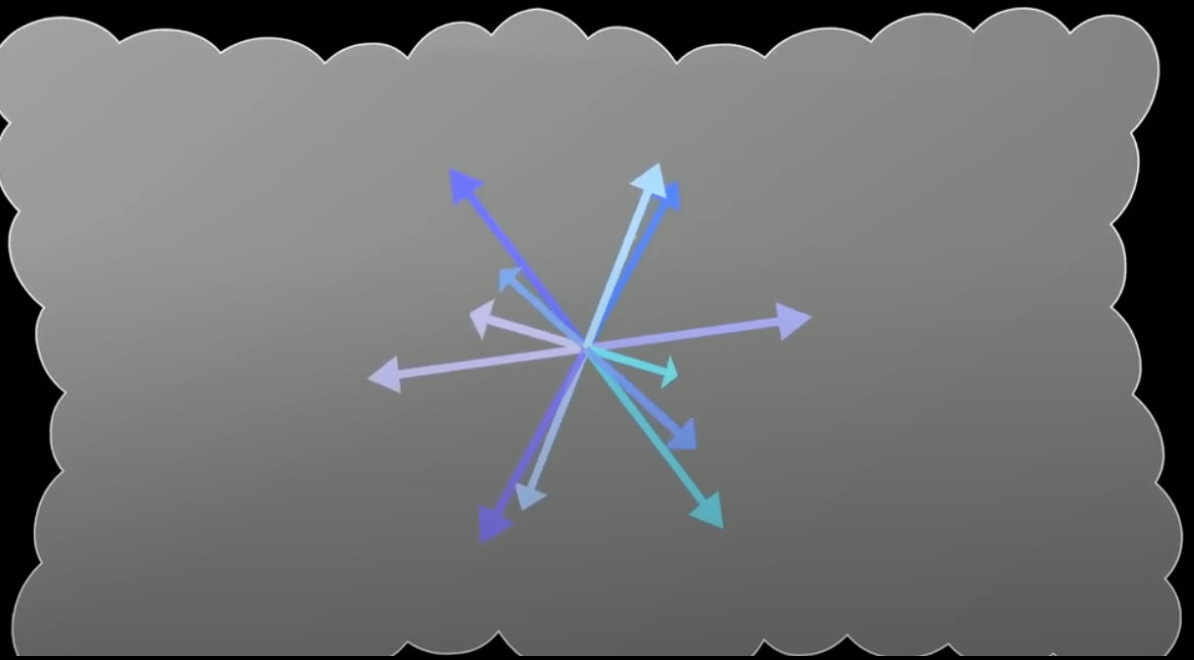
The lemma likely explains how large language models (LLMs) like transformers are able to represent highly nuanced word meanings and context in relatively low-dimension embeddings.[2]



Choose multiple vectors,
each pair between 89° and 91° apart

Johnson–Lindenstrauss Lemma $\Rightarrow$ Maximum # of vectors: $\approx \exp(\epsilon \cdot N)$

N-dimensional Space



The **Johnson–Lindenstrauss (JL) Lemma** is a foundational result in high-dimensional geometry that explains *how complex, high-dimensional relationships can be preserved under surprisingly strong dimensionality reduction*.

🧭 The Lemma Itself

Statement (informal):
Given any set of n points in a high-dimensional space $\mathbb{R}^d$, and any small error tolerance $\varepsilon \in (0,1)$, there exists a linear map
$f : \mathbb{R}^d \to \mathbb{R}^k$
with $k = O\left(\frac{\log n}{\varepsilon^2}\right)$
such that for all pairs of points x_i, x_j,
$(1-\varepsilon)\|x_i - x_j\|^2 \leq \|f(x_i) - f(x_j)\|^2 \leq (1+\varepsilon)\|x_i - x_j\|^2.$

In other words:
you can randomly project very high-dimensional data into a space of dimension only logarithmic in the number of samples — *without distorting pairwise distances by more than a small factor*.

🧠 How It Relates to LLMs and Word Embeddings

LLMs (Large Language Models) rely on *embeddings* — vector representations of tokens or words in high-dimensional spaces (often 768–8192 D).
The **semantic relationships** among words (similarity, analogy, nuance) are expressed geometrically as distances and directions in this embedding space.

The JL lemma gives a **mathematical explanation** for why these relatively compact embeddings can capture the nuances of meaning in the vast combinatorial space of language:

1. **Preservation of Semantic Structure:**

Even though the "true" semantic space of language may be enormously high-dimensional (reflecting culture, context, syntax, pragmatics, etc.), the lemma assures us that a *random or learned projection* into a much smaller space can *approximately preserve relative distances*.

→ Word pairs like ("king", "queen") and ("man", "woman") retain consistent geometric relationships even in 1–2 k dimensions.

2. **Feasibility of Compression:**

The number of distinct word-meaning relationships that matter in practice (the "n" points) is vastly smaller than the total possible combinations of meanings.

By JL, you only need dimension

$k \sim \frac{\log n}{\varepsilon^2}$.

For example, if $n = 10^6$ meaningful tokens and $\varepsilon = 0.1$,

then $k \approx 10^3$—about the embedding sizes used in GPT-like models.

3. **Robustness of Linear Projections:**

Transformer layers mix representations linearly (plus nonlinear gating).

Because linear projections approximately preserve geometry (as JL guarantees for random ones), the model can safely compress and recombine information across layers without catastrophic loss of meaning.

4. **Distributed Semantics:**

The lemma supports the intuition that *semantic meaning doesn't need explicit symbolic coordinates*.

Nuances can be represented as *distributed directions* in a continuous space — distances and angles encode relationships robustly under random or learned linear transformations.
